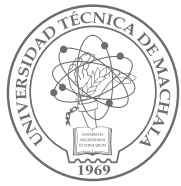
Fundamentos de bases de datos

Zea Ordoñez Mariuxi, Honores Tapia Joofre, Rivas Asanza Wilmer



Universidad Técnica de Machala

Fundamentos de base de datos



Ing. César Quezada Abad, MBA
Rector

Ing. Amarilis Borja Herrera, Mg. Sc.
Vicerrectora Académica

Soc. Ramiro Ordóñez Morejón, Mg. Sc.
Vicerrector Administrativo

COORDINACIÓN EDITORIAL
VICERRECTORADO ACADÉMICO

Tomás Fontaines-Ruiz, PhD.
Investigador Becario Prometeo-Utmach
Asesor Del Programa De Reingeniería

Ing. Karina Lozano Zambrano
Coordinadora Editorial

Ing. Jorge Maza Córdova, Ms.
Ing. Cyndi Aguilar
Equipo de Publicaciones

Fundamentos de base de datos

Mariuxi Zea Ordoñez
Joofre Honores Tapia
Wilmer Rivas Asanza

UNIVERSIDAD TÉCNICA DE MACHALA
2015

Primera edición 2015

ISBN: 978-9942-24-074-3

D.R. © 2015, universidad técnica de machala
Ediciones utmach
Km. 5 1/2 Vía Machala Pasaje
www.utmachala.edu.ec

Este texto ha sido sometido a un proceso de evaluación por pares externos con base en la normativa editorial de la utmach.

Portada:
Concepto editorial
Samanta Cabezas (est. Comunicación Social)
Fotografia: Dir. de Comunicación UTMACH

Diseño, montaje y producción editorial: UTMACH

Impreso y hecho en Ecuador
Printed and made in Ecuador

Advertencia: “Se prohíbe la reproducción, el registro o la transmisión parcial o total de esta obra por cualquier sistema de recuperación de información, sea mecánico, fotoquímico, electrónico, magnético, electroóptico, por fotocopia o cualquier otro, existente o por existir, sin el permiso previo por escrito del titular de los derechos correspondientes”.

Índice

PREFACIO 17
PRÓLOGO 19
DEDICATORIA 21
INTRODUCCIÓN 23
1. BASE DE DATOS RELACIONALES 25
1.1. Generalidades de bases de datos 26
1.2. Ejemplificación 27
1.3. Concepto de modelos de datos. Funciones y sublenguajes 29
1.3.1. Diseño de los sistemas de bases de datos 31
1.3.2. Funciones de un SGBD 31
1.4. Clasificación de los varios tipos de diseño de datos de acuerdo al nivel de abstracción 33
1.4.1. Modelos de datos conceptuales 33
1.4.2. Modelos de datos lógicos 35
1.4.3. Modelos de datos físicos 37
1.5. Enumeración de las reglas de Codd para un sistema relacional 38
2. DESCRIPCIÓN Y APLICACIÓN DEL MODELO ENTIDAD-RELACIÓN PARA EL MODELADO DE DATOS 43
2.1. Proceso de realización de modelo entidad - relación 44
2.2. Elementos 44
2.2.1. Entidad 44
2.2.2. Relación 45

2.2.3. Atributo y dominio 47
2.3. Diagramas entidad-relación extendidos como elementos para resolver las carencias de los diagramas entidad-relación simples. 49
2.3.1. Cardinalidad de las relaciones. 49
2.3.2. Entidades fuertes y frágiles 50
2.3.3. Atributos en relaciones 52
2.3.4. Herencia 52
2.3.5. Agregación 54
2.4. Diseño lógico a partir de diagramas entidad - relación .. 54
2.4.1. Reglas concernientes al modelo entidad - relación básico 55
2.4.2. Reglas concernientes al modelo entidad – relación extendido 60
2.5. Desarrollo de diversos supuestos prácticos de modelización mediante diagramas Entidad-Relación 63
3. ANÁLISIS DEL MODELO RELACIONAL Y DE LOS ELEMENTOS QUE LO INTEGRAN 67
3.1. Concepto de relaciones y sus propiedades 68
3.2. Concepto de claves en el modelo relacional 70
3.3. Las restricciones en el modelo relacional 73
3.3.1. Restricciones inherentes 73
3.3.2. Restricciones semánticas o de usuario 73
3.4. Nociones de álgebra relacional 76
3.4.1. Operadores tradicionales de la teoría de conjuntos 76
3.4.2 Operadores relacionales específicos 79
3.5. Nociones de Cálculo relacional de filas para resolver ejercicios prácticos esenciales 83
3.6. Nociones de Cálculo relacional de Dominios 87
2.7. Teoría de la normalización y sus objetos 88
3.7.1. Concepto de dependencias funcionales 92
3.7.2. Análisis y aplicación de las diferentes formas normales: forma normal y la forma normal de Boyce/Codd 98
3.7.3 Ventajas e inconvenientes que justifican una desnormalización de las tablas. Valoración en diferentes supuestos prácticos. 107

3.7.4. Desarrollo de diferentes supuestos prácticos de normalización de datos incluyendo propuestas de desnormalización de datos 114
4. MODELO ORIENTADO A OBJETOS 117
4.1. Contextualización del modelo orientado a objeto dentro del modelado UML..... 117
4.2. Modelo de datos orientado a objetos 120
4.2.1. Conceptos de objeto, clase, método y mensaje 120
4.2.2. Representación de clases en UML..... 121
4.2.3. Relaciones entre clases en UML..... 123
4.3. Comparación del modelo de clases con el modelo Entidad-Relación 130
EJERCICIOS PROPUESTOS Y RESUELTOS 135

Índice de figuras

Figura. 1: Relación empleados con sus campos y registros. 28
Figura. 2: Esquema entidad relación de una biblioteca..... 34
Figura. 3: Diagrama de clases para una biblioteca..... 35
Figura. 4: Esquema de una base de datos jerárquica..... 36
Figura. 5: Esquema de una base de datos en red..... 37
Figura. 6: Diseño de una base de datos relacional con datos..... 37
Figura. 7: Representación gráfica de una entidad 44
Figura. 8: Representación gráfica de una relación 45
Figura. 9: Relación reflexiva 45
Figura. 10: Relación ternaria..... 45
Figura. 11: Relación 1:1 binaria 46
Figura. 12: Relación 1:1 reflexiva. 46
Figura. 13: Relación 1:N binaria..... 47
Figura. 14: Relación 1:N reflexiva 47
Figura. 15: Relación N:M binaria. 47
Figura. 16: Relación N:M reflexiva 48
Figura. 17: Representación de atributos 48
Figura. 18: Representación de atributos alternativa..... 48
Figura. 19: Representación de cardinalidades en una relación binaria N:M..... 44
Figura. 20: Representación de cardinalidades en una relación reflexiva N:M..... 49
Figura. 21: Representación de cardinalidades en una relación ternaria 50

Figura. 22: Representación gráfica de una entidad frágil. ... 50
Figura. 23: Dependencia en identificación. 51
Figura. 24: Dependencia en existencia 51
Figura. 25: Relación N:M con atributo. 52
Figura. 26: Relación 1:1 con atributo 52
Figura. 27: Representación de Herencia 53
Figura. 28: Jerarquía de tipos subtipos total y exclusiva 53
Figura. 29: Representación de agregación..... 54
Figura. 30: Esquema ejemplo de ER..... 55
Figura. 31: Esquema ER con relación reflexiva N:M..... 56
Figura. 32: Esquema ER con relación 1:N..... 58
Figura. 33: Esquema ER con relación 1:N..... 58
Figura. 34: Esquema ER con relación 1:1. 59
Figura. 35: Esquema ER con relación ternaria M:N:P 60
Figura. 36: Esquema ER con una dependencia en existencia..... 60
Figura. 37: Esquema ER con una jerarquía de tipos y subtipos..... 62
Figura. 38: Esquema relacional..... 73
Figura. 39: Diseño de modelo relacional 91
Figura. 40: Representación de las formas normales..... 92
Figura. 41: Diagrama de dependencias funcionales 93
Figura. 42: Diagrama de dependencias funcionales 96
Figura. 43: Comprobación de dependencia de reunión..... 98
Figura. 44: Diseño Relacional..... 99
Figura. 45: Diagrama de dependencias funcionales 102
Figura. 46: Esquema relacional..... 103
Figura. 47: Relación de uno a uno (1:1) 109
Figura. 48: Relación Instituto..... 109
Figura. 49: Relación de uno a muchos (1:n)..... 109
Figura. 50: Tabla Empleados..... 110
Figura. 51: Relación de muchos a muchos 110
Figura. 52: Partición vertical 111
Figura. 53: Partición horizontal..... 112
Figura. 54: Relación Universidad..... 113
Figura. 55: Relación en primera forma Normal 113

Figura. 56: Diagrama de Estructuras. 118
Figura. 57: Dos representaciones de la clase libro. 123
Figura. 58: Asociación binaria sin nombre..... 124
Figura. 59: Asociación reflexiva con nombres y con valores. 124
Figura. 60. Asociación Ternaria..... 125
Figura. 61: Clase Asociativa. 125
Figura. 62. Agregación 126
Figura. 63: Composición. 127
Figura. 64: Generalización. 127
Figura. 65: Generalización de dos niveles. 128
Figura. 66: Especialización disjunta, total y dinámica 129
Figura. 67: Herencia Múltiple. 129
Figura. 68: Relación de Dependencia..... 129
Figura. 69: Objeto de la clase cliente 131
Figura. 70: Representación detallada de un objeto de la clase cliente 131
Figura. 71: Diagrama de objetos. 132

Índice de tablas

Tabla 1: Definición matemática de relacion 68
Tabla 2: Relación Artículos..... 69
Tabla 3: Relación Pedidos..... 71
Tabla 4: Tabla Linea_pedidos 72
Tabla 5: Esquema relacional con datos..... 74
Tabla 6: Ejemplo de relaciones para el álgebra relacional 76
Tabla 7: Unión de dos relaciones..... 76
Tabla 8: Intersección de dos relaciones..... 76
Tabla 9: Diferencia entre dos relaciones 76
Tabla 10: Relaciones para ejemplos posteriores 78
Tabla 11: Producto cartesiano de dos relaciones..... 78
Tabla 12: Selección sobre una relación..... 79
Tabla 13: Proyección sobre una relación..... 79
Tabla 14: Selección y proyección sobre una relación..... 79
Tabla 15: Relaciones a utilizar en la división 80
Tabla 16: División de las relaciones..... 80
Tabla 17: Relaciones de ejemplo para la combinación..... 81
Tabla 18: Producto cartesiano..... 81
Tabla 19: Combinación o reunión..... 82
Tabla 20: Combinación Natural..... 83
Tabla 21: Ejemplo de relaciones para el cálculo relacional de filas:..... 83
Tabla 22: Ejemplo de relaciones para el cálculo relacional de fila..... 84
Tabla 23: Relación Pedidos..... 90

Tabla 24: Reacio Libro..... 94
Tabla 25: Relación Coche 95
Tabla 26 : Relación Con Dependencias Multivaluadas 97
Tabla 27: Relación Alumnos 100
Tabla 28: Relación Alumnos 100
Tabla 29: Relación Alumnos 100
Tabla 30: Relación Postal 105
Tabla 31: Relaciones en Cuarta Forma normal 106
Tabla 32: Datos de un Directorio..... 135
Tabla 33: Código de Provincia 135
Tabla 34 Registro de Directorio 136
Tabla 35: Nombre - Fecha de un Directorio 136
Tabla 36: Tabla Ciudad 136
Tabla 37: Datos de un Encargo 137
Tabla 38: Tabla pedido..... 1146
Tabla 39: Tabla línea de pedido..... 147

Prefacio

En la actualidad, se considera a las bases de datos como un elemento indispensable en las empresas, instituciones u organizaciones que utilizan sistemas computarizados, ya que éstas permiten operar datos de manera organizada, segura, confiable y en grandes cantidades.

Cada vez se necesita más recursos en tecnología, es por ellos que como avanzan las generaciones de las computadoras, las bases de datos de igual manera avanzan en conjunto con los leguajes de programación.

Resulta impresionante poder visualizar como se procesa los datos en microsegundos, mientras se realizan otras transacciones, al mismo tiempo en la misma base de datos en lugares y estados diferentes; definitivamente la tecnología avanza a pasos agigantados es por eso que debemos arriesgarnos con el pasar de los años a las nuevas tecnologías que harán más fácil ciertas acciones como la administración y seguridad de la información.

Existen dos razones por las cuales se ha realizado el presente libro, la primera es dar a conocer los acontecimientos, por los cuales surgen las bases de datos relacionales, el sistema gestor de base de datos, Aplicación del Modelo Entidad-Relación para el modelado de datos y modelos orientados a objetos.

Prólogo

Hoy en día las bases de datos han tomado gran importancia mundial, es totalmente indispensable para las organizaciones puesto que brinda mucha facilidad en el acceso a la información y una gran capacidad de almacenaje de datos.

Es por este motivo que resulta muy grato dar a conocer este libro que se titula "FUNDAMENTOS DE BASES DE DATOS", el mismo que ha sido creado para orientar a cualquier persona: desde un estudiante en sus primeros pasos en el aprendizaje del mundo de la base de datos, hasta un profesional ávido de nuevos conocimientos.

Con este libro se pretende que el lector pueda despejar toda clase de interrogantes acerca de bases de datos y le permita familiarizarse rápidamente con su contenido, no solo teórico sino también práctico mediante una serie de ejercicios.

La estructura de este libro está sujeta a un sinnúmero de temas, los mismos que se encuentran organizados en 4 capítulos, cuyo contenido se presenta de manera clara y precisa. Cada capítulo es sumamente importante en el proceso de aprendizaje de las bases de datos.

Los temas de este libro versan sobre las bases de datos relacionales y el modelado de datos, cada uno con definiciones claras y entendibles, así mismo reglas de transformación totalmente necesarias para crear un modelado relacional y además el modelo orientado a objetos.

El objetivo del libro es lograr que cada lector enriquezca sus conocimientos con los temas plasmados en cada capítulo, y servir como ayuda didáctica a cientos de docentes en el área de bases de datos.

Dedicatoria

Dedico la realización de este libro a mis padres por el apoyo que me han brindado a lo largo de toda mi vida ya que con su ayuda y consejos han sabido guiarme para ser una persona de bien tanto en el aspecto humano como en el profesional. Ellos han sido mi ejemplo a seguir para lograr ser una persona correcta, honesta y con objetivos claros

Introducción

Con base a las investigaciones se ha comprobado que en la actualidad el usuario tiene un acceso a la información más rápido que en el siglo XXIII. Además, solo los ordenadores poseen la capacidad de almacenar, gestionar y trasmitir una amplia gama de datos. Los mismos que pueden ser manipulados en momento por usuarios autorizados, ya sea para realizar una inserción, modificación, eliminación o búsqueda.

El libro que se presenta a continuación pretende fortalecer los conocimientos de los lectores empezando por conceptos básicos sobre bases de datos relacionales, gestores de bases de datos, diseñar un diagrama entidad – relación hasta llegar a una serie de ejercidos propuestos sobre cada uno de los temas.

Se divide en cuatro capítulos, los cuales están distribuidos de la siguiente manera:

Capítulo 1: Este capítulo trata sobre los conceptos de base de datos relaciones, ejemplos, modelos de datos, funciones y sublenguajes, clasificación de los modelos y las reglas de Codd.

Capítulo 2: Este capítulo trata sobre el análisis del modelo relacional y de los elementos que lo integran, conceptos de relaciones, claves en el modelo relacional, restricciones, nociones de algebra relacional, nociones de cálculo de tuplas para poder resolver ejercicios prácticos básicos, nociones de cálculo relacional de dominios y teoría de la normalización y sus objetivos.

Capítulo 3: En este capítulo se hablará sobre la Descripción y Aplicación del Modelo Entidad-Relación para el modelado de datos, el proceso de realización de diagramas Entidad-Relación, elementos, diagramas Entidad-Relación extendidos, diseño lógico a partir de dia-

gramas Entidad-Relación y desarrollo de diversos supuestos prácticos de modelización mediante diagramas Entidad-Relación.

Capítulo 4: Este capítulo trata sobre la contextualización del Modelo orientado a objetos dentro del modelado UML, modelo de datos orientado a objetos, comparaciones del modelo de clases con el modelo Entidad- Relación, diagrama de objetos como caso especial del diagrama de clases, modelo Distribuido y los enfoques para realizar el diseño. Además, consta de una sección únicamente de ejercicios propuestos que ayudaran al lector a enriquecer de manera práctica los conocimientos adquiridos.

Base de datos relacionales

Descripción del capítulo

A continuación, se presentará los objetivos principales de cada apartado del capítulo 1, sobre las Bases de Datos Relacionales:
El objetivo principal del apartado 1.1, es el de presentar una definición coherente de lo que representa una base de datos relacional.
En el apartado 1.2 tiene por objetivo ejemplificar de forma clara y precisa el diseño de una base de datos relacional, tomando en consideración varios aspectos que se ven involucrados en la misma como son, las claves candidatas, principales, alternativas, externas, y secundarias.
El apartado 1.3, tiene por objetivo exponer un concepto de modelos y los tipos de segmentos que surgen ya sean estos estáticos o dinámicos.
Además, se expone los siguientes puntos:
Diseño de los sistemas de bases de datos

- Funciones de un SGBD
- Función de definición
- Función de manipulación.
- Función de control

En el apartado 1.4 se tiene por objetivo exponer la clasificación de los diversos tipos de diseño de datos que existen de acuerdo al nivel de abstracción, tomando así en cuenta los siguientes:

- Modelos de datos conceptuales
- Modelos de datos lógicos
- Modelos de datos físicos

El apartado 1.5 tiene por objetivo tratar sobre la enumeración de las reglas de codd, para un sistema relacional, y se describirá de forma breve cada una de ellas.

1.1. Generalidades de bases de datos

- **Base datos relacional:** Es aquella que está compuesta por un conjunto de datos almacenados y relacionados entre sí, permitiendo manipular los datos de una empresa. La función principal de una base de datos es mantener la integridad y seguridad de los datos ante cualquier incidente. [1], además es considerada como un sistema de datos integrados los cuales pueden ser manipulados directamente por una serie de aplicaciones. [2]
- **Sistema gestor de bases de datos (SGBD):** También denominado por sus siglas en inglés como Data Base Management System (DBMS), es una herramienta que sirve como interfaz entre la base de datos, el usuario y las aplicaciones. Además, permite definir, construir, almacenar y manejar una base de datos, permitiendo el control de los tipos de datos y restricciones sobre la información. [3]
- **Personas en el entorno de bases de datos:** Dentro del entorno de una base de datos intervienen cuatro grupos de personas que son las siguientes:

  Administrador de la base de datos: Es la persona encargada de establecer las políticas de seguridad y del acceso concurrente.

  Diseñadores de la base de datos: Aquellos que diseñan la base de datos, identificando datos, relaciones entre ellos, restricciones sobre los datos y sobre las relaciones. También deben tener conocimientos sobre los datos de la empresa y sus reglas de negocio.

  Programadores de aplicaciones: Encargados de implementar los programas de aplicación que servirán a los usuarios finales. Los mismos que permitirán insertar, modificar, eliminar y consultar datos. [4]

A continuación, se definen los elementos de una base de datos relacional:

- **Tabla:** Es aquella que se encarga de alojar datos de la base de datos y esta compuestas por:

Tupla: Se considera tupla a cada una de las filas de una tabla.

Atributo: Son las columnas de una tabla.

Grado: Número de atributos por los que está compuesta una tabla.

Cardinalidad: Corresponde al número de tuplas de una tabla.

Dominio: Permite especificar los posibles valores válidos para un atributo. Los dominios

Valor nulo: Indican contenidos de atributos que no tienen ningún valor. Cuando hablamos de un valor nulo dentro claves secundarias este indica que el registro actual no está relacionado con ninguno. En cambio, para otros atributos indica que no se puede rellenar ese valor por alguna razón.

Relaciones: Son aquellas que permiten evitar los datos redundantes.

- **Formularios:** Se utilizan para actualizar los datos.
- **Consultas:** Permiten ver, modificar y analizar datos.
- **Informes:** Utilizados para mostrar datos en formato impreso. [5]

1.2. Ejemplificación

En una base de datos relacional los datos se obtienen en relaciones y una relación se puede personalizar por medio de una tabla. Toda relación tiene un nombre y está compuesta por un conjunto de filas y columnas. Las columnas pertenecen a los atributos de la relación. Por lo tanto las filas poseen una serie de valores para cada uno de los atributos de la relación.

Al diseñar nuestra base de datos indicamos las tablas con sus respectivos campos y el tipo de dato que le corresponde a cada uno de ellos, como, por ejemplo, al definir el código debe ser de tipo string (cadena) con un tamaño de caracteres asignado por el administrador según las políticas del negocio, el nombre con un tamaño de treinta caracteres, el género con un tamaño de nueve caracteres y el cargo con un tamaño de cuarenta caracteres.

Una base de datos relacional debe garantizar la integridad y calidad

de los datos, por lo cual se debe cumplir con las reglas de negocio que establece la empresa. Además, un aspecto importante a considerar es la privacidad de la base de datos, al hablar de privacidad estamos hablando de los permisos o privilegios que se le otorgan a cada uno de los empleados de una empresa, ya que eso depende las acciones u operaciones que ellos puedan realizar dentro de la base de datos.

La disponibilidad también es un elemento fundamental de la base de datos, ya que permite a los usuarios y aplicaciones manipular permanentemente la información de una organización.

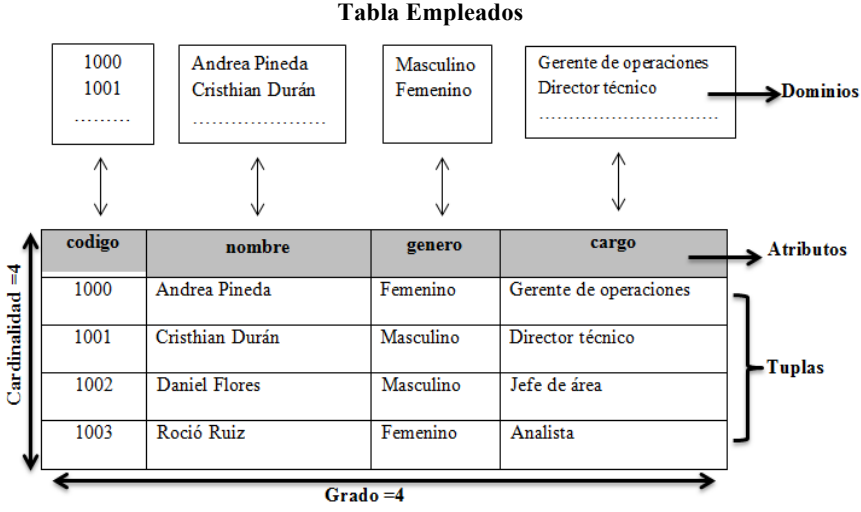


Figura. 1: Relación empleados con sus campos y registros.

Claves

Uno de los elementos primordiales de las tablas son las claves, una clave aspirante de una relación es un conjunto de atributos que representan un valor único de cada fila de la relación, lo cual quiere decir que no podrá haber en una relación más de una fila con el mismo valor único.

Una relación posee más de una clave aspirante o candidata, las cuales pueden ser clave principal y claves alternativas, o denominadas por otros autores como: candidata, externa o secundaria.

- **Clave candidata:** Es el conjunto de atributos de una tabla por los cuales se logra identificar unívocamente cada tupla de la tabla.
- **Clave principal o primaria:** Es la clave designada que permite identificar únicamente las filas de la relación, es decir esta será la clave principal. Por ejemplo, en la relación Empleado definimos al atributo codigo como la clave principal.
- **Claves alternativas:** Son aquellos atributos o claves candidatas que no han sido seleccionados como clave principal. Por ejemplo, en la relación Empleado, los atributos como: nombre, genero, cargo serían claves alternativas.
- **Clave externa o secundaria:** Es aquel atributo de una tabla que se encuentra relacionado con una clave de otra tabla. [5]

Si un atributo ya permite identificar a cada fila, no tiene sentido que la clave alternativa esté formada por ese atributo o quizás por algún otro, como podemos ver en la relación de la figura 1 el atributo código vendría hacer la clave principal ya que no puede haber dos empleados con el mismo valor para este atributo, pero en caso de definir el atributo nombre como clave principal no tendría que repetirse el nombre o apellido del empleado, lo cual suele pasar porque pueden existir varios empleados con el mismo nombre o con el mismo apellido por eso es recomendable usar el atributo codigo como clave principal por ser un campo único para cada persona. [6]

1.3. Concepto de modelos de datos. Funciones y sublenguajes

Un modelo de datos es un instrumento usado para crear una base de datos, además está compuesto por símbolos, conocimientos y normas que nos permiten representar los datos que se van a recopilar en una base de datos, es decir, por medio de estos modelos se puede reproducir la información. [7]

El esquema dentro de un modelo de datos es una descripción específica que permite representar a la base de datos. Existen diferentes modelos de datos que se ajustan en el transcurso de la creación de la base de datos, empezando por un esquema conceptual creado al inicio, pasando por esquemas intermedios hasta llegar al esquema interno que representa las características físicas que posee la base de datos.
Dentro de los modelos de datos existen dos tipos de segmentos: un

segmento estático y dinámico.

Segmento estático: Se clasifica en elementos autorizados y elementos no autorizados. Los elementos autorizados no son elementos similares para todos los modelos, pero están compuestos por entidades, atributos, dominios y relaciones. Estos elementos son personificados en manera de grafos.

En cuanto a los elementos no autorizados existen algunos valores que no se deben almacenar en una base de datos. Existen dos tipos de restricciones:

- Condiciones inherentes: Se define como condición inherente del modelo relacional a una relación en la que no debe tener dos filas idénticas.
- Condiciones semánticas: Estas condiciones suelen afectar al conjunto de valores que posee un atributo. Sin embargo, tiene como objetivo mostrar de la mejor forma posible el mundo real. La condición semántica en caso de usar un atributo llamado sueldo, sería que el sueldo solo puede adquirir valores enteros mayores a cero.

Segmento dinámico: Este segmento establece que unos conjuntos de operadores precisen sobre la estructura del modelo. Se llama ocurrencia del esquema o estado de la base de datos a los valores de los datos almacenados. Entonces la aplicación de una operación de una ocurrencia del diseño convierte a esta en otra ocurrencia.

$$0\ (BDi) = BDj$$

Por ejemplo, si guardamos información de cinco empleados en una tabla que pertenece a la base de datos, eso forma una ocurrencia del esquema. Si ingresamos un nuevo empleado estamos efectuando la operación de introducción, obteniendo otra ocurrencia del diseño de la base de datos, en el que ya no tendríamos cinco empleados guardados sino seis.

Al momento de crear una base de datos se deben seguir los siguientes pasos secuenciales:

- **Diseño conceptual:** Estos diseños son altamente semánticos y autónomos del tipo de base de datos que se va a usar posteriormente, independientemente del Sistema Gestor de Bases de Datos (SGBD). Hoy en día el diseño de datos usado para la

elaboración de esta tarea es el Modelo Entidad Relación.

- **Diseño lógico:** Parte del diseño conceptual elaborado anteriormente y convertirlo en el diseño lógico utilizando una serie de reglas de innovación dependientes del diseño lógico. Existen diversos tipos de diseños lógicos como son el diseño jerárquico, el diseño en red y el diseño relacional.
- **Diseño físico:** Convierte el diseño lógico en diseño físico a través de la herramienta SGBD, usando todos los elementos que contiene la base de datos como: dominios, tablas, prohibiciones, índices, etc.

1.3.1. Diseño de los sistemas de bases de datos

El objetivo de un SGBD es impedir a los usuarios las referencias sobre como los datos son almacenados y conservados, por lo cual el administrador de la base de datos tiene que representar la estructura de los datos en diferentes niveles; por lo cual el resto de usuarios pueden acceder, a esto se le conoce como diseño de los sistemas de bases de datos. Debido a la normativa ANS/X3/SPARC, el diseño de una base de datos debe tener tres niveles de abstracción:

- Nivel físico: Es el nivel más bajo de abstracción en el que se representa cómo se almacenan físicamente la información: la capacidad de los bloques de datos, procesos de direccionamiento, los índices, etc.
- Nivel lógico: En este nivel se describen los datos que se están siendo almacenados en la base de datos y que tipo de relaciones existen entre los datos.
- Nivel externo: Este nivel es el más cercano al usuario, ya que se describe lo que más le interesa al grupo de usuarios sobre la base de datos ocultándole el resto de la base de datos.

1.3.2. Funciones de un SGBD

Las funciones principales de un Sistema Gestor de Base de Datos son la función de definición, manipulación y de control:

1.3.2.1. Función de definición

Esta función permite al diseñador de la base de datos definir los elementos, estructura y las relaciones, normas de integridad y de confiabi-

lidad, así mismo la definir las particularidades de tipo fisico y las vistas de los usuarios. Por lo que se da el uso de un lenguaje de definición de datos (DDL) el cual provee precisar las estructuras fisicas, lógica global y externas, proporcionados a cada uno de los niveles del diseño.

1.3.2.2. Función de manipulación.

Esta función permite a los usuarios realizar búsquedas y poder reemplazar la información que se encuentra guardada en la base de datos. Para la actualización de la base de datos se involucra tres tipos de operaciones que son:

- **Inserción:** Consiste en aumentar los datos, por ejemplo: insertar los datos de un producto nuevo que se comercializa en una empresa.
- **Eliminación:** Consiste en disminuir la información de la base de datos, por ejemplo: eliminar o borrar los datos de un producto que se dejó de comercializar en una empresa.
- **Modificación:** Consiste en editar la información de la base de datos, por ejemplo: modificar el precio de un producto.

La función de manipulación se producirá a cabo por la interminación de un lenguaje de manipulación de datos (DML), es decir que estos lenguajes consiguen catalogar varios criterios:

A través de la forma independiente que maneja el DML, se ha logrado conocer a los lenguajes huésped, autocontenidos o duales. DML huésped estos lenguajes se tienen que infiltrar en otro lenguaje de programación. Los DML autocontenidos son aquellos lenguajes utilizados por los usuarios que poseen escasos conocimientos en cuanto a la programación. Los lenguajes como el SQL, son manipulados como lenguajes huéspedes, y son denominados como lenguajes duales.

De acuerdo al detalle se puede especificar el procedimiento debido para permitir los datos y consultarlos, ya que existen dos tipos de lenguajes: Los de tipo procedimentales permiten al usuario proporcionar instrucciones al sistema para que realice una serie de operaciones en la base de datos y los poco procedimentales en cambio permiten al usuario describir la información deseada sin un procedimiento específico con el fin de obtener esa información.

Los lenguajes de especificación y lenguajes navegacionales se diferencian en su forma de cómo recuperar o actualizar los datos. Los

lenguajes de especificación por medio de cada sentencia del DML logra recuperar un conjunto de registros, mientras que los lenguajes navegacionales recuperan un solo registro.

1.3.2.3. Función de control

Esta función está compuesta por una serie de herramientas las mismas que preparan la tarea del administrador de la base de datos. La administración del sistema es quien se encarga de la administración de usuarios. Debemos tomar en cuenta que los administradores tienen que monitorizar el trabajo de la base de datos, así mismo respaldar y realizar copias de seguridad de la base de datos en caso de algún incidente, etc.

1.4. Clasificación de los varios tipos de diseño de datos de acuerdo al nivel de abstracción.

Existen diferentes tipos de diseño de datos que se adaptan a lo largo del proceso de la elaboración de la base de datos, es decir, estos diseños son utilizados en las etapas del diseño de bases de datos como por el ejemplo el diseño conceptual, diseño lógico y diseño físico.

1.4.1. Modelos de datos conceptuales

Estos modelos son diseños semánticos y autónomos independientemente del tipo de base de datos que se procura crear, los mismos que reciben el nombre de diseños lógicos.

Hoy en día el modelo Entidad Interrelación es el modelo de datos más usado, mientras que otros autores definen que el modelo propuesto por Chen, conocido como el modelo extendido Entidad Relación es el más utilizado en la actualidad.

1.4.1.1. Modelos Entidad Relación

Chen plantea como elementos básicos del Modelo Entidad Relación a las entidades, relaciones y atributos:

- **Entidad:** Una entidad es como un objeto del cual se desea almacenar información en la base de datos. Por ejemplo, si deseamos crear una base de datos para una biblioteca, las entidades vendrían a ser los usuarios, libros, autores de los libros,

ejemplares, etc. Las entidades son representadas a través de rectángulos.

- **Relación:** Es la correspondencia entre entidades. Las relaciones se representan por medio de un rombo con el respectivo nombre en su interior.
- **Atributo:** Los atributos son las características de una entidad. Los atributos son representados de varias maneras; con un óvalo y el nombre del respectivo atributo en su interior puede ser incorporado directamente con la entidad o con la relación.

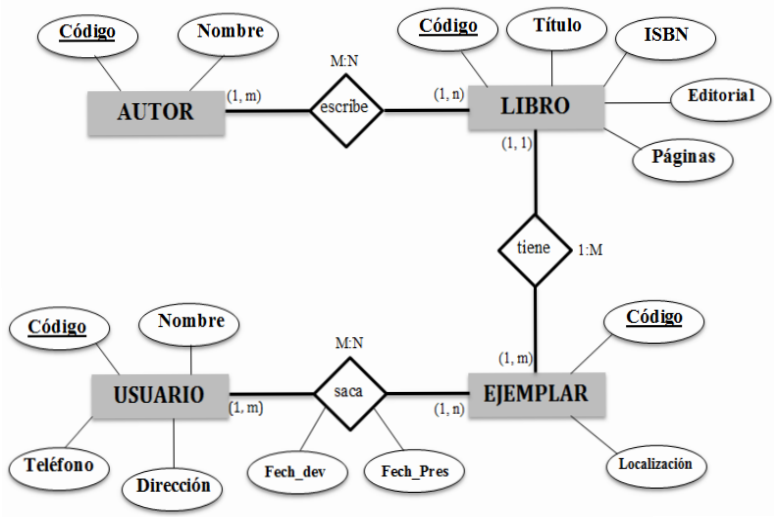


Figura. 2: Esquema entidad relación de una biblioteca

1.4.1.2. Diseño orientado a objetos

A raíz de que se formaron nuevos métodos de análisis y diseño orientado a objetos esto dio paso al Lenguaje de Modelado Unificado (UML), el mismo que constituye una serie de reglas empleadas al momento de crear cualquier tipo de diagrama.

El modelo orientado a objetos está constituido por los siguientes elementos:

- Clase: Se considera clase a un objeto cualquiera sobre el que se desea almacenar información en la base de datos.

- Atributo: Los atributos vienen hacer las características de una clase, es decir la información que se espera guardar sobre ella.
- Método: Los métodos son las operaciones que se van a aplicar sobre una clase. Por ejemplo, para la clase libro tendríamos como métodos: solicitar y devolver un libro por parte del usuario a la biblioteca.
- Relación: Las relaciones son las respectivas agrupaciones que se dan entre las clases. Existen varios tipos de relaciones en el lenguaje de modelado unificado como son: asociación, jerarquía o generalización, agregación y composición, cada una de ellas se representa de forma distinta.

A continuación, se muestra un ejemplo de un diagrama de clases sobre una biblioteca.

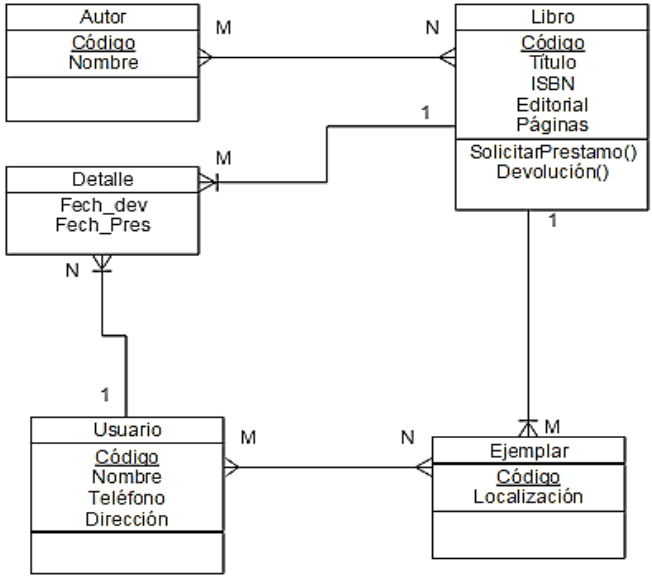


Figura. 3: Diagrama de clases para una biblioteca.

1.4.2. Modelos de datos lógicos

Estos modelos son usados para crear un diseño lógico que represente la estructura la base de datos que se va a diseñar. Un diseño lógico parte del diseño conceptual. Existen tres modelos de datos lógicos: el modelo jerárquico, modelo en red y el más usado en la actualidad el

modelo relacional.

1.4.2.1. Modelo jerárquico

Este modelo usa una estructura en forma de árbol, en la cual el padre puede poseer cualquier número de hijos, pero un hijo solo puede tener un padre.

A continuación, se mostrará el ejemplo de un diagrama de estructura de árbol con dos ejemplos de registros: sucursal y empleados, en donde varios empleados pueden tener muchos hijos.

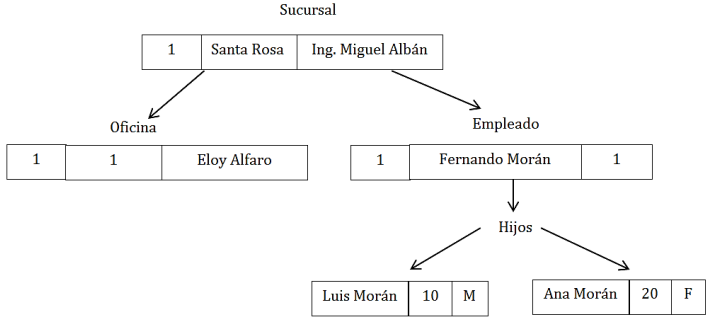


Figura. 4: Esquema de una base de datos jerárquica.

1.4.2.2. Modelo en red

Este modelo maneja una estructura no lineal en la que cada registro hijo puede tener más un nodo padre. La simbología de las entidades son nodos de un grafo y las asociaciones, mediante los arcos.

Codasyl se considera como el modelo en red más extenso. En la figura 5 se observa un ejemplo de la forma de la información en este diseño de datos.

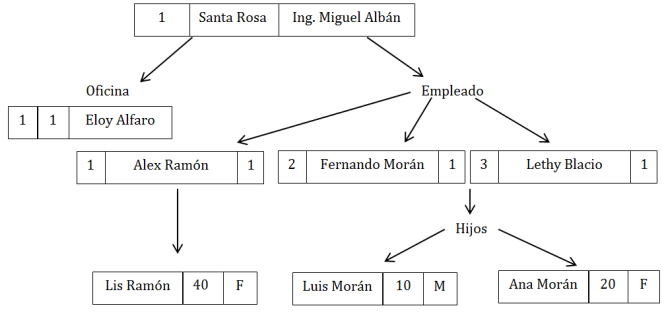


Figura. 5: Esquema de una base de datos en red.

1.4.2.3 Modelo relacional

En este modelo se hace uso de las tablas para la forma lógica de los datos y sus relaciones, este modelo es creado por Codd en el año de 1970. Se denomina tupla a cada fila de la tabla y atributo o campo a cada columna de la tabla. El campo principal de una tabla que marca la diferencia entre otros campos es la clave.

En la siguiente figura 6 se mostrará la información que se desea almacenar en la base de datos. Ésta manejará información sobre la sucursal de una empresa y sus empleados.

Num_Dep	Nombre_Dep	Localidad_Dep
1001	Administración	Machala
1002	Operaciones	Santa Rosa
1003	Gerencia	Pasaje

Num_Emp	Nombre_Emp	Sueldo_Emp	Num_Dep
40	Alex Ramón	2000	3
41	Lethy Blacio	1000	1
42	Paul Peña	1500	2

Figura. 6: Diseño de una base de datos relacional con datos.

1.4.3. Modelos de datos físicos

El diseño físico es la última etapa del diseño de una base de datos el cual consiste en crear en un SGBD los elementos que formarán parte de la base de datos. En el caso de una base de datos relacional se debería crear tablas, disparadores, vistas, índices, etc.

Una base de datos relacional está compuesta por una o varias tablas. Para la creación de una tabla mediante el lenguaje de definición de datos se utiliza la instrucción CREATE TABLE. Es necesario definir los atributos por los que estará compuesta una tabla como son: nombre, tipo de datos y restricciones asociadas al atributo.

A continuación, veremos un ejemplo de cómo crear en el SGBD Oracle la base de datos relacional, utilizando el mismo diseño lógico que se muestra en la figura 2, para lo cual se utilizarían las siguientes instrucciones CREATE TABLE:

create table **Autor**

(Código char (5) primary key,

```
Nombre varchar2 (30));
create table Libro
(Código char (7) primary key,
Título varchar2(60) not null,
ISBN char (10) unique,
Editorial varchar2(0),
Paginas number (5,0) check (Paginas >0));
create table Ejemplar
(Codlibro char (7) references Libro (Código),
Localización  varchar2(60) references Autor (Nombre),
primary key (CodLibro, Localización));
create table Usuario
(Código number (7) primary key check (Codigo >0),
Nombre varchar2(40) not null,
Teléfono char(9) not null),
Dirección varchar2(60));
create table Prestamo
(Codlibro char(8) references Libro (Código),
Código number(6) references Usuario (Código),
Fech_Pres date,
Fech_dev date,
primary key (CodLibro, Código, Fech_Pres, Fech_dev));
```

1.5. Enumeración de las reglas de Codd para un sistema relacional

A continuación, se detallan las doce reglas definidas por Codd en 1984 las cuales deben cumplirse al desarrollar un sistema relacional. Aunque resulta difícil que un sistema relacional llegue a efectuar algunas de estas normas, a continuación, se detallaran las reglas de Codd:

Regla 1: Se debe representar toda la información de la base de datos en el esquema lógico, es decir que todos los datos están en las tablas.

Regla 2: Regla del acceso garantizado: Todos los datos de una base de datos relacional deben ser accesibles al nivel lógico usando el nombre de tabla, valor de la clave principal y nombre de columna.

Regla 3: Tratamiento sistemático de valores nulos: A través de los valores nulos es posible representar información inédita o no aplicable,

independientemente del tipo de datos.

Regla 4: Catálogo dinámico en línea basado en el modelo relacional: Solo los usuarios autorizados pueden realizar consultar y modificar datos. Así mismo los usuarios pueden tener acceso a la estructura de la base de datos.

Regla 5: Regla del sublenguaje de datos completo: Debe por lo menos haber un lenguaje con una sintaxis bien definida que permita la administración de la base de datos.

Regla 6: Regla de actualización de vistas: Aquellas vistas que se pueden modificar deben también permitir la modificación por parte del sistema.

Regla 7: Inserción, modificación y eliminación de alto nivel: En el momento en que se realice una inserción, modificación o eliminación el sistema deberá ser capaz de proporcionar datos. El lenguaje de manejo de datos puede ser de declaración, es decir, debe permitir, por ejemplo, cambiar algunas filas de una tabla de forma conjunta.

Regla 8: Independencia física de datos: No interviene el usuario y está enfocada a que las modificaciones que se realicen en la base de datos no alteren el método de acceso. Por ejemplo, organización de archivos para mejorar el rendimiento ("separación física de aplicaciones y estructura de almacenamiento").

Regla 9: Independencia lógica de datos: Las aplicaciones no deben verse afectadas por las modificaciones que se realizan en las tablas de la base de datos, ya que ambas funcionan diferente. Ejemplo: Eclipse – Mysql.

Regla 10: Independencia de integridad: Se debe especificar las limitaciones de integridad para una base de datos relacional por separado de los programas de aplicación y almacenar en la base de datos.

Regla 11: Independencia de distribución: La distribución de una base de datos es independiente, ya sea cuando se trabaja con una base de datos compartida en diferentes ordenadores o cuando se trabaje con una base de datos centralizada.

Regla 12: Regla de la no subversión: Si en un sistema relacional cuenta con un lenguaje de bajo nivel, esto le impide que pueda saltarse las normas de integridad y las limitaciones mencionadas en los lenguajes relacionales de más alto nivel. Ejemplo: búsqueda del apellido de un empleado, resultado el número de teléfono. [5]

Resumen

- Una base de datos relacional está compuesta por un conjunto de datos almacenados y relacionados entre sí, permitiendo manipular la información de una empresa
- En una base de datos relacional la información se recopila en base al contenido de las tablas y las relaciones que existe entre ellas
- Un modelo de datos es un instrumento usado para crear una base de datos, además está compuesto por símbolos, conocimientos y normas que nos permiten personificar los datos que se van a recopilar en una base de datos.
- El objetivo de un SGBD es impedir a los usuarios las referencias sobre como la información es almacenada y conservada,
- Las funciones principales de un Sistema Gestor de Base de Datos son la función de definición, manipulación y de control
- Existen diferentes tipos de diseño de datos que se adaptan a lo largo del proceso de la elaboración de la base de datos, los cuales son: Modelos de datos conceptuales, Modelos de datos lógicos, Modelos de datos físicos

Descripción y aplicación del modelo entidad-relación para el modelado de datos

Descripción del capítulo

A continuación, se detalla cada contenido del capítulo 2, acerca de la descripción y aplicación del modelo Entidad - Relación para el modelado de datos.

En el apartado 2.1, tiene por objetivo conocer el origen del modelo entidad – relación.

En el apartado 2.2, tiene por objetivo conocer cada uno de los elementos que forman parte de un modelo entidad - relación, siendo estos:

- Entidad.
- Relación.
- Atributo.
- Dominio.

En el apartado 2.3, tiene por objetivo analizar el modelo entidad – relación extendido, el mismo que detalla puntos importantes como:

- Cardinalidad.
- Entidad frágil y fuerte.
- Atributos en entidades.
- Herencia.
- Agregación
- Generalización.

En el apartado 2.4, tiene por objetivo determinar la forma más opcional de pasar de un modelo entidad – relación a un modelo relacional.

En el apartado 2.5, tiene por objetivo plantear una serie de prácticas para diagramar modelos de entidad - relación y su correcta transformación a un modelo de datos mediante reglas definidas.

2.1. Proceso de realización de modelo entidad - relación

Actualmente para diseñar o modelar conceptualmente una base de datos se utiliza el modelo de datos llamado modelo entidad – interrelación, conocido también como modelo entidad – relación (ER); fue presentado por Peter Chen en el año de 1976, el mismo que estuvo sujeto a varias contribuciones por diferentes autores creando así el modelo entidad - relación extendido. [5]

Como elementos básicos del modelo entidad- relación tenemos: entidad, atributo, dominio y relación o interrelación, los cuales se explicarán más adelante.

2.2. Elementos

El modelo entidad – relación se compone de varios elementos, los mismos que se detallan a continuación.

2.2.1. Entidad

Se define como entidad al objeto sobre el cual se desea guardar información en una base de datos. Por ejemplo, al diseñar una base de datos para gestionar el inventario de una empresa, las entidades que formarían el modelo serían almacén, departamento, producto, etc. [8]

Dentro de una entidad tenemos las llamadas ocurrencias de una entidad, que son aquellos valores determinados que toman cada uno de los atributos de una entidad. En este caso un ejemplo de ocurrencia de una entidad Producto seria el producto “Televisor Oled Curvo De 55” EC9300”.

La representación gráfica de una entidad, es por medio de un rectángulo en el cual se coloca el nombre de la entidad. Ejemplo, entidad Producto.

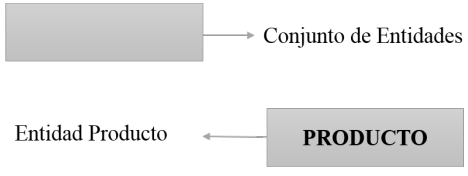


Figura. 7: Representación gráfica de una entidad

2.2.2. Relación

Se entiende por relación a la asociación o vínculo entre dos o más entidades, estableciendo alguna interacción entre ellas. En el caso de un Curso y Alumno se establecería como relación tiene para mostrar que los cursos tienen alumnos.

La representación gráfica de una relación es por medio de un rombo en el cual va el nombre de la relación, asociadas de las entidades que forman parte de la relación. Para demostrar la representación de la relación se usará el ejemplo dado anteriormente entre Curso y Alumno. [5]



Figura. 8: Representación gráfica de una relación

Las relaciones poseen las siguientes características:

- **Nombre:** Cada relación poseerá un nombre que las distinguirá de las demás.
- **Grado**: Número de entidades que son parte de una relación. Se clasifican en:
  - Relaciones binarias o de grado 2: Relaciones que asocian a dos entidades distintas.
  - Relaciones reflexivas o de grado 1: Relaciones que solo relacionan una entidad consigo misma.

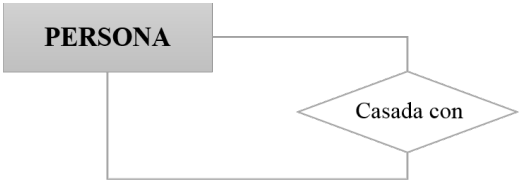


Figura. 9: Relación reflexiva

  - Relaciones ternarias, cuaternarias, o de grado 3,4: Relaciones que asocian más de dos entidades.

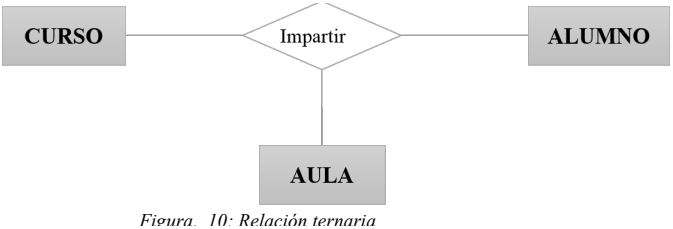


Figura. 10: Relación ternaria

- Las relaciones poco usuales son las relaciones de grado 2, a diferencia de las relaciones binarias que son las más utilizadas, le siguen las relaciones reflexivas.

Tipo de correspondencia: Es el número de entidades en la que se puede asociar otra entidad a través del conjunto de relaciones. Se clasifican en:

1:1: Se da cuando un registro de cada entidad está relacionado con un único registro de otra entidad. Ejemplo:
Una Persona tiene un usuario y un Usuario le pertenece a una sola persona.



Figura. 11: Relación 1:1 binaria

Una Persona está casada con una sola Persona.

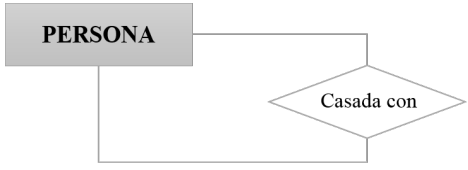


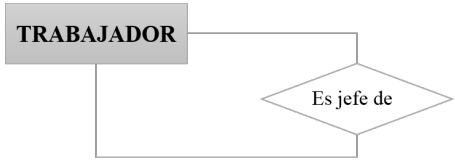
Figura. 12: Relación 1:1 reflexiva.

- 1:N: Se da cuando un registro de una entidad A solo está relacionado con un único registro de la entidad B, y un registro de la entidad B tiene más de un registro relacionado con la entidad A. Ejemplo: Un Cliente tiene varias facturas, pero una Factura le pertenece a un solo cliente.



Figura. 13: Relación 1:N binaria

Un Trabajador puede ser jefe de varios trabajadores, pero un trabajador solo puede tener un jefe.



- N:M: Se da cuando un registro de una entidad puede estar relacionado con más de un registro de otra entidad, en este caso de correspondencia es necesario crear una nueva tabla que incluya los registros relacionados entre sí. Ejemplo: Una Factura puede incluir varios productos, y un Producto puede estar en varias facturas.



Figura. 15: Relación N:M binaria.

Un Menú puede contener varios menús.

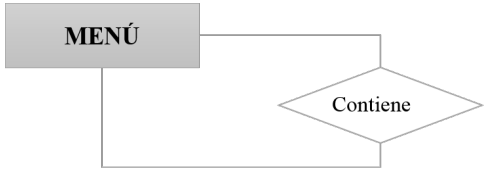


Figura. 16: Relación N:M reflexiva

2.2.3. Atributo y dominio

Un atributo se define como las propiedades o características que están asociadas a una entidad o relación. En una entidad Producto sus atributos serían código, nombre, descripción, valor, etc. Un dominio es el valor permitido para cada atributo de una entidad, en el caso de un atributo código, su dominio sería una cadena de cinco caracteres.

Una manera de representar los dominios es a través de un círculo, el mismo que deberá contener su nombre y en la línea que se relaciona con la entidad se colocará el nombre del atributo; esta representación es muy poco factible debido a que el modelo entidad – relación resultan muy pesados y confusos. [9]

Es por aquella razón que la representación del modelo entidad - re-

lación tuvo un pequeño cambio, se omitiría el nombre del atributo en la línea de asociación con la entidad y solo se colocaría el nombre del atributo en el círculo prescindiendo del nombre del dominio.

Existen diferentes tipos de atributos entre ellos tenemos:

- Atributo identificador principal [AIP): Atributo que permite identificar cada registro de la entidad adecuada, por ejemplo, en la entidad producto el atributo identificador principal sería el código debido a que no existirán dos o más productos registrados con el mismo código.
- Atributo identificador candidato (AIC]: Si existe más de un atributo que cumpla con la condición de ser un atributo AIP, a los atributos no seleccionados se los denominarán atributo identificador candidato.
- Atributo identificador alternativo (AIA): Es el atributo identificador candidato no designado como atributo identificador principal. [10]

Otra forma de representar un modelo de entidad – relación, es colocando dentro de los óvalos o círculos el nombre de los atributos de determinada entidad. El atributo identificador principal se lo resalta mediante una línea de subrayado, mientras que los atributos alternativos no se subrayan.

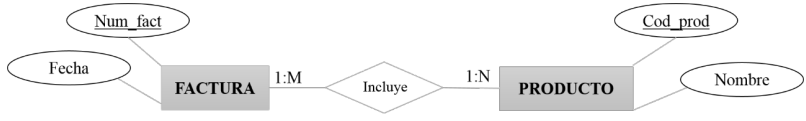


Figura. 17: Representación de atributos

También se puede representar colocando un círculo para cada atributo, el círculo rellenado de negro será para el atributo identificador principal, y el círculo vacío será para el atributo identificador alternativo.

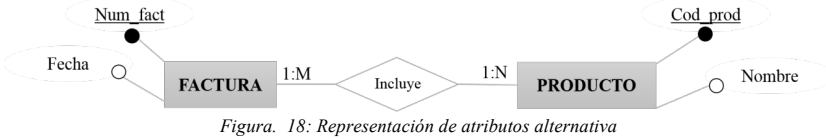


Figura. 18: Representación de atributos alternativa

2.3. Diagramas entidad-relación extendidos como elementos para resolver las carencias de los diagramas entidad-relación simples.

Luego que Peter Chen haga la presentación de su modelo entidad – relación extendido, varios autores han creado diversos modelos atractivos basándose en el modelo entidad – relación. de Chen.

2.3.1. Cardinalidad de las relaciones.

Se define como cardinalidad al número de veces en que se asocia una entidad con otra, cuando hablamos de cardinalidad máxima y mínima hacemos referencia al máximo y mínimo de ocurrencias de una entidad asociada a otra entidad por medio de una relación. Las cardinalidades se clasifican en: uno a uno, uno a muchos y muchos a muchos, cada una de estas cardinalidades poseen las etiquetas de (1,1), (1,N) y (N,M). [8]

Para representar una etiqueta N, M es necesario definir el máximo y mínimo de la cardinalidad. En este caso tenemos la entidad Factura y Producto, se entiende que una factura incluye como mínimo un producto y como máximo varios productos; así mismo un producto no podrá estar en ninguna factura o pertenecer a muchas facturas.

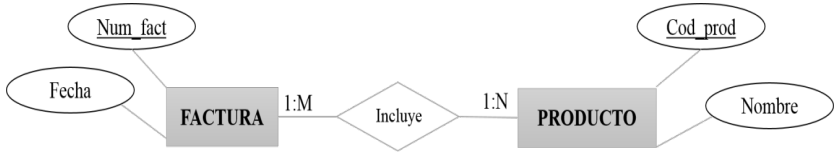


Figura. 19: Representación de cardinalidades en una relación binaria N:M.

En una relación reflexiva, decimos que muchos trabajadores tienen un solo jefe o no, y que un trabajador puede o no ser jefe de un trabajador.

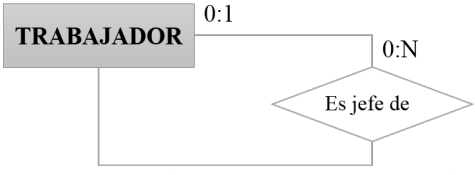


Figura. 20: Representación de cardinalidades en una relación reflexiva N:M

Las relaciones ternarias o también conocidas como relaciones con grado mayor que 2, representan la cardinalidad de manera diferente. En este caso tenemos la entidad: Curso, Aula y Alumno las relaciones serían las siguientes:

- Un Curso puede tener como mínimo una impartición o varias.
- Un Aula puede albergar varias imparticiones o una.
- Una impartición puede tener un o varios alumnos.

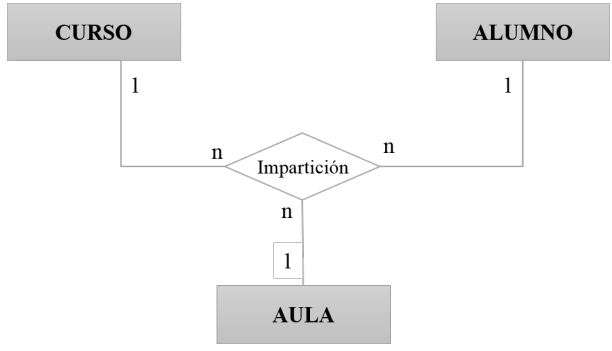


Figura. 21: Representación de cardinalidades en una relación ternaria

2.3.2. Entidades fuertes y frágiles

Existen dos tipos de entidades y estas son:

- Entidades fuertes: Se representa con un rectángulo dentro del cual se coloca el nombre de la entidad. También conocida como entidad regular la cual es posible identificarla unívocamente por cada uno de sus atributos.
- Entidades frágiles: Este tipo de entidad se caracteriza por no ser identificada unívocamente por sus atributos y que no puede existir sin ser parte de una relación. Se la representa gráficamente mediante un doble rectángulo y dentro de él nombre de la entidad. [10]



Figura. 22: Representación gráfica de una entidad frágil.

De igual manera podemos encontrar tipos de relaciones dependiendo de las entidades que se asocien, tenemos a las relaciones habituales y frágiles.

- **Relaciones habituales:** Relaciones entre entidades habituales.
- **Relaciones frágiles:** Relaciones entre entidades frágiles y habituales. Se subdividen en:
  - **Dependencia en identificar:** Se da cuando solo es posible identificar las ocurrencias de entidades frágiles con el atributo identificador principal de una entidad habitual. Ejemplo: Préstamo – Pago, la entidad Pago es una entidad habitual y no permitirá realizar un pago sino se cuenta con el identificador principal de la entidad Préstamo.

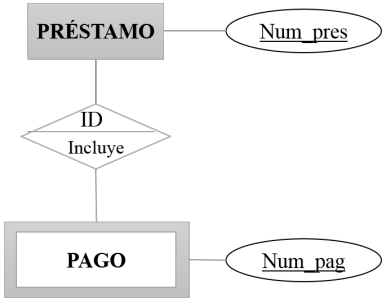


Figura. 23: Dependencia en identificación.

- **Dependencia en existencia:** Se da cuando existe una asociación entre una entidad fuerte y una entidad frágil, de tal manera que un registro de la entidad frágil no exista sin un registro de la entidad fuerte del cual es dependiente. Ejemplo: Cliente – Préstamo, decimos un cliente puede poseer o no un préstamo, y varios préstamos pueden pertenecer a un cliente. Si un cliente no existe, tampoco los préstamos debido a la dependencia un préstamo solo es generado por un cliente. Dentro de la relación se colocó una E que representa la existencia.

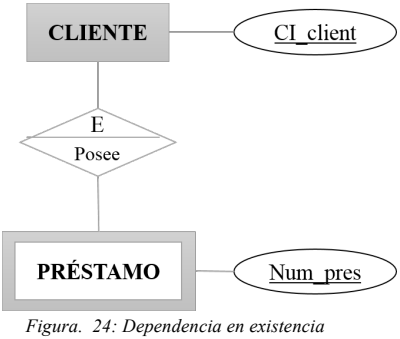


Figura. 24: Dependencia en existencia

2.3.3. Atributos en relaciones

Un atributo puede ser parte tanto de una entidad como de una relación. Su representación es igual a la representación de los atributos de las entidades, cada relación puede tener atributos asociados. Por ejemplo, en el caso que se desee la cantidad de productos de una factura de un cliente, el atributo cantidad de la factura debería colocarse en la relación "agrupa".

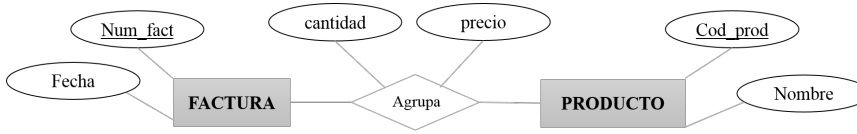


Figura. 25: Relación N:M con atributo.

Se puede fijar atributos a relaciones 1,1 o 1,N, a cualquiera de las entidades que se encuentren asociadas. En la relación "tiene", la entidad Curso y Alumno se agregará el atributo dirigente en la relación "tiene", ya que cada curso tiene designado un dirigente. [8]

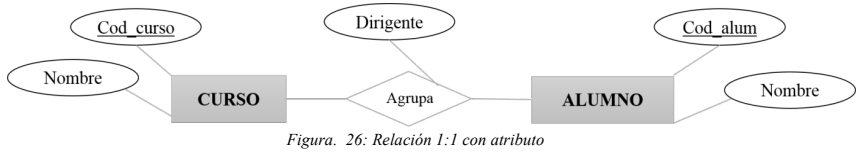


Figura. 26: Relación 1:1 con atributo

2.3.4. Herencia

En ciertos casos hemos encontrado entidades parecidas, es por esta razón que la herencia es un tipo de asociación entre una entidad padre y una entidad hijo. La entidad hijo hereda todos los atributos de la entidad padre, omitiendo la representación de una de ellas.

Se representan mediante un triángulo inverso que se conecta con las entidades mediante una línea. En el siguiente ejemplo se muestre la herencia de las entidades Estudiante, Docente, Representante y Usuario a una Entidad Persona.

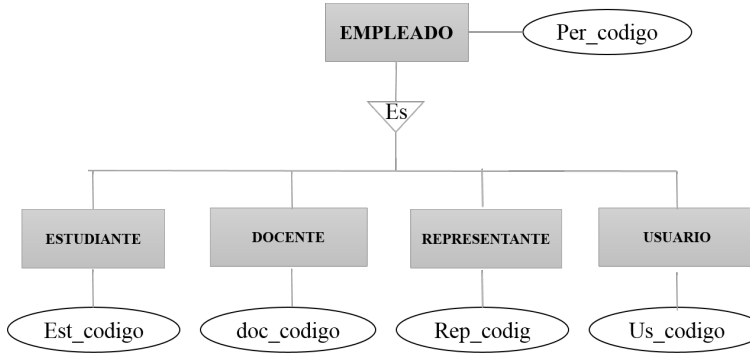


Figura. 27: Representación de Herencia

Las generalizaciones podrán ser de varios tipos:

Generalización integral: Se da cuando no existen ocurrencias que no pertenezcan a ninguna de las entidades que heredan.

Generalización Parcial: Se da cuando si existen ocurrencias que no pertenezcan a ninguna de las entidades que heredan

Generalización Exclusiva: Se da cuando una entidad que hereda consigue ser parte de una entidad supertipo (tipo genérico que puede dividirse en subtipos, contiene atributos comunes), su representación se realiza mediante un arco que contiene las líneas de asociación con las demás entidades.

Si la entidad Empleado formara parte de una de sus entidades subtipos (entidad cuyas instancias heredan algunos atributos de una entidad supertipo), se señalaría este suceso con un arco [11]. Se representaría de la siguiente manera:

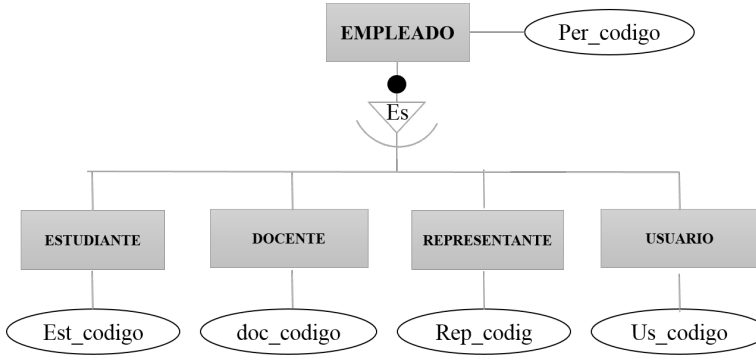


Figura. 28: Jerarquía de tipos subtipos total y exclusiva

3.5. Agregación

El modelo entidad – relación no permite crear relaciones entre relaciones, es decir no admite crear relación entre relación y entidad, es por esta razón que recibe el concepto de agregación.

La agregación permite crear a partir de más de dos entidades y una relación que las asocia entre sí, una entidad de nivel mucho más alto. Logrando a partir de objetos elementos, crear objetos compuestos. [12]

Un ejemplo de relación N,M tenemos las entidades Estudiante, Curso y Acta de Notas, las cuales muestran que algunos estudiantes se han matriculado en un curso y muestra en qué fecha y en qué estado se encuentra la matrícula. Para relacionar las entidades Estudiante y Curso se instaura la relación se matricula y la relación se registra que asociará a la entidad Acta de Notas.

Una manera de representar este diagrama es envolviendo en un rectángulo las entidades y relaciones que deseamos asociar (Estudiante, Curso, se matricula) a través de una nueva relación que la llamaremos matrícula.

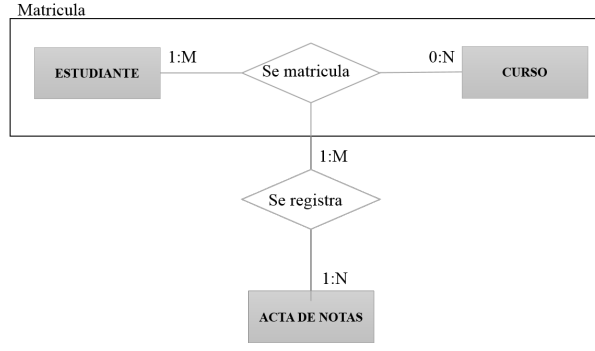
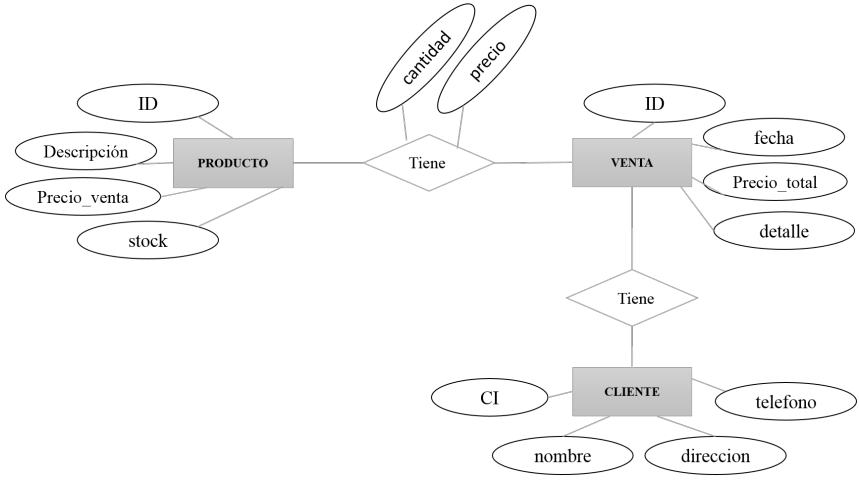


Figura. 29: Representación de agregación.

2.4. Diseño lógico a partir de diagramas entidad - relación

Para diseñar un esquema lógico relacional de una base de datos se mostrarán algunas de las formas posibles. Puntualmente se estudiará la forma de transformar el modelo Entidad – Relación a un modelo relacional. Lo primero en analizar serán las reglas dadas para convertir el modelo Entidad – Relación en un modelo relacional.
El modelo ejemplo de Entidad Relación será el siguiente:



2.4.1. Reglas concernientes al modelo entidad - relación básico

2.4.1.1. Transformación de entidades

Para transformar una entidad del modelo entidad – relación al modelo relacional, cada una de las entidades pasarán como relaciones al modelo relacional.

2.4.1.2. Transformación de atributos

Al transformar las entidades del modelo entidad - relación al modelo relacional pasan como relación, y los atributos de estas entidades pasarán como atributos de la relación en el esquema relacional. Los atributos identificadores principales pasarán como la clave principal de la relación. En la mayoría de sistemas gestores de base de datos ya no es prescindible detallar los atributos con la restricción Not Null debido a que las claves principales nunca tomarán un valor nulo. [13]

En el ejemplo de modelo entidad - relación dado anteriormente las relaciones y sus atributos quedarían de la siguiente manera:

Cliente $\longrightarrow$ CI (Cédula de Identidad), nombre, dirección, teléfono.

Venta $\longrightarrow$ ID (Identificación), fecha precio_total, detalle.

Producto $\longrightarrow$ ID (Identificación), descripción, precio_venta, stock.

Para los atributos identificadores alternativos si es prescindible la restricción Not Null, ya que cada uno de esos atributos serán necesarios al momento de insertarlos.

2.4.1.3. Transformación de interrelaciones binarias y reflexivas

Para transformar las interrelaciones binarias y reflexivas al modelo relacional, es necesario conocer el grado y la conectividad, para que así su transformación sea la más adecuada.

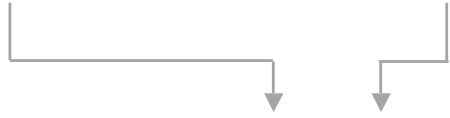
Interrelaciones N M

Toda interrelación N, M se transforma en una relación, la misma que posee como claves secundarias a cada una de las claves principales de las entidades interrelacionadas. La nueva relación tendrá como clave principal a la agrupación de las claves secundarias, los atributos de la entidad pasarán a la nueva relación.

Considerando el ejemplo del modelo entidad – relación de la figura 30 con la asociación entre las entidades Producto y Venta, se creará una nueva tabla que tendrá como atributos las claves principales de cada tabla asociada y los atributos de la relación que serían cantidad y precio.

Venta ⟶ID, fecha precio_total, detalle

Producto⟶ID, descripción, precio_venta, stock



Detalle_venta → Id_venta, Id_producto, cantidad, precio

En el caso de una relación reflexiva, de igual manera se creará una nueva tabla con las claves principales de cada entidad asociada. En el siguiente ejemplo se muestra que un menú puede dividirse en algunos submenús y dicho submenú puede constar en varios menús. [14]

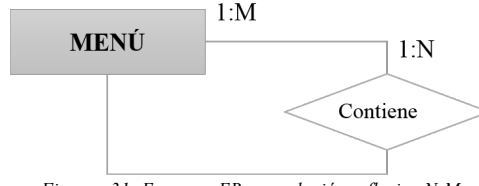


Figura. 31: Esquema ER con relación reflexiva N:M

La nueva tabla creada constará de claves secundarias Cod_Menu, Cod_Submenu.

Menú ⟶ Cod_Menu, Contenido



Interrelaciones 1: N

Las entidades que forman parte de una interrelación 1, N se transforman en relaciones cada una con sus correspondientes atributos.

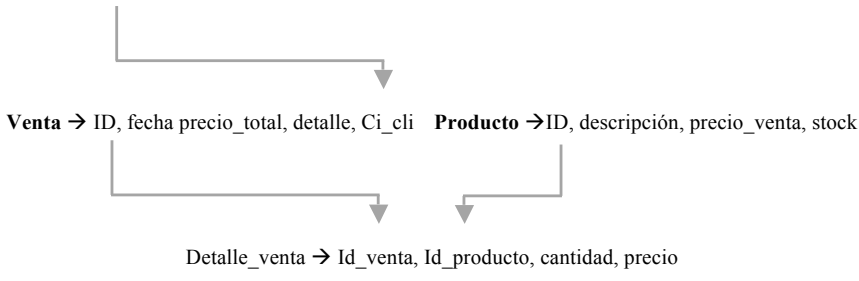
Para este tipo de interrelación es necesario solamente colocar del lado de la relación que corresponde a N, la clave secundaria que haga referencia a la otra relación.

Si la cardinalidad es de 1,1, colocaremos la clave secundaria con la restricción Not Null, ya que este atributo será necesario al momento de su inserción.

Si la cardinalidad es 0,1, indica que los atributos de la clave secundaria si pueden recibir un valor nulo. [13] [14]

Como ejemplo nos guiamos del modelo entidad – relación de la figura 30, en donde se establecerá un vínculo entre las entidades Venta y Cliente, en la relación Venta irá como clave secundaria el atributo CI. Este atributo tomará la restricción Not Null, ya que toda venta se la realiza a un cliente.

Cliente⟶ CI, nombre, dirección, ciudad



Si encontramos un caso de interrelación en donde esta posea atributos propios, estos atributos podrán ser colocados en otra entidad además de la clave secundaria.

A estos casos de interrelación también es posible aplicar el mismo proceso de una interrelación N, M, en la cual se crearía una nueva tabla con las claves secundarias y los atributos propios de la relación;

pero solo bajo las siguientes condiciones:

- Si la cardinalidad es 0,1, la clave secundaria tomará valor nulo.
- Si se desea conservar la semántica de la interrelación con sus atributos propios.
- Si la interrelación pueda transformarse en una correspondencia N:M. [13]

En el siguiente modelo entidad- relación como mejor opción resultaría conservar la semántica original del modelo y guardar el atributo propio de la relación entre las entidades Comerciante y Pedido.

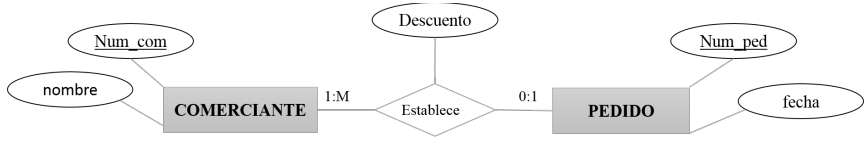


Figura. 32: Esquema ER con relación 1:N.

En una relación reflexiva, el método más común es el primero, lo que generaría una relación correspondiente a la misma entidad, con clave secundaria que haga relación con la clave principal de la misma entidad.

En el siguiente ejemplo se muestra que todo estudiante tiene un delegado y que un estudiante es delegado de 0, 1 o varios estudiantes.

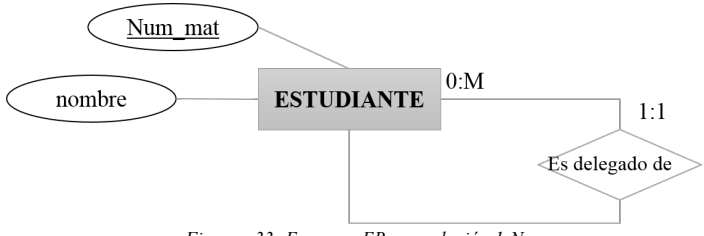


Figura. 33: Esquema ER con relación 1:N

Estudiante (Num_Mat, nombre, Num_Mat_Del)

Interrelaciones 1:1

Este tipo de interrelaciones pasan del modelo entidad – relación al modelo relacional con la transformación de sus entidades en relaciones y con sus atributos correspondientes.

Para representar esta interrelación solo bastaría añadir en cualquiera de las dos relaciones una clave secundaria que haga referencia a la otra relación. [14]

Para la transformación de esta interrelación hay dos opciones:

Primera

Delegación ⟶Nombre_del, nombre_ciu

Ciudad ⟶ Nombre_ciu

Segunda

Delegación ⟶ Nombre_del

Ciudad ⟶Nombre_ciu, Nombre_del

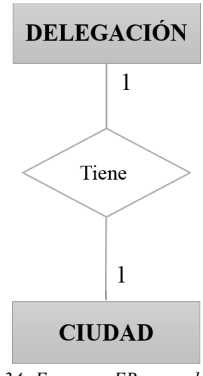
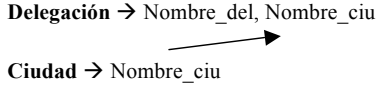


Figura. 34: Esquema ER con relación 1:1.



2.4.1.3. Transformación de interrelaciones de grado superior a 2

Una relación ternaria genera la creación de una nueva tabla que contiene los atributos propios de la relación, juntos con las claves principales de las tablas que se encuentran relacionadas. La nueva tabla creada tendrá como clave principal la vinculación de las claves secundarias de las entidades que forman parte de la relación.

Tenemos el siguiente modelo entidad – relación que muestra una interrelación ternaria.

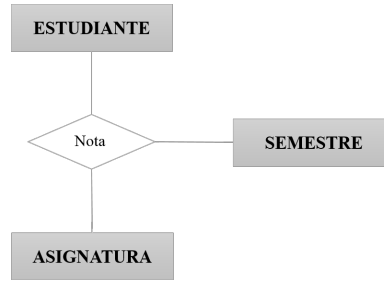


Figura. 35: Esquema ER con relación ternaria M:N:P

Cada una de las entidades del modelo entidad - relación se transformará en tablas y se creará una nueva tabla con las tres claves secundarias que referencian a las claves principales de las entidades asociadas, en la cual con la vinculación de las claves secundarias se tendrá la clave principal de la nueva tabla creada. [15]

2.4.2. Reglas concernientes al modelo entidad – relación extendido

2.4.2.1. Transformación de dependencias en existencia y en identificación

En primer lugar, tenemos la dependencia en existencia que son relaciones con cardinalidad 1, N, es decir que la entidad que posee la cardinalidad N, es una entidad débil. Lo que indica que no podrá haber registros si no existen registros en la entidad regular de la cual depende.

Así mismo tenemos la dependencia en identificación que de igual manera como la dependencia en existencia son relaciones con cardinalidad 1, N, pero la diferencia es que para poder identificar cada registro de la entidad frágil es necesario el AIP de la entidad frágil y de la entidad fuerte. [16]

Tenemos el siguiente esquema ER:

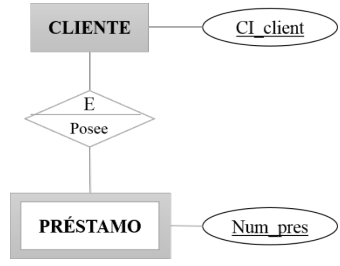


Figura. 36: Esquema ER con una dependencia en existencia

El esquema relacional es el que se presenta a continuación:
Cliente (CI, Nombre, Dirección)
Préstamo (Num_pres, CI_cliente)

2.4.2.2. Transformación de jerarquías de tipos y subtipos

Existen varias soluciones para la transformación de un modelo entidad - relación con entidades tipo y subtipos, entre las cuales tenemos:

- Agrupar los atributos de la entidad tipo y sus subtipos en una única relación. Las condiciones para esta solución son:
  1. Subtipos se diferencian entre sí en pocos atributos.
  2. Las interrelaciones que las vinculen sean las mismas para cada subtipo.
  3. La jerarquía es total o casi total.
  4. Prestaciones, mejor opción en cuanto a mayor velocidad en el acceso a los datos de un objeto.
- La segunda opción sería crear una relación para el supertipo y otra relación que agrupe todos los subtipos. Con esto es posible crear una única relación que abarque todos los atributos de las entidades subtipo y siendo la clave principal la misma de la entidad supertipo. Si tenemos una jerarquía que no es total, debemos dividir al menos en dos tablas la entidad supertipo y los subtipos. Si no se realiza dicha división tendremos problemas con valores nulos. Si deseamos mantener la semántica esta solución no es muy factible.
- La tercera opción será crear una relación para un supertipo y una relación para cada subtipo. Esta solución es la más adecuada cuando ninguno de los parámetros de la primera opción se cumple. Esta solución permite mantener la originalidad de la semántica. Para recuperar datos se deben crear varias tablas por lo que ofrecen menor velocidad en el acceso a datos en comparación con la segunda opción.
- La cuarta opción es crear una relación para cada subtipo que contenga tanto atributos propios como atributos comunes. Es muy adecuada ya que posee casi los mismos términos de la tercera opción, en cuanto acceso a datos suelen afectar a los atributos comunes.

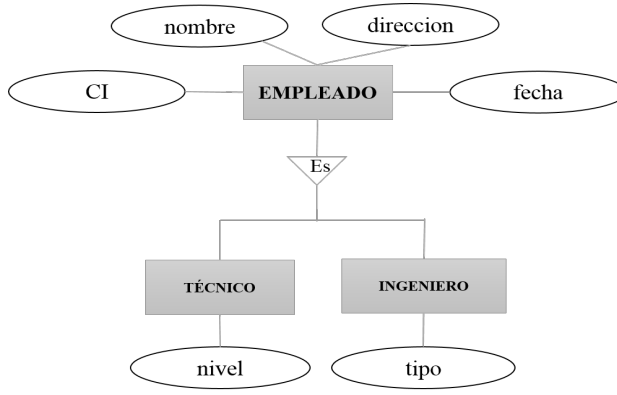


Figura. 37: Esquema ER con una jerarquía de tipos y subtipos

Opciones de paso al modelo relacional:
Opción A: Empleado (CI, nombre, fecha, dirección, nivel, tipo)
Opción B: Empleado (CI, nombre, fecha, dirección)

Datos Persona (CI, nivel, tipo)

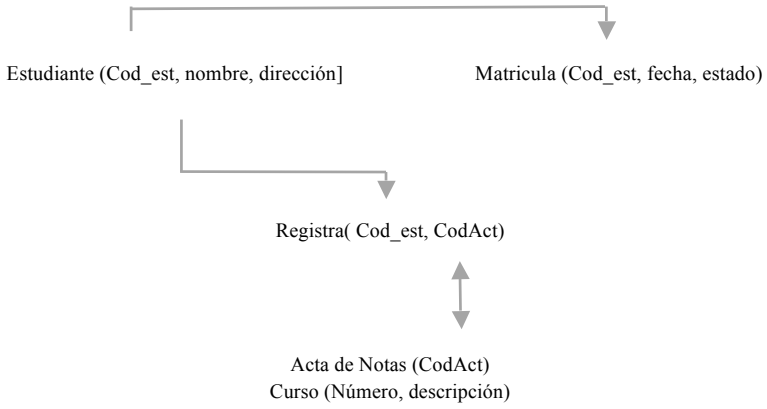
Opción c: Empleado (CI, nombre, fecha, dirección]

Técnico (CI, nivel] Ingeniero (CI, tipo)
Opción D: Técnico (CI, Nombre, nivel)

Ingeniero (CI, Nombre, Tipo)

2.4.2.3. Transformación de agregaciones

Las agregaciones del modelo entidad – relación al modelo relacional, se transforman como entidades. En el siguiente ejemplo tenemos las entidades Estudiante, Curso y Acta de Notas, las relaciones matrícula y registra han dado lugar a nuevas tablas que estarán formadas por las claves secundarias. [15]



2.5. Desarrollo de diversos supuestos prácticos de modelización mediante diagramas Entidad-Relación

Dentro del planteamiento de ejercicios a resolver del tema, se proponen varias prácticas de modelado de ER. Se ha elaborado una transformación para cada diagrama de la entidad. Se encuentran varias suposiciones en los ejercicios propuestos.

Resumen

- Como elementos básicos del modelo entidad- relación podemos nombrar a las entidades, dominios, atributos y las relaciones o interrelaciones.
- Se define como entidad al objeto sobre el cual se desea guardar información en una base de datos.
- Se entiende por relación a la asociación o vínculo entre dos o más entidades.
- Un atributo se define como las propiedades o características que están asociadas a una entidad.
- Un dominio es el valor permitido para cada atributo de una entidad.
- Se define como Cardinalidad al número de veces en que se asocia una entidad con otra.
- Para transformar una entidad del modelo entidad – relación al modelo relacional, cada una de las entidades pasaran como relaciones al modelo relacional.

Análisis del modelo relacional y de los elementos que lo integran

Descripción del capítulo

A continuación, se detalla el objetivo principal de cada contenido del capítulo 3, Análisis del modelo relacional y de los elementos que lo integran

El apartado 3.1 tiene por objetivo exponer de forma clara un concepto en lo que se refiere a una base de datos relacional y sus propiedades

El objetivo del apartado 3.2 es el de describir las de funciones de las claves que se ven presentes en un modelo relacional.

El apartado 3.3 tiene como objetivo general, exponer de forma clara las restricciones que se ven presentes en el diseño de un modelo relacional de las cuales son consideras dos tipos que son:

- Restricciones Inherentes
- Restricción semántica o de usuario

En el apartado 3.4 se tiene por objetivo el hablar sobre las nociones del álgebra relacional dando una descripción de la misma y sobre los distintos operadores que son utilizados:

- Los operadores tradicionales de la teoría de conjuntos
- Los operadores relacionales específicos

El objetivo principal del apartado 3.5 es precisamente el de dar a conocer las nociones de cálculo relacional por filas utilizando la ejemplificación mediante ejercicios prácticos esenciales.

El objetivo del apartado 3.6 consiste en exponer las nociones de cálculo relacional de dominio.

El apartado 3.7 tiene como objetivo explicar de forma clara la teoría

de la normalización y sus objetos, de lo cual se realizará una conceptualización de las dependencias funcionales, así como un análisis de las ventajas e inconvenientes que resultan de la aplicación tanto de la normalización como de la desnormalización mediante el uso de diferentes supuestos prácticos.

3.1. Concepto de relaciones y sus propiedades

En una base de datos relacional, los datos son recolectados mediante relaciones, y estas a su vez son generalmente representadas mediante tablas. En las cuales vemos involucrados conceptos de elementos que ya han sido mencionados en el capítulo anterior, tales como, tuplas (filas), atributos (columnas), grado (número de atributos de una relación), y cardinalidad (número de filas de una relación).
En síntesis, se puede afirmar que una base de datos relacional es un conjunto de tablas o relaciones identificadas por nombres únicos en las cuales las filas representan hechos (tupla) y las columnas representan las propiedades (atributos).

En esta sección se va a explicar un concepto de relación, visto desde un punto de vista más matemático y flexible. Siendo así que una relación R basado en un conjunto determinado de dominios (D1, D2, …Dn), es combinada en dos partes que comprenden una cabecera y un cuerpo.

Cabecera: Comprende un conjunto de n pares de atributo-dominio, en el cual n representa el grado y A representa los atributos, de tal manera que cada atributo A, pertenece únicamente a uno de los dominios D.

Cuerpo: Comprende un conjunto de m filas de las cuales cada fila representa a un conjunto de n pares atributo-valor: (A1:V1), (A2:V2) …, (An: Vn) (i=1…m).

D_1	D_2	D_3	(…)	D_n
A_1	A_2	A_3	(…)	A_n

Tabla 1: Definición matemática de relacion

El esquema representativo de una relación es R y su cabecera será la siguiente:

$$R \{(A_i: D_i)\}\ i=1..n$$

Además, el estado de una relación, se encuentra conformado por el esquema y el cuerpo de la misma, donde el cuerpo de una relación está comprendido por un conjunto determinado de filas que en su momento llegan a satisfacer un adecuado esquema de relación.

Por ejemplo, considerando una relación Artículos, de la cual su cabecera cuenta con tres atributos definidos en sus respectivos dominios de la siguiente manera:

- A1 (codigo_art), establecido sobre el primer dominio D1, el cual es una cadena con una longitud máxima de cinco caracteres.
- A2 (descrip_art), establecido sobre el segundo dominio D2, el cual es una cadena con una longitud máxima de cincuenta caracteres.
- A3 (pvp_art), establecido sobre el tercer dominio D3, el cual es un número con una longitud máxima de cuatro cifras, donde las dos últimas son decimales.

Articulos		
codigo_art	**descrip_art**	**pvp_art**
A0025	Teclado	8.00
A0026	Mouse Inalámbrico	13.5
A0027	Disco Duro Samsung	85.00
A0028	Impresora Canon	300.50
A0029	Monitor Samsung	375.50
A0030	Portátil HP	800.00

Tabla 2: Relación Artículos

De la relación Artículos, se puede observar que el cuerpo, está determinado por seis filas, de las cuales cada una cuenta de tres pares de atributo-valor. Tomando como ejemplo la primera fila de la relación Articulos se tiene la siguiente estructura: {(A1:V11), (A2:V12), (A3:V13)}, la cual hace referencia a los siguientes datos: {codigo_art: ´A0025´.], (descrip_art: "Teclado"), (pvp_art:8,00)}

Propiedades de una relación:

- Toda relación está identificada por un nombre que es único dentro de la base de datos relacional a la que pertenece.
- Es indispensable el valor de un atributo en una relación, además en la intersección de una fila y una columna se encuentra un solo valor.
- En una tabla los atributos tienen, y deben llevar nombres diferentes, más sin embargo es posible que en tablas diferentes dentro de una misma base de datos se encuentren atributos con nombres similares.
- En una relación los atributos no se suelen encontrar en orden.
- No es posible que en una tabla puedan existir dos filas idénticas
- Las filas en una relación no se suelen encontrar en orden.

Generalmente una relación es representada mediante su nombre, seguido de sus atributos entre paréntesis y estos a su vez separados por comas. De tal manera que la relación Artículos quedaría representada de siguiente forma:

Artículos (codigo_art, descrip_art, pvp_art)

3.2. Concepto de claves en el modelo relacional

Una clave candidata, consiste en una agrupación de atributos que identifican de manera única a cada fila perteneciente a la relación. Dicho de otra forma, en una relación no debe existir más de una fila identificada con el mismo valor de su clave aspirante. Considerando la relación Articulos de la Tabla 2, el atributo codigo_art vendría siendo la clave candidata, ya que no se podría asignar el mismo valor de este atributo a dos artículos diferentes.

Realmente en una relación es posible que exista más de una clave candidata, de las cuales es necesario reconocer entre la clave principal y las claves alternativas.

Clave principal: Es aquella que está destinada a identificar cada fila de una relación. En situaciones en las que solo existe una clave aspirante, esta es tomada directamente como una clave principal.

Una clave principal se la presenta subrayada o anteponiendo ante su nombre el símbolo “#”.

Así se puede representar la relación Artículo de estas dos maneras:

Articulo (#codigo_art, descrip_art, pvp_art)
Articulo (codigo_art, descrip_art, pvp_art)

Claves alternativas: Son todas aquellas claves candidatas que no han sido seleccionadas como clave principal, por ejemplo, en la relación Artículos, si en esta relación se supiera que no van a existir dos artículos con una misma descripción, se podría considerar el campo *descrip_art* como otra clave candidata teniendo así al atributo *codigo_art* como clave principal y el atributo *descrip_art* como una clave alternativa.

Existen otros tipos de claves en un modelo relacional, tales como las claves foráneas, las cuales corresponden a un conjunto de atributos no nulos de una relación cuyos valores deben tener concordancia con los valores de una clave candidata, que generalmente es una clave principal de otra relación, las cuales además deben de estar definidas en un mismo dominio.

Si además de la relación Artículos, se considera una relación Pedidos, la misma que contiene datos referentes a los pedidos propiamente, que han sido realizados por los clientes de una empresa, para lo cual por cada pedido realizado se almacenará un código *(referencia_ped)* y la fecha en la que se ha efectuado el pedido *(fecha_ped)*.

Teniendo en cuenta que la clave principal será el atributo referencia_ped, la representación de la relación Pedidos quedaría de la siguiente manera:

Pedidos (#referencia_ped, fecha_ped)
Pedidos (referencia_ped, fecha_ped)

Representado mediante un ejemplo, los datos que se podría almacenar en la tabla de la relación Pedidos.

Pedidos	
referencia_ped	**fecha_ped**
P011	10/09/2015
P012	11/10/2015
P013	12/11/2015
P014	13/12/2015

Tabla 3: Relación Pedidos

Continuando con el ejemplo anterior, se considerará una relación llamada Linea_pedidos, la misma que contendrá las líneas de pedidos de los encargados desarrollados por los clientes. En esta relación además se almacenará un tercer atributo denominado cantidad_art, el cual representará la cantidad de unidades por cada artículo considerado en el pedido. Además, esta nueva relación constara de una clave principal que está compuesta de los atributos referencia_ped y codigo_art. El cual se puede representar de la siguiente manera.

Linea_pedidos (referencia_ped, codigo_art, cantidad_art)

Linea_pedidos (#referencia_ped, #codigo_art, cantidad_art)

Representado mediante un ejemplo, los datos que se podría almacenar en la tabla de la relación Linea_pedidos.

Linea_pedidos		
referencia_ped	**codigo_art**	**cantidad_art**
P010	A0025	4
P011	A0026	12
P012	A0027	17
P013	A0028	20
P014	A0029	10
P014	A0030	5
P014	A0025	15

Tabla 4: Tabla Linea_pedidos

En la tabla *Linea_pedidos*, el atributo *referencia_ped*, es una clave foránea que hace referencia al atributo *referencia_ped* de la tabla Pedidos. Por tal motivo el valor que se asigne a este atributo en la tabla *Linea_pedidos*, deberá tener concordancia con uno de los valores que contenga el atributo *referencia_ped* de la tabla Pedidos.

La misma situación se repite con el atributo *codigo_art*, el cual vendría siendo una calve foránea de la tabla Artículos, por tal motivo, cada valor que tome el atributo *codigo_art* en la tabla *Linea_pedidos*, deberá tener concordancia con uno de los valores que corresponden al atributo *codigo_Art* de la tabla Articulo.

La representación de una clave foránea se la realiza mediante flechas, que van desde la clave foránea, hasta la cave principal corres-

pondiente. Por lo tanto, la base de datos que se ha creado mediante las tres tablas utilizadas hasta el momento, quedaría representada de la siguiente manera:

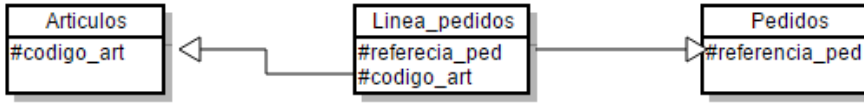


Figura. 38: Esquema relacional

3.3. Las restricciones en el modelo relacional

Una restricción trata de aquellas ocurrencias no permitidas en un modelo relacional de las cuales se pueden distinguir dos tipos, que son las restricciones inherentes y las restricciones semánticas.

3.3.1. Restricciones inherentes

Este tipo de restricciones son características que diversifican la relación de una tabla, las cuales se describen a continuación:

- No existen dos filas idénticas, por lo que es de carácter obligatorio contar con una clave principal.
- El orden tanto de las filas como de los atributos no tienen relevancia.
- Cada uno de los atributos solo pueden contar con un único valor de domino, por lo tanto, no se permiten los grupos iterativos o atributos multivaluados.

Por otro lado, existe otra limitación de este tipo que es justamente la regla de la integridad de una entidad la cual dice que “Ningún atributo que forme parte de la clave principal de una relación puede tomar un valor nulo”.

3.3.2. Restricciones semánticas o de usuario

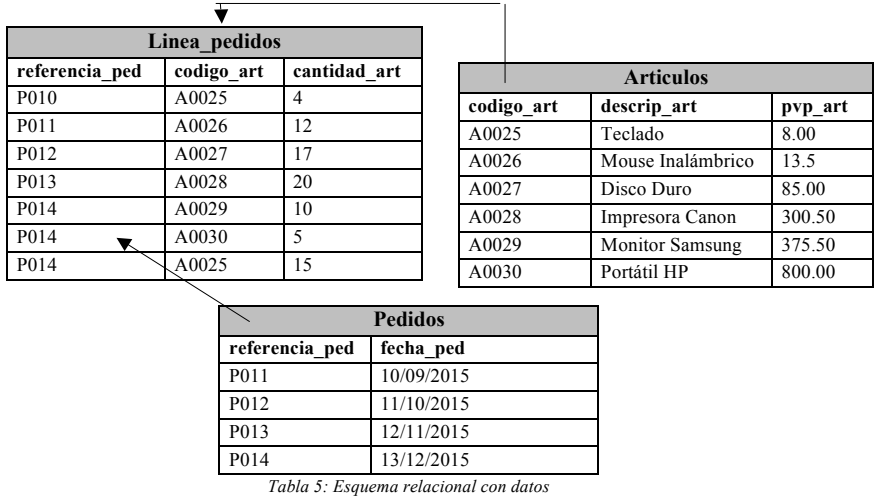
A continuación, se describirá las limitaciones semánticas o de usuario más importantes en un modelo relacional.

Clave principal: En una relación es posible estipular uno o varios atributos como clave principal para identificar cada fila de una tabla, sabiendo que los valores asignados a estos atributos no podrán redundar ni ser nulos.

Unicidad: Cuando se identifican atributos de los cuales sus valores no consiguen repetirse en una relación, esta limitación permite la definición de claves alternativas.

Obligatoriedad de uno o varios atributos: Con esta restricción se sugiere que los atributos no contengan valores nulos o inexistentes. Esta restricción no es realmente necesaria establecerse en cuanto a loa atributos de las claves principales, ya que en su mayoría los sistemas gestores de bases de datos ya cuentan con la restricción de integridad para las entidades, la cual se explica en el apartado 3.3.1.

Integridad referencial: Esta restricción indica que, si una relación consta de una clave foránea que hace referencia a la clave principal de otra relación, esta clave foránea podrá almacenar ya sea un valor nulo o únicamente alguno de los valores que estén contenidos en la clave principal de la relación a la que se está haciendo la referencia.
De tal manera el atributo *referencia_ped* definido en la relación *Linea_pedidos* siendo clave foránea de la relación *Pedidos*, contendrá o bien un valor nulo o bien uno de los valores correspondientes al atributo *referencia_ped* de la relación Pedidos y de igual manera ocurre con el atributo *codigo_art* en la relación *Linea_pedidos*, siendo este, clave foráneo de la relación *Artículos*, deberá contener valores que coincidan con los valores establecidos en la clave principal que es el atributo codigo_art de la relación Artículos.



Linea_pedidos

referencia_ped	codigo_art	cantidad_art
P010	A0025	4
P011	A0026	12
P012	A0027	17
P013	A0028	20
P014	A0029	10
P014	A0030	5
P014	A0025	15

Articulos

codigo_art	descrip_art	pvp_art
A0025	Teclado	8.00
A0026	Mouse Inalámbrico	13.5
A0027	Disco Duro	85.00
A0028	Impresora Canon	300.50
A0029	Monitor Samsung	375.50
A0030	Portátil HP	800.00

Pedidos

referencia_ped	fecha_ped
P011	10/09/2015
P012	11/10/2015
P013	12/11/2015
P014	13/12/2015

Tabla 5: Esquema relacional con datos

Otra acción importante a más de definir las claves foráneas de una relación, es la de determinar las consecuencias que conllevaría el hecho de borrar o editar las filas de una relación referenciada para lo cual se ha definido las siguientes opciones:

Operación restringida: Esta indica que no es permitido tanto la edición como la eliminación de una fila que esta referenciada a otra relación. Aplicando esta operación al ejemplo anterior, esto implicaría que para eliminar un artículo de la relación *Articulos*, no debería existir ningún registro (fila) en la tabla *Linea_pedidos* que contenga el artículo que se piensa eliminar, por otro lado, tampoco será posible modificar el atributo *codigo_art* de la relación Articulos mientras se tuviese algún registro en la relación *Linea_pedidos* que haga referencia a ese artículo.

Operación de transferencia en cascada: Esta operación indica que tanto la eliminación o modificación de una fila en la relación que contenga la clave principal, implicaría inmediatamente la eliminación o modificación en cascada de todas aquellas filas que contengan la clave foránea y que hacían referencia a la fila eliminada de la otra relación. Por ejemplo, si de la relación *Pedidos* se llegara a modificar el atributo *referencia_ped*, en la tabla *Linea_pedidos*, se deberá modificar todas las líneas de pedidos que hacían referencia al pedido modificado.

Operación con puesta a nulos: Esta operación indicaría que en caso de que se eliminara una fila de la relación que contiene la clave principal, inmediatamente modificaría el valor de la clave foránea en otra relación a un valor nulo. De tal manera en el ejemplo anterior, si se llegara a eliminar un registro (fila) de la relación *Artículos*, se modificaría el valor a nulo del atributo *codigo_art* de la relación Linea_pedidos de todas aquellas filas cuyo valor del atributo c*odigo_art* hacían referencia a la fila del artículo eliminado. Sin embargo, esta operación no es válida en este ejemplo debido a que el atributo *codigo_art* en la relación *Linea_pedidos* no puede tomar un valor nulo o inexistente ya que anteriormente ya se había establecido que dicho atributo formaría parte de la clave principal de la relación *Linea_pedidos.*

Operación con puesta de valor por defecto: Esta operación tiene un comportamiento similar a la operación anterior, con la diferencia de que, en este caso en lugar de establecer un valor nulo o inexistente para el campo de la clave foránea de una relación, se establece un valor predeterminado o por defecto para tal atributo.

En un modelo relacional existen otras limitaciones, a las cuales se puede llamar de comprobación o de rechazo y aserciones. En estas dos limitaciones el usuario debe determinar un predicado en base a uno o varios atributos de una relación, de esta forma cada que se efectué una operación de modificación a través de la base de datos, primero se verificará si se cumple con el predicado estipulado, y de no ser así la ejecución de la operación no será autorizada. Realmente la diferencia entre estas dos limitaciones es que en la de comprobación únicamente se verá afectada una relación, mientras que en la de rechazo y aserciones se verán afectadas varias relaciones.

3.4. Nociones de álgebra relacional

El álgebra relacional es un lenguaje formal de consultas a través de bases de datos relacionales que compone la base del lenguaje de consulta SQL. El álgebra relacional es un lenguaje de consulta procedimental que está formada por un conjunto de operadores que retornan una o más relaciones como manejando y que causan otra relación como resultado. Se dispone de dos grupos de operadores que son los operadores tradicionales de la teoría de conjuntos y los operadores relacionales concretos.

3.4.1. Operadores tradicionales de la teoría de conjuntos

Generalmente todo par de tablas (relaciones), necesitan ser relacionadas con respecto a la unión, es decir que es necesario que sean de un mismo grado y que sus atributos deben corresponder a un mismo dominio. En esta sección se utilizará las siguientes relaciones para los ejemplos posteriores.

Empleados 1		
codigo_emp	**nombre_emp**	**Cargo**
E001	Marlon Rodríguez	Jefe
E002	Marcos Cevallos	Director
E003	Jonathan Pereira	Analista
E004	Andrés Flores	Vendedor

Empleados 2		
codigo_emp	**nombre_emp**	**Cargo**
E010	Cristian Quezada	Contador
E011	Oliver Añasco	Vendedor
E004	Andrés Flores	Vendedor

Tabla 6: Ejemplo de relaciones para el álgebra relacional

Operaciones de la teoría de conjuntos:

Unión: La unión de las relaciones A y B (AUB) es una relación en la que se ven contenidas todas las filas de las relaciones A y B. y que en caso de existir redundancia se eliminan.

Empleados1 U Empleados 2		
codigo_emp	**nombre_emp**	**Cargo**
E001	Marlon Rodríguez	Jefe
E002	Marcos Cevallos	Director
E003	Jonathan Pereira	Analista
E004	Andrés Flores	Vendedor
E010	Cristian Quezada	Contador
E011	Oliver Añasco	Vendedor

Tabla 7: Unión de dos relaciones

Intersección: Esta relación consiste en contener todas las filas que se encuentran tanto en la relación A como en la relación B a la vez.

Empleados 1 ∩ Empleados 2		
codigo_emp	**nombre_emp**	**Cargo**
E004	Andrés Flores	Vendedor

Tabla 8: Intersección de dos relaciones

Diferencia: En una diferencia entre relaciones, (A - B) es una relación en la que se ven contenidas todas aquellas filas que se encuentran en la relación A pero que no tienen correspondencia con la relación B.

Empleados 1 - Empleados 2		
codigo_emp	**nombre_emp**	**Cargo**
E001	Marlon Rodríguez	Jefe
E002	Marcos Cevallos	Director
E003	Jonathan Pereira	Analista

Tabla 9: Diferencia entre dos relaciones

Producto cartesiano: Entre dos relaciones A y B, el producto cartesiano (AxB), corresponde a una relación en la que se ven involucrados todas las filas posibles que se obtengan de relacionar una fila de A con una fila de B. Considerando que si la relación A tiene un grado m y la relación B tiene un grado n, la relación de (AxB) tendrá un grado de (m+n). Mientras tanto si la cardinalidad de la relación A es p y la cardinalidad de la relación B es q, la cardinalidad de la relación de (AxB) será (pxq).

Empleados			
codigo_emp	**nombre_emp**	**Cargo**	**codigo_dep**
E001	Marlon Rodríguez	Jefe	4
E002	Marcos Cevallos	Director	6
E003	Jonathan Pereira	Analista	6
E004	Andrés Flores	Vendedor	8

Departamentos		
codigo_dep	**nombre_dep**	**ciudad**
4	Sistemas	Machala
6	Financiero	Arenillas
8	Ventas	Santa Rosa

Tabla 10: Relaciones para ejemplos posteriores

Empleados x Departamentos						
codigo_emp	**nombre_emp**	**cargo**	**codigo_dep**	**codigo_dep**	**nombre_dep**	**ciudad**
E001	Marlon Rodríguez	Jefe	4	4	Sistemas	Machala
E001	Marlon Rodríguez	Jefe	4	6	Financiero	Arenillas
E001	Marlon Rodríguez	Jefe	4	8	Ventas	Santa Rosa
E002	Marcos Cevallos	Director	6	4	Sistemas	Machala
E002	Marcos Cevallos	Director	6	6	Financiero	Arenillas
E002	Marcos Cevallos	Director	6	8	Ventas	Santa Rosa
E003	Jonathan Pereira	Analista	6	4	Sistemas	Machala
E003	Jonathan Pereira	Analista	6	6	Financiero	Arenillas
E003	Jonathan Pereira	Analista	6	8	Ventas	Santa Rosa
E004	Andrés Flores	Vendedor	8	4	Sistemas	Machala
E004	Andrés Flores	Vendedor	8	6	Financiero	Arenillas
E004	Andrés Flores	Vendedor	8	8	Ventas	Santa Rosa

Tabla 11: Producto cartesiano de dos relaciones

3.4.2 Operadores relacionales específicos

En el álgebra relacional se defienden los siguientes operadores:
Operador de Selección o Restricción (G): Este operador algebraico, produce un subconjunto horizontal de una relación existente, el mismo que comprende todas las filas de la relación para las cuales se realiza un predicado detallado. Considerando las relaciones del ejemplo anterior, si se requiere seleccionar todos los empleados que pertenecen al departamento código es 6, la expresión es la siguiente:

σ codigo_dep=6 (Empleados)

Y su resultado será el siguiente:

Empleados			
codigo_emp	**nombre_emp**	**cargo**	**codigo_dep**
E002	Marcos Cevallos	Director	6
E003	Jonathan Pereira	Analista	6

Tabla 12: Selección sobre una relación

Operador de proyección *(π)*: Este operador produce un subconjunto vertical de una relación ya existente, el mismo que resulta de seleccionar los atributos, especificados en un orden conveniente de izquierda a derecha, para luego eliminar las filas copiadas.
Por ejemplo, para conseguir los cargos de los empleados en la relación Empleados, el resultado de la proyección seria la siguiente:

Empleados
cargo
Jefe
Director
Analista
Vendedor

Tabla 13: Proyección sobre una relación

De igual manera para obtener los nombres y los cargos de cada empleado que labora en el departamento cuyo código (codigo_dep) es 6, la expresión algebraica será la siguiente:

π nombre_emp, cargo (σ codigo_dep= 6 (Empleados))

De la cual se obtendrá como resultado lo siguiente:

Empleados	
nombre_emp	**Cargo**
Marcos Cevallos	Director
Jonathan Pereira	Analista

Tabla 14: Selección y proyección sobre una relación

Operador división (/): Este operador se encarga de dividir una relación A con un grado m, entre una relación B con grado n, de lo cual se obtiene como resultado una relación con grado mn es decir, que si la relación A tiene un grado 2 y la relación B un grado 1, la relación de división A/B dará como resultado un conjunto de aquellas filas <x> que en el instante en que se unan con las filas <y> de la relación B produzcan filas contenidas en A <x,y>.

Además, si se efectuara un producto cartesiano a la relación A/B y la relación B se obtendría como resultado un conjunto de filas que corresponden a la relación A.

$$A/B \times B \quad A$$

Por ejemplo, si se realiza la división entre la relación Empleados, la misma que tiene un grado 4, y la relación Departamentos de grado 1. La relación Empleados/Departamentos tendrá un grado 3, y estará conformada por todos aquellos atributos que pertenecen a la relación Empleados menos los atributos pertenecientes a la relación Departamentos y a su vez presentara únicamente las filas de empleados que laboran en el departamento 6.

Departamentos
codigo_dep
6

Empleados			
codigo_emp	**nombre_emp**	**Cargo**	**codigo_dep**
E001	Marlon Rodríguez	Jefe	4
E002	Marcos Cevallos	Director	6
E003	Jonathan Pereira	Analista	6
E004	Andrés Flores	Vendedor	8

Tabla 15: Relaciones a utilizar en la división

Empleados/ Departamentos			
codigo_emp	**nombre_emp**	**cargo**	**codigo_dep**
E002	Marcos Cevallos	Director	6
E003	Jonathan Pereira	Analista	6

Tabla 16: División de las relaciones

Operador de combinación (*): De la combinación entre una relación A y una relación B, se obtiene una relación en la que constan todas las filas que resultan de enlazar una fila de la relación A y otra fila de

la relación B, con el objetivo de que estas desempeñen una condición específica para los valores correspondientes a un atributo de dominio habitual en ambas relaciones. Es por esto que usualmente se realiza primero el producto cartesiano de las dos relaciones para luego realizar una selección de aquellas filas que desempeñen la condición establecida.

Empleados		
codigo_emp	**nombre_emp**	**cargo**
E001	Marlon Rodríguez	Jefe
E002	Marcos Cevallos	Director
E003	Jonathan Pereira	Analista
E004	Andrés Flores	Vendedor

Departamentos		
codigo_dep	**nombre_dep**	**Ciudad**
4	Sistemas	Machala
6	Financiero	Arenillas
8	Ventas	Santa Rosa

Tabla 17: Relaciones de ejemplo para la combinación

De las relaciones anteriores, al efectuar la combinación, queda de tal manera que el atributo codigo_dep de la tabla Empleados, tenga un mismo valor que su atributo correspondiente en la tabla Departamentos, lo cual quedaría expresado de la siguiente forma:

(Empleados * Departamentos) Empleados codigo_dep =Departamentos codigo_dep

Cabe mencionar que para realizar esta operación es necesario que primeramente se efectúe un producto cartesiano, de lo cual se obtiene como resultado lo siguiente:

Empleados * Departamentos						
codigo_emp	**nombre_emp**	**Cargo**	**codigo_dep**	**codigo_dep**	**nombre_dep**	**ciudad**
E001	Marlon Rodríguez	Jefe	4	4	Sistemas	Machala
E002	Marcos Cevallos	Director	6	4	Sistemas	Machala
E003	Jonathan Pereira	Analista	6	4	Sistemas	Machala
E004	Andrés Flores	Vendedor	8	4	Sistemas	Machala

Tabla 18: Producto cartesiano

Empleados X Departamentos						
codigo_emp	**nombre_emp**	**Cargo**	**codigo_dep**	**codigo_dep**	**nombre_dep**	**ciudad**
E001	Marlon Rodríguez	Jefe	4	4	Sistemas	Machala
E001	Marlon Rodríguez	Jefe	4	6	Financiero	Arenillas
E001	Marlon Rodríguez	Jefe	4	8	Ventas	Santa Rosa
E002	Marcos Cevallos	Director	6	4	Sistemas	Machala
E002	Marcos Cevallos	Director	6	6	Financiero	Arenillas
E002	Marcos Cevallos	Director	6	8	Ventas	Santa Rosa
E003	Jonathan Pereira	Analista	6	4	Sistemas	Machala
E003	Jonathan Pereira	Analista	6	6	Financiero	Arenillas
E003	Jonathan Pereira	Analista	6	8	Ventas	Santa Rosa
E004	Andrés Flores	Vendedor	8	4	Sistemas	Machala
E004	Andrés Flores	Vendedor	8	6	Financiero	Arenillas
E004	Andrés Flores	Vendedor	8	8	Ventas	Santa Rosa

Tabla 19: Combinación o reunión

Luego será necesario, tomar en cuenta únicamente aquellas filas que cumplen directamente con el predicado establecido, es decir, que el atributo codigo_dep de la relación Empelados, tome el mismo valor que el atributo codigo_dep de la relación Departamentos.

En esta situación se observa que es una comparación equivalente, es decir se trata de una combinación por igualdad. La combinación que se suele dar con mayor frecuencia entre relaciones o tablas es la denominada join o también conocida como reunión natural, el cual no es más que el resultado de la eliminación de los atributos duplicados de una combinación por igualdad.

El operador que representa esta combinación es |x|, por lo tanto, la reunión natural entre las relaciones A y B se representa de la siguiente forma: AxB

Por lo tanto, de estas relaciones se seleccionarán todas aquellas filas que contengan valores equivalentes en sus respectivas columnas de igual dominio, es decir tanto la clave foránea y la clave principal, y se suprimen las que se consideran iguales.

Por ejemplo, en una reunión natural dada entre las relaciones Empleados y Departamentos tendría que suprimir una de las columnas codigo_dep, tal como se muestra a continuación.

Empleados * Departamentos						
codigo_ emp	**nombre_emp**	**cargo**	**codigo_ dep**	**codigo_d ep**	**nombre_ dep**	**ciudad**
E001	Marlon Rodríguez	Jefe	4	4	Sistemas	Machala
E002	Marcos Cevallos	Director	6	4	Sistemas	Machala
E003	Jonathan Pereira	Analista	6	4	Sistemas	Machala
E004	Andrés Flores	Vendedor	8	4	Sistemas	Machala

Empleados * Departamentos					
codigo_emp	**nombre_emp**	**cargo**	**codigo_ dep**	**nombre_dep**	**ciudad**
E001	Marlon Rodríguez	Jefe	4	Sistemas	Machala
E002	Marcos Cevallos	Director	6	Sistemas	Machala
E003	Jonathan Pereira	Analista	6	Sistemas	Machala
E004	Andrés Flores	Vendedor	8	Sistemas	Machala

Tabla 20: Combinación Natural

3.5. Nociones de Cálculo relacional de filas para resolver ejercicios prácticos esenciales

En el álgebra relacional se exponen todos aquellos procedimientos que son precisos realizar para obtener el resultado de una consulta. Por otro lado, en el caso del cálculo relacional de las filas se refiere a la utilización de un lenguaje no procedimental en el que se muestra directamente la información deseada sin necesidad de exponer el procedimiento que se lleva a cabo para lograr la obtención de dicha información. Este tipo de consultas en el cálculo relacional de las filas se expresa o se enuncian de la siguiente manera:

$$\{t \mid P(t)\}$$

Dada esta estructura, se entiende que el conjunto de fila t al igual que el predicado propuesto P, es cierto para t. De lo cual se muestra t.A o t|A| al valor del atributo A para la fila t y t ϵ r para indicar que la fila t está en la relación r.

Para demostrar el cálculo relacional de las filas, se ejemplificará utilizando las relaciones Empleados 1 y Empleados 2 indicadas ya en ejemplos anteriores.

Empleados 1		
codigo_emp	**nombre_emp**	**cargo**
E001	Marlon Rodríguez	Jefe
E002	Marcos Cevallos	Director
E003	Jonathan Pereira	Analista
E004	Andrés Flores	Vendedor

Empleados 2		
codigo_emp	**nombre_emp**	**cargo**
E010	Cristian Quezada	Contador
E011	Oliver Añasco	Vendedor
E004	Andrés Flores	Vendedor

Tabla 21: Ejemplo de relaciones para el cálculo relacional de filas:

Por lo tanto, si se desea conseguir los datos correspondientes tanto a la relación Empleados 1 y Empelados 2, se utilizará el operador v(o). Lo que se entiende como el conjunto de filas que se encuentren tanto en la relación Empleados 1 como en Empleados 2.

$$\{t \mid (t \in \text{Empleados 1}) \text{ v } (t \in \text{Empleados2})\}$$

Por otro lado, para obtener las filas únicamente de los empleados que sean comunes entre las relaciones Empleados 1 y Empleados 2, es necesario utilizar el operador Λ(y), de la misma forma en que se demuestra en la siguiente expresión:

$$\{t \mid (t \in \text{Empleados 1}) \Lambda (t \in \text{Empleados2})\}$$

Para obtener los datos que correspondan a la relación Empleados 1 pero que no se encuentren en la relación Empleados 2, se debe utilizar el operador ¬ (no) tal cual se denota en la siguiente expresión.

$$\{t \mid (t \in \text{Empleados 1}) \Lambda \neg (t \in \text{Empleados2})\}$$

Considerando la relación Empleados y la relación Departamentos.

Empleados		
codigo_emp	**nombre_emp**	**Cargo**
E001	Marlon Rodríguez	Jefe
E002	Marcos Cevallos	Director
E003	Jonathan Pereira	Analista
E004	Andrés Flores	Vendedor

Departamentos		
codigo_dep	**nombre_dep**	**ciudad**
4	Sistemas	Machala
6	Financiero	Arenillas
8	Ventas	Santa Rosa

Tabla 22: Ejemplo de relaciones para el cálculo relacional de fila

Si se desea obtener las filas de la relación Empleados que pertenezcan únicamente a departamento cuyo valor de codigo_dep es igual a 1, su expresión quedaría demostrada de la siguiente manera:

$$\{t \mid (t \in \text{Empleados}) \Lambda (t.\text{codigo_dep}=1)\}$$

Para obtener una proyección en base a una relación sobre uno o varios atributos, es conveniente emplear el constructor "existe" de la logia matemática ($\exists$). Por ejemplo, si se desea obtener todos los cargos que se encuentren asignados a los empleados de la relación Empleados, sería necesario emplear la siguiente expresión.

$$\{t \mid \exists\, s \in \text{Empleados}\ (t.cargo = s.cargo)\}$$

De esta manera la expresión anterior describe a todo el conjunto de filas t que existen en la relación Empleados cuyo valor en t tiene correspondencia con los valores de s.

Para obtener tanto el nombre como el cargo de cada empleado que se encuentre laborando en el departamento cuyo valor en el atributo codigo_dep sea 1, se utiliza la siguiente expresión:

$$\{t \mid \exists\, s \in \text{Empleados}\ ((t.nombre_emp = s.nombre_emp) \wedge (t.cargo = s.cargo) \wedge (t.codigo_dep = 1))\}$$

Por otro lado, para realizar la combinación natural entre las relaciones Empleados y Departamentos los términos para el cálculo relacional resultan aún más complejos, quedando su expresión planteada de la siguiente manera:

$$\{t \mid \exists\, s \in \text{Empleados}\ ((t.nombre_emp = s.nombre_emp) \wedge (t.nombre_emp = s.nombre_emp)\ (t.cargo = s.cargo) \wedge (t.nombre_emp = s.nombre_emp) \wedge \exists\, u \in \text{Departamentos}\ ((s.codigo_dep = codigo_dep) \wedge (t.nombre_emp = u.nombre_emp) \wedge (t.nombre_emp = u.nombre_emp) \wedge (t.ciudad = u.ciudad)))\}$$

En esta expresión se utilizan dos constructores para hacer referencia a las filas en ambas relaciones (tablas), además se estipulan condiciones unidas por el operador (y). De las cuales las primeras son utilizadas con el fin de que las filas t del resultado comprendan todos los atributos de la relación Empleados. La condición siguiente, es la que vincula ambas relaciones mediante la clave foránea en relación a la clave principal codigo_dep. Y las dos últimas condiciones se encargan de que las filas t del resultado comprendan todos los atributos de la relación Departamentos.

Luego de ya haber comprobado cómo se formulan ciertas consultas en base al cálculo relacional de filas, es posible dar una definición más formal a este lenguaje, de tal manera que las expresiones del cálculo relacional de filas se obtienen partiendo de la siguiente forma:

$$\{t \mid P\ (t)\}$$

De lo anterior se describe a P como la fórmula del cálculo relacional, de la cual surgen distintas variables de las filas, Siendo estas variables libres mientras no sean consideradas mediante $\exists$ o $\forall$, como se demuestra a continuación:

{t | (t ϵ Empleados 1) v (t ϵ Empleados2)}

t, correspondería a una variable libre, sin embargo, en la siguiente expresión la variable s es considerada una variable unida ya que se encuentra seguida del símbolo $\exists$.

{t | $\exists$ s ϵ Empleados (t.cargo = s.cargo)}

Las formulas del cálculo relacional de filas se construyen mediante átomos, los cuales pueden tener una de las siguientes formas:

$t \epsilon r$, de las cuales t es una variable fila y r es una relación. No está permitido la utilización del operador.

t.A op s.B, de las cuales t y s son variables de fila, A representa un atributo de la relación en la que se encuentra determinada t, B representa a un atributo que pertenece a la relación en la que se encuentra determinada s, op es un operador de comparación (<, , >, , =, ≠) y los atributos A y B tiene dominios mientras sus miembros sean comparables.

t.A op c, donde t viene siendo una variable de fila, A es un atributo que pertenece a la relación en que se encuentra determinada t, op es un operador de comparación y c es una constante en el dominio de A.

Como ya se había mencionado anteriormente, estas fórmulas se construyen partiendo de átomos, para lo cual es necesario seguir las siguientes normas.

- Cada átomo es una fórmula
- Si P es considerado una fórmula, entonces ¬P y P, también lo son.
- Si P1, y P2 Son fórmulas, entonces también lo son P1 P2, P1 v P2 y P1⇒ P2
- Si P(s) es una fórmula que contiene una variable fila libre, s y r son relaciones

$$\exists s \in r\,(P(s)) \text{ y } \forall s \in r\,(P(S))$$

En el cálculo relacional de filas, se puede escribir términos e, utilizando distintas nomenclaturas, para lo cual se consideran las siguientes normas.

- P1 P2 es equivalente a ¬ (¬ (P1) v ¬ (P2)).

- $\forall t \in r\ (P(t))$ es equivalente a $\neg \exists t \in r\ (\neg P(t))$.
- $P1 \Rightarrow P2$ es equivalente a $\neg (P1) \vee P2$.

3.6. Nociones de Cálculo relacional de Dominios

Existe una forma diferente de cálculo relacional conocida como calculo relacional de dominós, en el cual se utilizan variables de dominio propiamente, las mismas que adquieren valores del domino de un atributo en vez de obtenerlas de una fila completa, y sus términos se representan de la siguiente manera:

$$\{< Z1, Z2, \ldots, Zn > I\ P(Z1, Z2, \ldots, Zn)\}$$

En la expresión anterior Z1, Z2, …, Zn, representan variables de dominio mientras que P, corresponde a una formula conformada por átomos, similar a lo que se presentó en el caso del caculo relacional de filas, y se deben presentar de la siguiente manera:

$$< Z1, Z2, \ldots, Zn > \in r$$

De la cual r es una relación que contiene n atributos y Z1, Z2, …, Zn , son las variables o constantes de domino.

z op y, de las cuales z e y, son las variables de dominio, mientras que op es un operador de comparación (<, , >, , =, ≠), además los atributos z e y, tienen dominios miembros son semejantes mediante op.

z op c, de las cuales z es una variable de dominio, mientras que op es un operador de comparación y por otro lado c es una constante del dominio del atributo para que z sea una variable de dominio.

Cada formula se construye partiendo de átomos, considerante antes las siguientes reglas:

- Un átomo es realmente una fórmula.
- Si P es una fórmula, de igual manera los serán $\neg P$ y (P)
- Si P1, y P2 Son fórmulas, de igual modo lo son P1 P2, $P1 \vee P2$ y $P1 \Rightarrow P2$
- P(z) es una fórmula que involucra una variable de dominio z, también son fórmulas

$$\exists x\ (P(z)) \text{ y } \forall x\ (P(z))$$

A continuación, se demostrarán las expresiones de cálculo relacional de domino aplicado a las consultas que fueron utilizadas para el cálculo relacional de las filas.

Para obtener los datos de las filas de la relación Empleados 1 conjun-

tamente con los de la relación Empleados 2, es necesario utilizar el operador v(o), de tal manera que la expresión necesaria quedaría expresada de la siguiente forma:

{<u, o, p> I <u, o, p> ∈ Empleados 1 v <u, o, p> ∈ Empleados 2)}

Si se desea obtener los datos de las filas correspondientes a los empleados frecuentes tanto en la relación Empelados 1 y Empleados 2, se usará el operador (y), por lo cual la expresión a utilizar seria la siguiente:

{<u, o, p> I <u, o, p> ∈ Empleados 1 <u, o, p> ∈ Empleados 2)}

Si se desea obtener los datos correspondientes a las filas de los empleados de la relación Empleados 1 que no se encuentren contenidos en la relación Empleados 2, se utiliza el operador ¬(no), de lo cual quedaría constituida la siguiente expresión.

{<u, o, p> I <u, o, p> ∈ Empleados 1 ¬ (<u, o, p> ∈ Empleados 2))}

Considerando que se desea obtener los datos de las filas de la relación Empleados 1 cuyo valor del atributo codigo_dep sea 1, se utiliza la siguiente expresión:

{<u, o, p> I <u, o, p> ∈ Empleados 1 d=1)}

Si se desea obtener todos los cargos que desempeñan los empleados registrados en la relación Empleados 1, se debe utilizar el operador de existe (∃), por lo que su expresión quedaría estructurada de la siguiente manera:

{<p> I ∃ u, o, d (<u, o, p, d> ∈ Empleados 1)}

Si se desea obtener tanto el nombre como el cargo de los empleados que laboran en el departamento cuyo valor del campo codigo_dep es 1, es necesario utilizar la siguiente expresión.

{<o, p> I ∃ u, d (<u, o, p, d> ∈ Empleados 1 d=1)}

Ahora para realizar una combinación natural entre las filas de la relación Empleados 1 y la relación Departamentos, la expresión del cálculo relacional se logrará de la siguiente manera:

{<ue, oe, p, nd, id, od, c> I ∃ nd(<ue, oe, p, nd> ∈ Empleados 1 ∃ od, c(<nd, od, c>∈ Departamentos))}

2.7. Teoría de la normalización y sus objetos

Diseñar una base de datos consiste en representar y personalizar un escenario del mundo real, mediante la utilización de objetos y reglas

que son proporcionados por el mismo modelo de datos que se esté utilizando, como la prohibición de determinadas agrupaciones o la posibilidad de introducir algunas restricciones.

Cuando se realiza el diseño de una base de datos, utilizando un modelo relacional, se pueden obtener distintas soluciones desarrolladas en diferentes esquemas relacionales, que, aunque no todos tengan similitudes, algunos se representan con mayor realismo que otros.

En esta sección se expondrá la teoría de la normalización, la cual propone enfrentar y resolver problemas de bases de datos de forma objetiva, detallando como llevar a cabo la normalización en un diseño de base de datos relacional, dando a conocer las características y propiedades con las que debe contar el diseño relacional para lograr una personificación apropiada con la realidad además de cuáles son los inconvenientes que se pueden presentar al realizar un mal diseño.

Considerando una relación denominada Pedidos, que almacene datos referentes a los pedidos que son realizados por clientes, en el cual se registran una referencia y una fecha de pedido, los mismos que son representados por los campos referencia_ped y fecha_ped, además que los artículos que se requieren para cada pedido se almacenan en la relación Articulos considerando como datos su código, descripción, número de unidades requeridas y el precio unitario, los cuales so representados por los siguientes campos respectivamente: codigo_art, descrip_art, cantidad_art y pvp_art. La clave principal en esta relación está determinada por el encadenamiento de los atributos referencia_ped y codigo_art. Sin embargo, esta relación presenta varios problemas de los cuales se menciona los siguientes:

- Existe una gran cantidad de datos repetidos, debido a que la fecha del pedido se estaría repitiendo si en cada pedido se requieren diferentes artículos, de igual manera si un mismo artículo es requerido en varios pedidos se repetirán los atributos descrip_art y pvp_art.
- Alteraciones de inserción, debido a que no es aceptable incluir en una base de datos, los datos de ciertos artículos que no existan en algún pedido, luego de establecer el atributo referencia_ped parte de la clave principal de la relación. Además, debido a la regla de integridad de una entidad, ningún atributo que sea parte de la clave principal puede tener un valor nulo.

Por tanto, al ingresar un pedido que involucre varios artículos requerirá que se incluyan varias filas en la base de datos.

- Alteraciones en las modificaciones, ya que si se intentara modificar el precio o la descripción de cualquiera de los artículos que ya haya sido registrado en algún pedido, se lo tendría que hacer en diferentes filas de la relación para evitar que se produzca inestabilidades. De la misma manera sucedería si se pretendiera modificar la fecha de un pedido en el que se han registrado varios artículos.
- Alteraciones de eliminación, esto se produce ya que, si se intenta eliminar un pedido de la base de datos y así mismo un artículo que esté vinculado a ese pedido, se eliminarían de esta relación todas las filas de artículos que son requeridos en dicho pedido. De igual manera si se necesita eliminar algún artículo que se encuentren vinculados a diferentes pedidos, se debería de eliminar una línea por cada pedido.

Pedidos					
referencia_ped	**fecha_ped**	**codigo_art**	**descrip_art**	**cantidad_art**	**pvp_art**
P001	10/09/2015	A0025	Teclado	4	8.00
P001	11/10/2015	A0026	Mouse Inalámbrico	12	13.5
P002	12/11/2015	A0027	Disco Duro Samsung	17	85.00
P003	13/12/2015	A0028	Impresora Canon	20	300.50
P004	10/09/2015	A0029	Monitor Samsung	10	375.50
P004	11/10/2015	A0030	Portátil HP	5	800.00
P004	12/11/2015	A0025	Teclado	15	8.00

Tabla 23: Relación Pedidos

En este tipo de relación se demuestra todos estos problemas ya que se guardan situaciones diferentes (pedidos y artículos) en una relación semejante.

Existe un procedimiento formal que se adapta a cualquier esquema relacional que se apega a la realidad, y de no ser así, nos muestra la manera para lograr que sea lo más idéntico posible del mundo real.

El procedimiento mencionado anteriormente es la teoría de la normalización, ya que facilita la realización de esquemas relacionales sin redundancias, por lo tanto, no presentan los problemas que se habían

descrito anteriormente.

Si se hubiera realizado un proceso de normalización a la relación anterior, se hubiera tenido en cuenta el siguiente diseño.

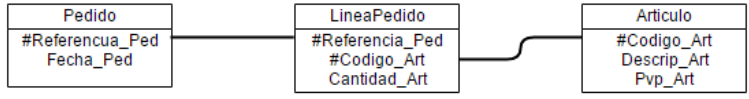


Figura. 39: Diseño de modelo relacional

En diseño relacional anterior se han guardado diferentes hechos en relaciones desiguales.

Se puede establecer a la teoría de la normalización como una técnica formal, que permite la organización de los datos que pueden ayudar a determinar los errores en un diseño relacional y de igual manera presente la manera de corregirlos.

La teoría de la normalización está fundamentada bajo el concepto de forma normal.

Se considera que un diseño relacional de base de datos se encuentra dentro de los parámetros de una forma normal definida si este cumple con un conjunto específico de limitaciones.

Realmente hay seis formas normales:

- Primera forma normal
- Segunda forma normal
- Tercera forma normal
- Forma normal de Boyce/Codd
- Cuarta forma normal
- Quinta forma normal

En la siguiente figura se podrá apreciar que algunas relaciones se encuentran en primera forma normal, de aquellas algunas se hallan en segunda forma normal, de estas únicamente una porción está en tercera forma normal, y así sucesivamente; es decir que en la segunda forma normal se proporciona más limitaciones que en la primera forma normal, la tercera forma normal más que la segunda forma normal, etc. Por tanto, la quinta forma normal es la que asigna las limitaciones más fuertes.

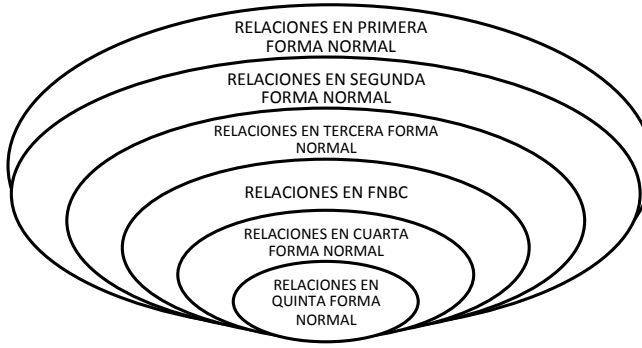


Figura. 40: Representación de las formas normales

Para llegar a comprender las formas normales, es necesario que previamente se conozca el concepto de dependencia funcional, el cual se lo expondrá en el siguiente punto.

3.7.1. Concepto de dependencias funcionales

"Una dependencia funcional es una restricción entre dos conjuntos de atributos de la base de datos." [17]

La teoría de la normalización se fundamenta del concepto de dependencia funcional, de lo cual se expondrán varios tipos:

3.7.1.1. Dependencia funcional

Si se establecen los subconjuntos X e Y en una relación, se hablaría de que Y tiene dependencia funcional de X es decir que X se encuentra involucrado con Y, únicamente si cada valor correspondiente a X contiene un único valor de Y.

Esta dependencia queda representada de la siguiente forma:

$$X \rightarrow Y$$

Por ejemplo, considerando la relación Articulo con sus atributos, codigo_art, descrip_art, pvp_art, en donde el campo codigo_art, determina la descripción y su precio del mismo debido a que al proporcionar un código de un artículo se conseguirá obtener una única descripción y precio. Lo cual se ve expresado de la siguiente manera.

$$\text{codigo_art} \rightarrow \text{descrip_art}$$

$$\text{codigo_art} \rightarrow \text{pvp_art}$$

De igual manera en la relación Linea_pedidos con sus campos, referencia_ped, codigo_art, cantidad_art, de los cuales el dúo de atributos referencia_ped y codigo_art, se establece el campo cantidad_art, debido

a que por cada pedido identificado por el campo referencia_ped y por cada artículo identificado por el campo codigo_art, solo existe una cantidad, dicho de otra manera, por cada línea de pedido únicamente se necesita un valor especifico de unidades, es por esto que:

referencia_ped, codigo_art → cantidad_art

El grafo o también llamado diagrama de dependencias funcionales es una herramienta útil para describir estas dependencias, en donde se simboliza una agrupación de atributos y las dependencias funcionales existentes entre ellos.

Dado a que para cada pedido identificado por una referencia existe únicamente una fecha en que fue realizado, surge la siguiente dependencia funcional.

referencia_ped → fecha_ped

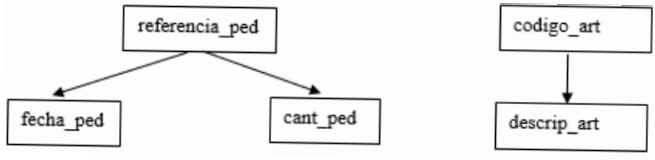


Figura. 41: Diagrama de dependencias funcionales

3.7.1.2. Dependencia funcional completa

Definidos los subconjuntos de atributos X e Y en una relación se indica que, Y conlleva una dependencia funcional completa de si, la cual depende funcionalmente de X, pero esta no depende de ningún subconjunto de X, lo que se representa de la siguiente manera:

$$X \Rightarrow Y$$

Tomando el ejemplo, con la relación Linea_pedidos y sus atributos, referencia_ped, codigo_art, cantidad_art, se consigue plantear si la dependencia funcional

(referencia_ped, codigo_art) → cantidad_art

Esta dependencia estará completa si el atributo cantidad_art tiene dependencia de los atributos, referencia_ped, codigo_art y no de uno de ellos por separado, es decir si las siguientes dos dependencias funcionales no son ciertas:

referencia_ped → cantidad_art

codigo_art → cantidad_art

La primera de las dos dependencias funcionales anteriores no es verdadera, debido a que un pedido identificado por su referencia (referencia_ped) puede tener diferentes cantidades de artículos requeridos; de hecho, esta situación se repetirá siempre que el pedido involucre algunas líneas de pedido, es decir, siempre que el pedido requiera distintos artículos.

Por otro lado, la segunda dependencia funcional, será verdadera si por cada artículo identificado por su código (codigo_art) solo pudiera existir una cantidad solicitada para dicho artículo. Sin embargo, ya que se puede solicitar un artículo en varios pedidos en los cuales se puede solicitar grandes cantidades de dicho artículo, la dependencia funcional conveniente tampoco es verdadera.

Dado que ambas dependencias funcionales desarrolladas no son verdaderas, se puede decir que la dependencia funcional

(referencia_ped, codigo_art) → cantidad_art

Es completa por lo que se constituye de la siguiente manera:

(Referencia_Ped, Codigo_Art) ⇒ cantidad_art

Cabe recalcar, que las de pendencias funcionales completas son generalmente simbolizadas partiendo la flecha de un rectángulo en el cual constan más de un atributo.

3.7.1.3. Dependencia funcional mutua o interdependencia

Si tenemos las siguientes dependencias funcionales X → Y e Y → X de forma paralela en una relación, se diría que entre los atributos X e Y existe una dependencia funcional recíproca o interdependencia, la cual se representa de la siguiente forma:

$$X \leftrightarrow Y$$

Por ejemplo, si se tiene la relación Libro con los siguientes atributos, #codigo_lib, isbn_lib, titulo_lib, paginas_lib, editorial_lib:

Libro				
codigo_lib	**isbn_lib**	**titulo_lib**	**paginas_lib**	**editorial_lib**
Lib001	0130417173	Ingeniería de Software	205	McGraw Hill
Lib002	0140418184	Programación Web	306	McGraw Hill
Lib003	0150419195	Fundamentos de Oracle	407	Paraninfo, S.A.
Lib004	0160420206	Auditoria de Sistemas	508	Paraninfo, S.A.

Tabla 24: Reacio Libro

se presentarán las dependencias funcionales que se describe a continuación.

$$codigo_lib \rightarrow isbn_lib$$
$$isbn_lib \rightarrow codigo_lib$$

Dado un libro identificado por un código, este tendrá un único valor de ISBN, o inversamente, si se tiene un libro que se identifica por su ISBN, este tendrá un único código. Esto se debe a que ambos atributos son claves candidatas de las cuales unas de estas dos podrán ser seleccionadas como clave principal. Pues ahora, al otorgarse las dependencias funcionales en ambos sentidos, se podría decir que los atributos codigo_lib e isbn_lib conservan una dependencia funcional recíproca o interdependencia:

$$codigo_lib \leftrightarrow isbn_lib$$

3.7.1.4. Dependencia funcional transitiva

Teniendo una relación R (X, Z) en la cual se ven presentes las siguientes dependencias funcionales:

$$X \rightarrow Y$$
$$Y \rightarrow Z$$
$$Y \rightarrow X$$

Por lo tanto, Z contiene una dependencia funcional transitiva con relación a X a través de Y, además se simboliza de la siguiente forma.

$$X \text{ - } \rightarrow Y$$

Considerando la relación Vehículos con los atributos, #matricula, marca, modelo, color.

Vehiculos			
matricula	**Marca**	**modelo**	**color**
AAC-0123	Chevrolet	Aveo Family	Gris
AEC-0123	Ford	F-150 FX4	Azul
AMC-0123	Hyundai	Accent	Blanco
CD-0123	Kia	Rio Stylus	Rojo

Tabla 25: Relación Coche

En esta relación se ven presentes las siguientes dependencias funcionales:

$$matricula \rightarrow marca$$
$$matricula \rightarrow modelo$$
$$matricula \rightarrow color$$

Debido a que, teniendo vehículo que se identifica por su matrícula, este pertenece a una sola marca, de igual manera pertenecerá a un único modelo y un único color. Por lo cual se ve presente la siguiente dependencia funcional.

modelo → marca

Ya que a cada modelo de vehículo le corresponde una sola marca.
Todas las dependencias funcionales mencionadas anteriormente se pueden representar a través del siguiente grafo:

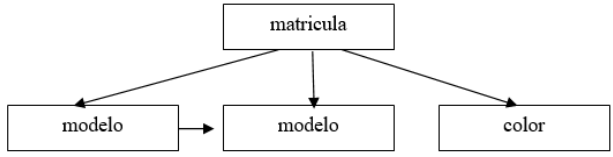


Figura. 42: Diagrama de dependencias funcionales

Si se presentan las siguientes dependencias funcionales:
matricula → marca
modelo → modelo
modelo → color
Se puede decir que el atributo marca tiene una dependencia funcional transitiva con relación a al atributo matricula mediante el modelo y se representa de la siguiente manera:

matrícula -→ marca

3.7.1.5. Dependencia multivaluada

Teniendo una relación R (X, Z), se podría decir que, Y tiene una dependencia multivaluada de otro atributo de X o que X multidetermina a Y si por cada valor de X se establece un conjunto definido de valores de Y, así mismo dicho conjunto es individual de cualquier valor que contenga otro atributo Z el cual tiene dependencia del valor de X. Estas dependencias multivaluadas se representan de la forma siguiente:

X → → Y

Por ejemplo, si se tiene la relación Cursos con los atributos, curso, profesor, texto, en la cual se presenta que por cada uno de los cursos que se van a dar, los profesores que lo van a impartir y los libros que serán utilizados. Tanto los atributos Profesor y Texto son asignados con múltiples valores hacia un mismo valor de curso. Se ha establecido

una norma que sugiere que todos los profesores de un curso puedan usar cualquier libro de texto que se haya asignado a curso que dictara.

Cursos		
Curso	**profesor**	**texto**
Herramientas Case	Bertha Mazón Mariuxi Zea	Análisis y Diseño de Sistemas Herramientas cases y sus Aplicaciones Metodológicas
Aplicación de Sistemas Operativos	Joffre Honores Milton Valarezo	Fundamentos de Sistemas Operativos Administración de sistemas Operativos

Tabla 26 : Relación Con Dependencias Multivaluadas

Como se observa en la tabla anterior, el atributo curso asigna múltiples valores tanto de profesor como de texto y que además estos dos atributos no tienen dependencias entre sí. Sin embargo, en un curso, existirá un conjunto de profesores que lo van dictar y de igual manera un conjunto de textos que serán utilizados, lo cual es realizado a través de las siguientes dependencias multivaluadas:

$$\text{curso} \rightarrow\rightarrow \text{profesor}$$
$$\text{curso} \rightarrow\rightarrow \text{texto}$$

3.7.1.6. Dependencia de reunión

Se dice que una relación tiene una dependencia de reunión cuando esta se puede arreglar sin que se pierda información partiendo de una mezcla de cualquiera de sus proyecciones. Suele ser difícil descubrir estas dependencias.

En un segundo nivel se aprecian sus proyecciones con dos atributos (binarias). Luego, su mezcla se efectúa en dos fases: de las cuales en la primera se alistan sus dos primeras proyecciones binarias y después el resultado es mesclado con la tercera proyección binaria. Dado que el resultado es justamente equivalente a la relación de partida, esta relación original presenta una dependencia de reunión. Por lo cual se indica que para cada relación declarada la clave principal estará conformada por todos los atributos correspondientes a cada relación.

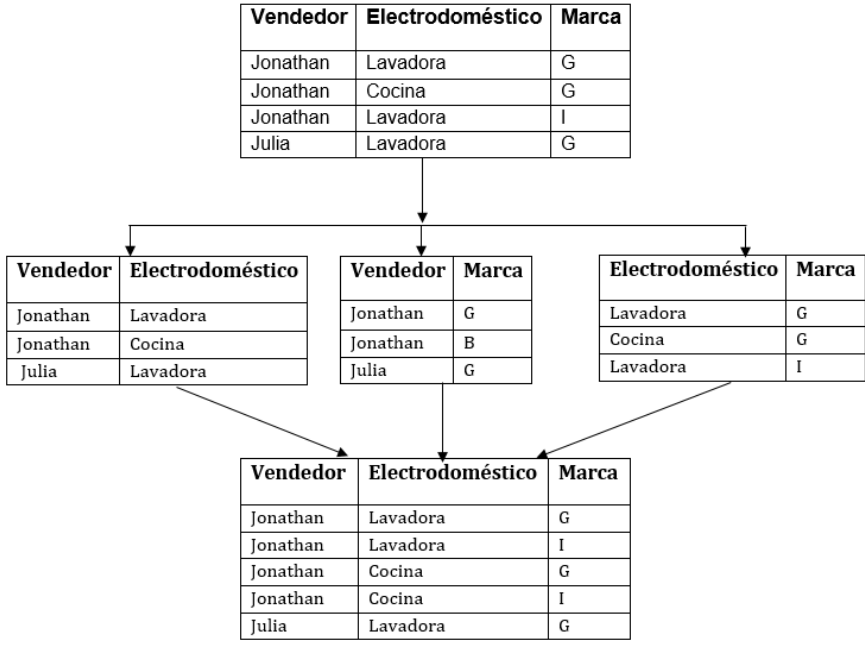


Figura. 43: Comprobación de dependencia de reunión

3.7.2. Análisis y aplicación de las diferentes formas normales: forma normal y la forma normal de Boyce/Codd

En esta sección se efectuará un análisis de cada una de las formas normales.

3.7.2.1. Primera forma normal

Se considera que una relación está en primera forma normal cuando todos sus elementos son atómicos, es decir, si estos no contienen grupos iterativos. Además, se puede declarar un grupo iterativo como un atributo o como un conjunto de atributos que contengan valores múltiples.

Para poder eliminar un grupo iterativo, o, dicho de otra manera, para convertir una relación en primera forma normal, se debe suprimir de la relación de origen todos aquellos atributos que formen parte del grupo iterativo, logrando así crear una nueva relación utilizando la clave principal de la relación de partida y de igual manera cada atributo del grupo iterativo.

La clave de la nueva relación generalmente consiste en la vinculación de la clave principal de la relación original con la clave del grupo iterativo.

Por ejemplo, teniendo la siguiente relación:

Pedidos (codigo_ped, fech_ped, codigo_art, descrip_art, cantidad_art, pvp_art)

Se puede observar la información referente a un pedido y a todos los artículos que son requeridos en él, presentando por cada artículo solicitado, los siguientes datos: código, descripción, número de unidades solicitadas y precio. Los atributos codigo_art, descrip_art, cantidad_art y pvp_art forman un grupo iterativo ya que si se considera como clave principal de la relación Pedidos el atributo codigo_ped, para el mismo pedido identificado por su referencia codigo_ped, se podrán solicitar diversos artículos, de los cuales cada uno de ellos tendrá sus atributos codigo_art, descrip_art, cantidad_art y pvp_art.

Esta relación no está en primera forma normal, ya que se encuentra en un grupo iterativo. Para lograr convertirla en primera forma normal, será necesario suprimir de la relación Pedidos cada uno de sus atributos que forman parte del grupo iterativo, logrando de esta manera crear una nueva relación, a la cual se la denominara Linea_pedidos. A más de eso se deberá determinar una nueva relación que contendrá los cuatro atributos del grupo iterativo junto a la clave principal de la relación de origen codigo_ped, siendo esta la clave de la nueva relación obtenida, adicionalmente el atributo clave del grupo iterativo codigo_art quedando así el siguiente diseño relacional:

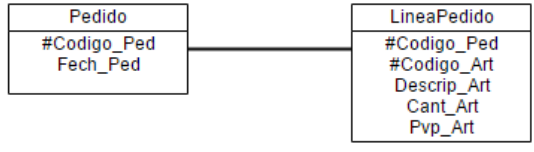


Figura. 44: Diseño Relacional

Otro aspecto a considerar para que una relación este en primera forma normal, es que los atributos contengan valores únicos para cada registro, es decir que en cada dominio existan valores únicos. Por ello si una relación no está en primera forma normal se debe eliminar de ella todos los atributos que tengan valores múltiples en cada tupla de la relación. Para eliminar esto, se debe colocarlos en una nueva relación y relacio-

narla a través de la clave primaria a la relación de origen.

Así por ejemplo si se tiene la relación Alumnos con los siguientes campos, codigo_alum, nombre_alum y asignaturas_alum, de los cuales el campo asignaturas_alum, almacena las asignaturas en las que recibe clases cada alumno.

Alumnos		
codigo_alum	**nombre_alum**	**asignaturas_alum**
1	Jonathan	Matemáticas
2	Andrés	Física, Algebra
3	Jaime	Geometría, Física

Tabla 27: Relación Alumnos

Como se observa en la relación Alumnos, los registros dos y tres tienen dominios que contienen varios valores por tanto la relación no se encuentra en primera forma normal. Para convertir esta relación en la primera forma normal se crea una nueva relación Asignatura, en donde cada asignatura responderá a una clave principal.

Alumnos	
codigo_alum	**nombre_alum**
1	Jonathan
2	Andrés
3	Jaime

Tabla 28: Relación Alumnos

Asignaturas	
codigo_asig	**nombre_asig**
1	Matemáticas
2	Física
3	Algebra
4	Geometría

Tabla 29: Relación Alumnos

3.7.2.2 Segunda forma normal

Se considera que una relación se encuentra en segunda forma normal si primeramente esta en primera forma normal, además cada atributo que no pertenezca a una clave candidata guarda una dependencia funcional con relación a dicha clave, lo que indicaría que cada uno de sus

atributos tiene que depender de toda la clave mas no solo de una parte de ella. Para lograr convertir la relación en segunda forma normal, se necesita prescindir de la relación al atributo que forma la dependencia parcial y determinar una relación nueva con ese atributo y con el o los atributos del cual depende como clave principal.

Siempre que una relación que se encuentra en primera forma normal presenta una clave principal, la cual se encuentra formada por un único atributo, de forma automática estará en su segunda forma normal. Asimismo, se considerará que se encuentran en su segunda forma normal todas aquellas relaciones que se encuentren en primera forma normal y que además no ostenten atributos.

Utilizando el esquema relacional logrado anteriormente, se tendrá que la relación Pedidos ya se encuentra en su segunda forma normal inicialmente por el hecho de estar en primera forma normal y contar como clave principal con un único atributo. Mientras que en lo que respecta a la relación denominada Linea_pedidos, para que logre estar en su segunda forma normal se deberá verificar que todos sus atributos que no son considerados clave, como lo son la descrip_art, cantidad_art y pvp_art, respondan completamente a la clave principal, por lo tanto, lo siguiente quiere decir que deben tener totalmente una dependencia funcional con relación al par de atributos (codigo_ped, codigo_art):

$$(\text{codigo_ped}, \text{codigo_art}) \Rightarrow \text{Descrip_art}$$
$$(\text{codigo_ped}, \text{codigo_art}) \Rightarrow \text{cantidad_art}$$
$$(\text{codigo_ped}, \text{codigo_art}) \Rightarrow \text{pvp_art}$$

Primera dependencia funcional total: No es verdadera ya que la descripción del artículo responde únicamente al código del mismo y no al código del pedido que lo solicita, por ende, es considerada como verdadera la dependencia parcial entre codigo_art → descrip_art. Por este motivo se puede asegurar que la relación Linea_pedidos no se encuentra en su segunda forma normal.

Segunda dependencia funcional total: Es considerada como verdadera ya que la cantidad que es solicitada de un artículo por un pedido no está determinada únicamente por el pedido con el que se está tratando o el artículo que es requerido, sino por los dos. Es decir que se tendría lo siguiente codigo_ped → cantidad_art, puesto que en un pedido sería posible solicitar varias y distintas cantidades de artículos

de forma similar, codigo_art → cantidad_art, debido a que un artículo puede ser solicitado en diferentes pedidos, de los cuales a cada uno solicitará cierta cantidad de artículos.

Tercera dependencia funcional total: Es considerada verdadera cuando los valores correspondientes a los artículos no son siempre los mismos para los pedidos. Es común que el valor de cada artículo sea el mismo para todos y cada uno de los clientes, es por esto que se comprueba la dependencia funcional codigo_art → pvp_art, motivo por el cual tampoco se cumple que aquel atributo que no es clave pvp_art vaya totalmente a depender de la clave, lo cual indicaría otra razón para que la relación no se encuentre en su segunda forma normal. Otra manera de observar cuando una relación se encuentra en su segunda forma normal es examinando su diagrama de dependencias funcionales.

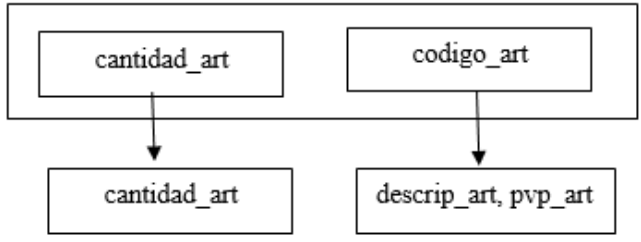


Figura. 45: Diagrama de dependencias funcionales

Figura. 45: Diagrama de dependencias funcionales

Si existiera alguna flecha en la figura anterior que cruce el área sobrante entre los atributos que crean la clave compuesta y todos aquellos atributos que no son claves, por consiguiente, existirán atributos que requieren en su totalidad de la clave, por lo que la relación más conveniente no está en su segunda forma normal, como sucede en este caso.

Se transforma la relación Línea_pedidos a la segunda forma normal, para lo cual es necesario eliminar de la relación cada uno de sus atributos que no requieran de la clave (descrip_art y pvp_art) y definir en primera instancia una nueva relación con todos los atributos que se suprimieron de la relación anterior y los que dependan de, codigo_art. Como se ha descrito en el ejemplo, los dos atributos requieren del mismo codigo_art, por lo que no es indispensable definir una nueva relación por cada uno de ellos, sino solamente una que contenga am-

bos atributos y el atributo que se requiera como clave. De lo cual se representa en el siguiente esquema relacional.

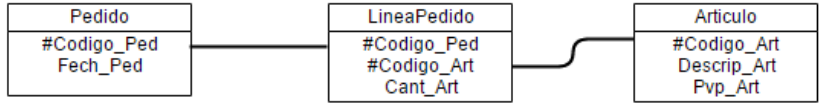


Figura. 46: Esquema relacional

3.7.2.3 Tercera forma normal

La tercera forma normal se fundamenta del concepto de dependencia transitiva (esto sucede cuando un atributo depende funcionalmente de otro y ninguno es parte de la clave), por lo tanto, se dice que una relación está en tercera forma normal solo si sus atributos dependen únicamente a la clave principal mas no de otros atributos. [18]

Para renunciar a las dependencias transitivas se debe descartar de la relación que no se encuentra en tercera forma normal el atributo que genera la dependencia transitiva y se crea una relación que contenga los atributos transitivos conjuntamente con el atributo del que requiere es decir mediante el cual conserva su transitividad. Y de esta manera se asume la relación R (A1, A1, A3) junto a las siguientes dependencias funcionales:

$$\#A1 \longrightarrow A2 \overset{\nleftarrow}{\longrightarrow} A3$$

Existe una dependencia funcional transitiva de A3 con relación a A1 ya que A3 tiene dependencia de A2 y no la tiene directamente de A1, la cual es la clave principal. Esto tendría solución pasando la relación a Tercera Forma Normal, utilizando el siguiente esquema relacional:

$$R1\ (A1, A2)$$
$$\downarrow$$
$$R2\ (A2, A3)$$

Suponiendo que la relación Auto con los atributos (placa, marca, modelo, color) de la cual ya se había referido con anterioridad, en la cual marca tiene una dependencia funcional transitiva en relación a la placa a través del modelo debido a que:

$$placa \longrightarrow modelo \overset{\nleftarrow}{\longrightarrow} marca$$

Por tal motivo, la relación antes descrita no se encuentra en tercera forma normal y para convertirla a esta forma normal lo que se necesita hacer es descartar de la relación Auto el atributo transitivo marca y crear otra relación usando este atributo junto al que está condicionado, por lo que este último es la clave principal de la nueva relación. Por consiguiente, se tiene el esquema relacional descrito a continuación.

Auto (matrícula, modelo, color)

↘

Modelos (modelo, marca)

3.7.2.4 Forma normal de Boyce/Codd (FNBC)

El concepto inicial de Codd de tercera forma normal presentaba varios fallos. Entre los cuales se puede mencionar que no trataba de forma adecuada a una relación que contenga algunas claves candidatas, son compuestas, y las claves candidatas coinciden (Presenta algún atributo en común). De esta manera, se creó una forma normal más fuerte.

Se considera que una relación está en FNBC si tiene un determinante que por sí mismo sea clave de la misma, y que por ende es una llave candidata.

A1 $\longrightarrow$ A2	(A1, A2) $\longrightarrow$ A3
Determinante	Determinante

Por lo tanto, está comprobado que cualquier relación que se encuentre en FNBC se encuentra en tercera forma normal.

Tomando en cuenta la relación R (A1, A2, A3) de la cual (A1, A2) $\longrightarrow$ A3 y A3 $\longrightarrow$

A2. Existen dos determinantes que son (A1, A2) y A3, entre las cuales una solo es llave candidata (A1, A2), esto quiere decir que, la relación no se encuentra en FNBC.

Para lograr que una relación este en FNBC lo que se debe realizar es lo siguiente:

Establecer una clave usando la parte de la clave que es independiente de A1 y cada uno de los atributos que no son principales A3.

R1 (A1, A3)

Se declara otra relación utilizando la otra parte de la clave que sobra junto con el atributo secundario al cual se encuentra condicionado, por lo que este último será la clave de la nueva tabla.

R2 (A3, A2)

Considerando la siguiente relación: Postal con sus atributos, dirección, ciudad, cp.

Postal		
dirección	**Ciudad**	**cp**
Condado. 4	Quito	28809
Alborada. 5	Machala	28823
Los Sauces. 2	Machala	28823
El Bosque. 4	Guayaquil	28019
4. mil. 4	Guayaquil	28007

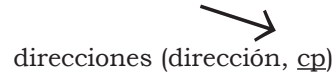
Tabla 30: Relación Postal

En la relación Postal se posee las reglas semánticas descritas a continuación:

- El mismo nombre de una calle puede presentarse en dos ciudades distintas
- A Cada calle se le permitirá poseer un tramo en códigos postales distintos
- No se podrá comprender dos ciudades en un mismo código postal
- No pueden existir dos ciudades con el mismo nombre
- Esta relación está en Tercera Forma Normal, pero no se encuentra en FNBC, A razón de las siguientes dependencias funcionales.

$$(\text{dirección, ciudad}) \rightarrow \text{cp}$$
$$\text{Cp} \rightarrow \text{ciudad}$$

Se cuenta con dos determinantes dirección y ciudad, las cuales representan una clave candidata, pero cp no lo es, por lo cual es posible que existan diversas tuplas utilizando el mismo cp. Convirtiendo la relación a FNBC:



3.7.2.5 Cuarta forma normal

Se considera que una relación se encuentra en cuarta forma normal

si la misma ya se encuentra en FNBC y que además no tenga dependencias multivaluadas. Ya que estas suelen ocasionar problemas en el mantenimiento de la relación que las muestra. Los cuales se resuelven dividiendo la relación que se muestra y las dependencias multivaluadas, de tal manera que si se tiene la relación R (A, B, C) la cuan cuenta con las dependencias multivaluadas $A \rightarrow \rightarrow B$ y $A \rightarrow \rightarrow C$, es posible dividir esta relación en R1 (A, B) y R2 (A, C). Es notorio, que estas relaciones comprenden los atributos con valores variados.

Dada la relación Cursos la cual contiene los atributos curso, profesor, texto presenta las dependencias multivaluadas

$$\text{curso} \rightarrow \rightarrow \text{profesor}$$

$$\text{curso} \rightarrow \rightarrow \text{texto}$$

Por ende, no se encuentran en cuarta forma norma, lo cual se resuelve estableciendo las relaciones siguientes

Cursos 1	
Curso	**Profesor**
Herramientas Case	Bertha Mazón
Herramientas Case	Mariuxi Zea
Aplicación de Sistemas Operativos	Joffre Honores
Aplicación de Sistemas Operativos	Milton Valarezo

Cursos 2	
Curso	**Texto**
Herramientas Case	Análisis y Diseño de Sistemas
Herramientas Case	Herramientas cases y sus Aplicaciones Metodológicas
Aplicación de Sistemas Operativos	Fundamentos de Sistemas Operativos
Aplicación de Sistemas Operativos	Administración de sistemas Operativos

Tabla 31: Relaciones en Cuarta Forma normal

3.7.2.6. Quinta forma normal

Se considera que una relación esta quinta forma normal cuando por cada dependencia de reunión, su proyección comprende una clave de la relación inicial. Es realmente difícil identificar las dependencias de reunión, además que no es muy común el uso de esta forma normal. Es por esto que el resultado de aplicar esta forma normal entra en discusión con mucha frecuencia debido a la dificultad que involucra la división de la relación inicial en un número transcendental de nuevas

relaciones.

Considerando la relación que se expuso en el apartado 3.7.1.6, a razón de que las proyecciones de la figura 28 no comprenden la clave de la relación de inicio, se reconoce que no se encuentra en Quinta Forma Normal por lo que, para solucionar este pequeño problema, la relación vendedores-electrodomésticos-marcas tendrá que dividirse en las tres relaciones que conforman las proyecciones en esa misma figura.

3.7.3 Ventajas e inconvenientes que justifican una desnormalización de las tablas. Valoración en diferentes supuestos prácticos.

Ocasionalmente es favorable suavizar las reglas de normalización, es por esto que una de las tareas que se suelen realizar en el diseño lógico una vez que ya se ha obtenido un esquema lógico normalizado, es justamente pensar en introducir redundancias controladas entre otras modificaciones en el esquema para lograr optimizar las prestaciones del sistema.

Existen dos tipos de bases de datos que sobresalen especialmente debido al tipo de operaciones que resaltan en ellas las cuales son: las bases de datos en las que generalmente se toma en cuenta más las consultas y aquellas en las cuales se efectúan con mayor frecuencia las modificaciones (altas, bajas y alteraciones). En el segundo tipo de bases de datos, es más adecuado realizar disminuciones de redundancia, sin sugerir que se suavicen las reglas de normalización. Aunque en las bases de datos en las que se considera habitualmente las consultas, suele ser apropiado que en ciertos casos se realice la desnormalización de la misma de tal forma que las consultas se den de una forma más satisfactoria.

En la fase de diseño lógico llegar al menos a la tercera forma normal, y de esta manera obtener un esquema relacional con una estructura más sólida y que carezca de redundancias. En ocasiones sucede que las bases de datos que se normalizan de esta manera no ofrecen la máxima eficacia, por lo que es recomendable desnormalizar ciertas relaciones (tablas) con el objetivo de perfeccionar su prestación. Sin embargo, el hecho de que bajo ciertas circunstancias sea favorable desnormalizar, en ningún caso es aceptable excluir la etapa de normalización del diseño lógico.

La desnormalización genera las desventajas descritas a continuación:

- Puede ocasionar que se detengan las modificaciones, por lo cual no es apropiado que se efectúe en las bases de datos en las que predominan las modificaciones.
- Por cada redundancia que se crea, a alguien le corresponde asegurar la integridad de los datos, y de esta manera evitar que se presenten inconsistencias. Por cada escenario que surja se puede proponer dos opciones, la primera implica que sean justamente los programas de aplicación los encargados de resguardar la integridad de los datos y la segunda opción y quizás la más favorable, es la de crear triggers (es un procedimiento almacenado en la base de datos que reacciona por cada actualización que se realice a la base de datos.) también conocidos como disparadores, que cumplan con la tarea de resguardar la integridad de los datos.
- Es así que gracias al trabajo de este procedimiento se asegura que todas y cada una de las copias redundantes se conserven equilibradas.
- Se complicará la agregación de tablas nuevas que refieran datos futuros a causa del crecimiento de la aplicación, en los cuales se hubieran ejecutado desviaciones con respecto al modelo de datos normalizado.
- Se dice que, si se realzan varias consultas a una base de datos en las cuales se comprenden más de cinco o seis tablas, se sugiere una desnormalización. Obviamente, esto solo sucedería si se estuviera hablando de una base de datos en las que las actualizaciones son más habituales.

Seguidamente se analizarán escenarios en los que generalmente se debe medir su eventual desnormalización.

3.7.3.1. Combinar relaciones de uno a uno

Se obtiene mayores beneficios agrupando en una misma tabla todos los atributos correspondientes a dos tablas involucradas en una relación de uno a uno. Por ejemplo, si se cuenta con las tablas Instituto y Director las cuales están vinculadas por medio de una relación de 1:1, lo más común seria realizar lo indicado en el esquema relacional descrito a continuación.

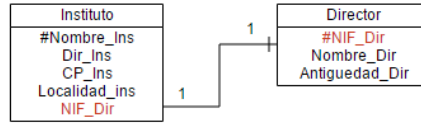


Figura. 47: Relación de uno a uno (1:1)

Sabiendo que se ingresan datos con frecuencia en ambas tablas y que esto sucede al mismo tiempo para lo cual es necesario realizar una combinación de ambas relaciones (tablas) y que por otro lado cuando se ingresa datos por separado, es posible acoplarlas en una misma tabla, como lo que se presenta a continuación:



Figura. 48: Relación Instituto

3.7.3.2. Duplicar atributos que no son llave en relaciones de uno a varios

Para hacer algo diferente a lo que se hizo en la sección anterior con respecto a las relaciones de 1:1, se puede contener en una composición de dos tablas todos aquellos atributos de la tabla a la cual le corresponde la cardinalidad uno (1:) en la tabla a la cual le corresponde la cardinalidad n (:n). Por citar un ejemplo, si se dispone de una tabla en la que son almacenados los datos de los departamentos con lo que cuenta una empresa y otra que contenga los datos de cada uno de los empleados que laboran en ellos, inicialmente se presentaría el siguiente esquema relacional

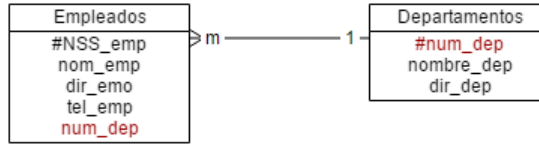


Figura. 49: Relación de uno a muchos (1:n)

Sabiendo que con frecuencia las consultas que involucra presentar los datos de empleados también requieren presentar los datos del departamento en el que se encuentra laborando cada uno de ellos, por lo cual es conveniente consolidar los atributos comprendidos entre las dos tablas en una solo, de lo cual se tendría como resultado la siguiente tabla

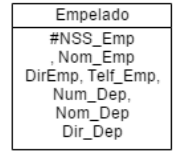


Figura. 50: Tabla Empleados

3.7.3.3. Duplicar atributos en relaciones de varios a varios

Cuando se presenta una relación con una cardinalidad de varios a varios entre dos tablas, en el momento que se realice el diseño lógico surgirá necesariamente una tabla con los atributos clave de las entidades afines en conjunto con los atributos correspondientes a la relación. Si se necesita obtener información de la relación con dicha cardinalidad de varios a varios usualmente será de carácter obligatorio realizar la composición de las tres tablas, lo que considerado desde un punto de vista tecnológico resulta ser muy caro. Es posible evadir esto guardando algunos de los atributos que forman parte de las tablas iniciales en la nueva tabla que resulta de la relación N:M. de lo cual resultaría el siguiente esquema relacional.

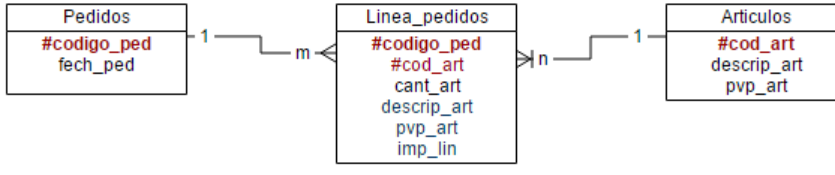


Figura. 51: Relación de muchos a muchos

Generalmente cuando se solicita información de la línea de pedido, es necesario presentar tanto la descripción de cada artículo como su valor, por lo cual se tiene que verificar la eventualidad de vincular los atributos antes mencionados (descrip_art y pvp_art) en la tabla Linea_pedidos.

3.7.3.4. Incluir datos derivados

En ciertas ocasiones una consulta debe obtener datos que se generen partiendo de otros, si este dato ya ha sido previamente calculado la consulta será más eficiente. Tomando en cuenta el esquema relacional del apartado anterior (3.7.3.3) en la cual se puede incrementar en la tabla Linea_pedidos el valor correspondiente al atributo imp_lin si se determina que se trata de un dato que es consultado con mayor frecuencia. El dato mencionado anteriormente es producto de la multiplicación entre el atributo cantidad_art de la tabla Linea_pedidos con el valor del atributo pvp_art de la tabla Artículo.

3.7.3.5. Partición de tablas

En una tabla grande generalmente durante la ejecución de la mayoría de procesos solo considera una porción reducida de la misma. Lo cual se puede acelerar el tiempo de ejecución de los procesos almacenando en una misma tabla todos los atributos que suelen ser utilizados con mayor frecuencia, y dejando a los demás atributos en una sola tabla. Si se tiene una tabla pequeña en la cual se realizan los ingresos más habituales, cada ingreso en dicha tabla será realizado con mayor agilidad. En esta tabla existen ciertos atributos que son utilizados con más frecuencias en las consultas y para mejorar el resultado de las consultas en apropiado realizar una partición vertical de la tabla para lo cual se debe tener presente que toda partición necesita contener atributos que son clave principal de la tabla para de esta manera estar en la capacidad de rescatar bajo cualquier circunstancia la información inicial.

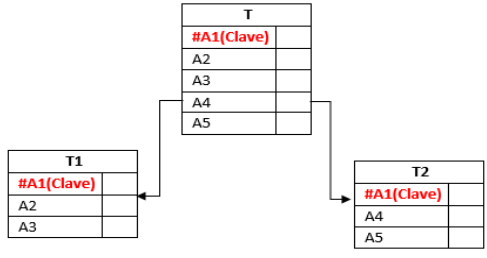


Figura. 52: Partición vertical

Figura. 52: Partición vertical

Si se cuenta con una tabla que almacena los datos correspondientes a

los empleados de una empresa la cual contiene entre sus datos, número de seguridad social, nombre, dirección y teléfono y en la otra, datos que referentes a su labor en una empresa cargo y cualificación y se sabe que los datos personales tienen mayores ingresos que los que se refieren al trabajo del empleado, se puede determinar dos segmentos verticales: de los cuales en uno constara los datos personales y en el otro los datos referentes a su trabajo. Estas dos particiones necesariamente deben contener la clave principal que será el número de seguridad social

Se debe tener siempre presente que la segmentación vertical soporta los siguientes problemas:

Se debe incrementar una tuplas en ambas tablas T1 y T2, por cada registro que ingresa a la tabla T, Y de la misma manera por cada registro que se elimine en la tabla T se deberán eliminar dos tuplas en ambas tablas T1 y T2.

Para consultar datos correspondientes a uno o varios registros de las tablas T1 o T2es preciso realizar una composición de las dos tablas T1 y T2.

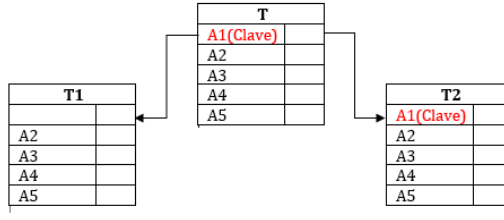


Figura. 53: Partición horizontal

Se debe tener siempre en cuenta que una partición horizontal es tolerante a los siguientes problemas:

Cada que se ingrese un nuevo registro a la tabla T es necesario que se incremente una tupla en cualquiera de las dos tablas T1 o T2, ya que las aplicaciones se fundamentan del criterio de partición que se haya seleccionado.

Si se necesita obtener los datos de uno de las tablas T1 y T2 y que se encuentran en la tabla T, se deberá tener conocimiento de a que tabla se tiene que acudir T1 o T2, puesto que los programas necesitan conocer el criterio de partición que se haya acogido.

Si se necesita efectuar consultas que involucran la obtención de datos

de las tablas T1 o T2, es necesario usar el operador unión, el cual resulta ser muy caro, desde el punto de vista tecnológico.
Si es necesario unir los datos obtenidos de la tabla T1 con datos provenientes de otras tablas, es preciso efectuar tantas uniones como sea necesario.

Posteriormente puede cambiar la frecuencia de utilización de las filas correspondientes a ambas tablas, por lo que puede ser necesario que se reformen las tablas, esto provoca que surjan alteraciones en los aplicativos que usen los datos de estas tablas.

3.7.3.6. Atributos repetidos en la misma fila

La primera forma normal se encarga de suprimir los conjuntos que se vean repetidos los cuales se presentan en una relación, provocando la creación de una nueva tabla que contendrá los atributos que corresponden al conjunto repetido adicionalmente con la clave principal de la relación de origen.

Si se tiene la tabla Universidad, en la cual serán almacenados por cada universidad el nombre de su rector y sus vicerrectores:

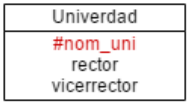


Figura. 54: Relación Universidad

Dado que por lo general una universidad cuenta con algunos vicerrectores (un máximo de tres), lo que representa un conjunto repetido, es por esto que la relación en primera forma normal se vería de la siguiente manera:

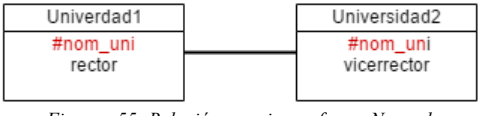


Figura. 55: Relación en primera forma Normal

Que se cuente con los datos tanto del rector como los del vicerrector de cada universidad implica unir ambas tablas y analizar alrededor de dos y cuatro filas de datos. Si se quiere evitar esto, lo que se necesita hacer es desnormalizar tal como se muestra en el caso que se describirá a

continuación.

Como en este caso se conoce que el número máximo de vicerrectores de la universidad es de tres, lo que se debe hacer es concentrar todos los atributos en una misma relación.

Si se tiene lo siguiente, Universidad (nom_uni, rector, vicerrector1, vicerrector2, vicerrector3) ocasionaría un problema en cada ocasión en la que el número de vicerrectores con los que cuente una universidad sea menor a tres ya que dada esta situación va a tener uno o varios atributos con valor nulo.

3.7.4. Desarrollo de diferentes supuestos prácticos de normalización de datos incluyendo propuestas de desnormalización de datos

En la Sección de Ejercicios que ya han sido resueltos para el tema 2 se proponen varias suposiciones prácticas de normalización, los cuales se señalan al lector. Se puede encontrar más suposiciones de normalización en la sección dedicada a ejercicios propuestos.

Resumen

- En una base de datos relacional, los datos son recolectados mediante relaciones, y estas a su vez son generalmente representadas mediante tablas
- Una clave candidata, consiste en una agrupación de atributos que identifican de manera única a cada fila perteneciente a la relación.
- En un modelo relacional se pueden distinguir dos tipos de restricciones, que son las restricciones inherentes y las restricciones semánticas.
- El álgebra relacional es un lenguaje formal de consultas a través de bases de datos relacionales que compone la base del lenguaje de consulta SQL grandiosamente usado
- Generalmente en todo par de relación, necesita ser relacionadas con respecto a la unión, es decir que es necesario que sean de un mismo grado y que sus atributos deben corresponder a un mismo dominio.
- Las operaciones de la teoría de Conjuntos son: Unión, Intersección, Diferencia, y Producto cartesiano
- Los operadores relacionales específicos son los siguientes: Operador de selección o restricción, Operador de proyección, Operador de división, Operador de combinación.
- Existe una forma diferente de cálculo relacional conocida como calculo relacional de dominós, en el cual se utilizan variables de dominio propiamente, las mismas que adquieren valores del domino de un atributo en vez de obtenerlas de una fila completa
- La teoría de la normalización, propone enfrentar y resolver proble-

mas de bases de datos de una forma objetiva e implacable, detallando como llevar a cabo la normalización en un diseño de base de datos relacional.

- Realmente hay seis formas normales: Primera Forma Normal, Segunda Forma Normal, Tercera Forma Normal, Forma Normal de Boyce/Codd, Cuarta Forma Normal, Quinta Forma Normal
- La teoría de la normalización se fundamenta del concepto de dependencia funcional, la cuales pueden ser, completa, mutua o interdependiente, transitiva, Multivaluada, de reunión

Modelo orientado a objetos

Descripción del capítulo

A continuación, se detalla el objetivo principal de cada contenido del capítulo 4: Modelos Orientados a objetos.
En el apartado 4.1 se persigue contextualizar los modelos orientados a objetos dentro del modelado UML
El objetivo del apartado 4.2 es el de exponer una descripción clara sobre el modelado de datos orientado a objetos, hablando así también de los siguientes aspectos:

- Conceptos de objeto, clase, método, y mensaje
- Representación de clases UML
- Relaciones entre clases UML

En el apartado 4.3 se tiene por objetivo realizar una comparación entre el modelo de clases con el modelo entidad relación
EL apartado 4.4 se orienta a describir los diagramas de objetos como caso especial del diagrama de clases
Y por último, en el apartado 4.5 se tiene como objetivo principal el exponer de forma clara una descripción sobre los modelos distribuidos y los enfoques que se deben tener en consideración para realizar un diseño

4.1. Contextualización del modelo orientado a objeto dentro del modelado UML

Un lenguaje UML (Lenguaje de Modelo Unificado) sirve para diseñar modelos y es independiente de los métodos de análisis y diseño. Ade-

más, permite dar un mejor soporte a la planeación y al control de proyectos.

Las principales características que se deben tener en cuenta en el UML son los bloques de construcción, reglas y mecanismos comunes aplicados por medio de UML.

El lenguaje UML posee tres bloques de construcción principales:

- Elementos: Estos elementos son actor, casos de uso y límite de un sistema.
- Relaciones: Son las conexiones entro los distintos elementos.
- Diagramas: Encargados de agrupar las colecciones de elementos. [19]

En las metodologías estructuradas una aplicación (tipo de programa informático diseñado para permitir a un usuario realizar todo tipo de trabajo), está compuesta por distintos componentes de software llamados generalmente módulos. Un módulo realiza determinadas tareas de procesamiento, por lo que es importante utilizar datos concretos. Una aplicación está compuesta por distintos módulos que realizan pequeñas tareas. Los módulos se relacionan entre sí realizándose llamadas de unos a otros. En estas llamadas los módulos se intercambian datos y a veces devuelven arquitectura de una aplicación informática consta de un módulo de control principal que llama a varios módulos subordinados. Estos módulos llaman a otros y así sucesivamente. La arquitectura de un programa se simboliza gráficamente mediante un diagrama de estructuras, en el que un rectángulo personifica un módulo y las flechas que unen los módulos significan llamadas del módulo. En el diagrama de estructuras que vamos a mostrar el módulo de control principal se llama A y llama a los módulos subordinados B y C; a su vez, b llama al D y al E, y el C llama al F.

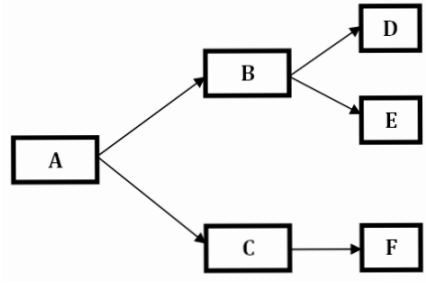


Figura. 56: Diagrama de Estructuras.

En las metodologías estructuradas es importante poner atención en los procesos o tareas que se necesita llevar a cabo para resolución de un determinado problema, en el cual se distribuyen estos procesos o tareas en módulos que las realizan. Para llevar a cabo estos procesos es necesario que los módulos manejen ciertos datos. Los datos que se almacenan de forma permanente se guardan en una base de datos, sobre la cual los diferentes módulos realizan operaciones de consulta, inserción, modificación y/o eliminación.

Los datos de una clase llevan el nombre de atributos y las operaciones que se aplican sobre esos datos se llaman métodos. La ejecución de todas las tareas encomendadas a un programa se logra mediante la participación cooperativa de los objetos que constan en la aplicación. Esta cooperación se da por medio de las llamadas de unos módulos a otros, en el paradigma orientado a objetos, se consigue mediante el envío de mensajes. Definimos un mensaje como una solicitud enviada desde un método de un objeto hasta otro objeto, o él mismo ejecute otro método.

La filosofía orientada a objetos provocó años más tarde un gran cambio de paradigma. Este cambio permitiría un cuidado más estricto a las técnicas brindando mayor seguridad a los datos que es preciso operar. Las metodologías orientadas a objetos proponen que una aplicación informática es formada por diferentes objetos que interactúan entre ellos. Un objeto vendría a ser una instancia de una clase y la clase está compuesta por una serie de datos y un conjunto de operaciones que se aplican sobre los datos. Los datos de una clase llevan el nombre de atributos y las operaciones que se usan en los datos se denominan métodos. Mediante la aportación conjunta de los objetos que se tiene en la aplicación se logra la ejecución de los trabajos encargados a un programa. Este conjunto se da por medio de las llamadas de un módulo a otro, en el ejemplo orientado a objetos, se obtiene con el envío de mensajes. Definimos un mensaje como una solicitud enviada desde un método de un objeto hasta otro objeto.

Al desarrollar una aplicación en la filosofía orientada a objetos es fundamental la preparación de diferentes diagramas. Se considera al diagrama de clases como lo fundamental. Se construyen otros tipos de diagramas como los que se muestran a continuación:

- Diagramas de objetos.

- Diagramas de casos de uso.
- Diagramas de secuencia.
- Diagramas de colaboración.
- Diagramas de actividad.
- Diagramas de estados.
- Diagramas de distribución.
- Diagramas de componentes. [19]

4.2. Modelo de datos orientado a objetos

Los datos del modelo orientado a objetos son considerados como un incremento del modelo Entidad-Relación, es decir para poder entender este modelo es preciso aprender anticipadamente ciertos significados, que se ven a continuación.

4.2.1. Conceptos de objeto, clase, método y mensaje

Tal como se mostró al comienzo, una aplicación en la metodología orientada a objetos consigue sus objetivos con la interacción de los diferentes objetos.

Se define a un objeto como una entidad que permite mostrar un procedimiento preciso. Un objeto puede llegar a considerarse igual a una ocurrencia de una entidad en el modelo Entidad-Relación. A continuación, se muestran algunas características que definen a un objeto:

Su estado: El estado de un objeto es definido por una serie de propiedades que tiene el objeto y los valores que se toman a cada instante. En el caso de hablar del sistema de gestión de una biblioteca el objeto puede ser un cualquier libro y su estado es determinado por el código, ISBN, número de páginas, editorial, título y la situación de entregado o no, junto con los valores que obtiene cada uno de los libros, por ejemplo, el código “C1352”, título: Bases de datos relacionales, ISBN: 0987654456987, editorial “Fortuna”, con 250 páginas y situación de entregado “Falso”.

Su comportamiento: El comportamiento de un objeto se da debido a la forma en que actúa al recibir un mensaje de otro objeto, sustituyendo su estado casualmente. El mensaje es una petición que recibe un objeto para que se produzca una operación y cumplir con un propósito. Por ejemplo, un objeto libro toma un mensaje en el que se

solicita que el libro sea traspasado, acción que cambia estado de entregado a "Verdadero".

El objeto es una instancia de la clase, o también, una clase está compuesta por una serie de objetos que poseen la misma estructura y el mismo procedimiento. Por ejemplo, la clase Libro constituye cualquier libro de la biblioteca siendo un objeto cada uno de los ejemplares de libros que se encuentran disponibles en la biblioteca.

Una clase está compuesta por los atributos que vendrían a ser las propiedades que tiene la clase. Por ejemplo, la clase libro posee los atributos código, título, ISBN, entregado y editorial. Cada atributo tendrá su nombre, un tipo de dato ya sea este (numérico, cadena de caracteres, texto, alfanumérico etc.) y un valor por defecto.

Los métodos son considerados como las operaciones que se llevan a cabo sobre los objetos de una clase y son quienes determinan el comportamiento de la clase. Khoshafian y Abnous (1990) clasifican los métodos en tres clases:

- De acceso: Este método proporciona la facilidad de acceder a las valore de los atributos para recuperar su valor.
- De actualización: Permite la modificación de estado o valor de un respectivo atributo.
- Constructores y destructores: Métodos se encargan de crear y destruir objetos de la clase a la que corresponden.

4.2.2. Representación de clases en UML

Cuando usamos la notación UML para los diagramas de clase, se representa una clase con un rectángulo divido en tres secciones. En la parte superior se coloca el nombre de la clase. Los atributos se colocan en la parte del centro y los métodos se colocan en la parte inferior. Se debe tomar en cuenta los siguientes aspectos sobre los atributos:

- Indicar por cada atributo su nombre.
- Indicar su visibilidad, el tipo de dato del atributo y su valor por defecto, en caso de que lo tenga.

El tipo de dato de los atributos pueden ser: numérico entero (integer), carácter (char), numérico real (float), cadena de caracteres (String), booleano (boolean), etc.

La visibilidad de un atributo genera una referencia en la aplicación en la que ese atributo es visible. La visibilidad puede tomar valores como:

Público: El atributo se muestra para toda la aplicación. El símbolo se coloca al frente del nombre del atributo.

Privado: El atributo se muestra únicamente para la misma clase. El símbolo se coloca al frente del nombre del atributo.

Protegido: El atributo se muestra para la misma clase y todas las sub clases. El símbolo se coloca al frente del nombre del atributo.

Se debe definir el tipo de visibilidad al crear los atributos, caso contrario se definen automáticamente como privados. El valor por defecto hace referencia al valor que toma el atributo si no se indica lo contrario. Este valor se declara después del tipo del atributo y el símbolo (=).

Esta información se enuncia de la siguiente forma:

- Visibilidad Nombre Atributo: Tipo Dato = ValorPorDefecto

Se debe tomar en cuenta los siguientes aspectos sobre los métodos: Nombre del método.

- La visibilidad del método, es similar a la de los atributos igualmente el símbolo se coloca antes del nombre del método.
- La lista de parámetros que recibe el método se especifica entre paréntesis y lleva la siguiente información.
- Nombre del parámetro, Tipo de dato y el valor por defecto del parámetro.

La información relativa a cada uno de los parámetros se escribirá de esta forma:

Nombre Parámetro: TipoDato = ValorPorDefecto, es decir el tipo de dato correspondiente al dato devuelto por el método.

La información referente a cada método adoptará la siguiente forma:

Visibilidad Nombre Método (Nombre Parámetro$_1$: TipoDato$_1$ = Valor Por Defecto1

Nombre Parámetro$_2$: TipoDato$_2$ = ValorPorDefecto, ...): TipoDatoDevuelto

Al desarrollar los diagramas de clase no es necesario indicar el tipo de método al inicio, este puede definirse más adelante en las fases de análisis y diseño de software.

La clase Libro se puede simbolizar de algunas formas. Anteriormente, se ha venido indicando que se debe definir para cada atributo su nombre, visibilidad, tipo de dato y valor por defecto. [20]

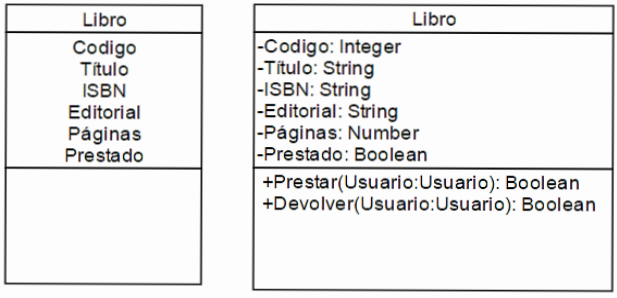


Figura. 57: Dos representaciones de la clase libro.

4.2.3. Relaciones entre clases en UML

Tal como en las entidades se suelen crean relaciones en los diagramas Entidad Relación, de igual forma se crean relaciones entre clases en los diagramas de clases. Existen cuatro tipos de relaciones entre clases: asociación agregación, composición, generalización y dependencia.

4.2.3.1. Asociación

A una relación normal entre las clases se la denomina asociación. Esta suele ser reflexiva si existe una relación de clase consigo misma, binaria si relaciona dos clases, ternaria si relaciona tres clases, y así sucesivamente.

Las relaciones reflexivas y las binarias utilizan como símbolo una línea para unir las asociaciones. Si la relación es de nivel superior a dos, se simboliza gráficamente por medio de un rombo quien se encarga de unir a las clases asociadas en la relación.

Las relaciones tienen un nombre, pero no de forma necesaria y se detalla sobre la línea que simboliza la relación o interiormente del rombo si la relación es de nivel superior a dos. Se dibuja una flecha en un extremo de la asociación, lo que indica en qué sentido hay que leerla.

En toda relación se debe mostrar las cardinalidades que le correspondan. Las cardinalidades tienen el mismo significado en los diagramas de clase que en los diagramas Entidad Relación, pero no se representan de la misma forma. Las posibles cardinalidades son:

*: Cero o varios.

1…*: Uno o varios

n…m: De n a m.

También se debe especificar en una relación los roles que desempeñan las clases participantes en la relación. Cada rol especifica con una o diversas palabras al lado de las clases participantes en la relación y al lado de la línea que le corresponde a la asociación.

Con la figura 58 se mostrará una asociación binaria entre las clases Cliente y Cuenta Bancaria, con el cual un cliente puede ser dueño de una o diversas cuentas bancarias, y una cuenta bancaria puede tener uno o muchas inscripciones. Como se observa, no se ha establecido nombre a la asociación, pues es opcional.

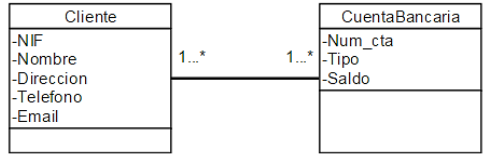


Figura. 58: Asociación binaria sin nombre.

La figura 58 muestra una asociación reflexiva con el nombre “es jefe de”, por lo que un practicante puede tener o no un jefe y, a la vez, un practicante puede ser jefe de varios empleados o de ninguno. Como se ve, a esta agrupación se le ha establecido un nombre y se han detallado roles.

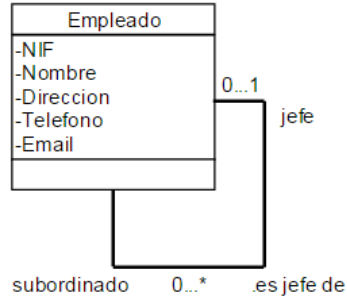


Figura. 59: Asociación reflexiva con nombres y con valores.

En la figura 60 se mostrará una asociación ternaria entre las clases Cliente, Proveedor y Producto. Esta asociación nos muestra qué bienes surte un distribuidor a qué clientes.

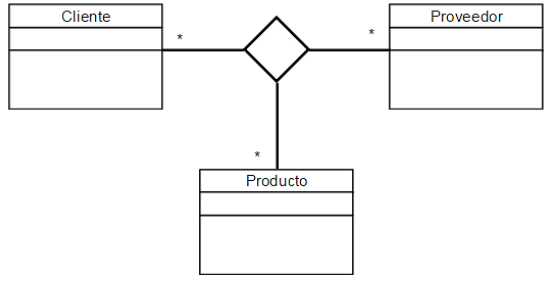


Figura. 60. Asociación Ternaria.

Las asociaciones poseen atributos y métodos, dando lugar a las llamadas clases asociativas. Las clases asociativas se simbolizan como unas clases cualquieras que se acopla con la línea que personaliza la asociación con una línea de trazado discontinuo.

La figura 61 indica una clase asociativa nombrada Línea Pedido, la que indica cada encargo y artículo pedido en él, la cantidad requerida de ese artículo en ese encargo.

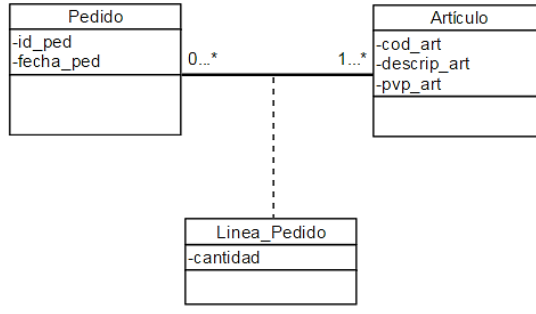


Figura. 61: Clase Asociativa.

4.2.3.2. Agregación

La agregación es un tipo especial de asociación que personifica un objeto mezclado por distintos objetos. Se trata de una relación entre un todo (agregado) y las porciones que lo disponen (componentes). Los componentes alcanzan a existir sin el añadido o, los componentes que se hallan anteriormente al elaborar el añadido e inclusive consecutivamente de esfumarse el añadido.

Una agregación se figura mediante una línea que realiza la unión con

el agregado y cada uno de sus componentes, y con un rombo con fondo blanco al lado del agregado.

En la figura 62 se representa un ejemplo de agregación en el cual el agregado Auto se compone de un motor y cuatro llantas. Las llantas y el motor pueden existir antes de que exista el auto como tal y pueden existir después de que el auto como tal haya desaparecido.

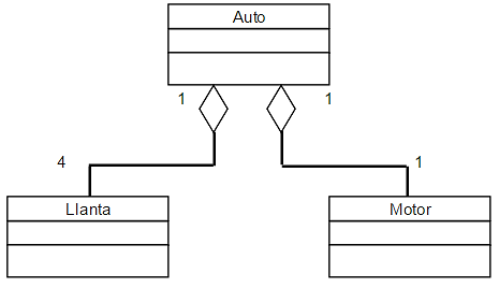


Figura. 62. Agregación

4.2.3.3. Composición

Una composición entre clases se facilita una relación más grande que una agregación. Naturalmente, asimismo presta el nombre de agregación fuerte.

Los componentes se realizan en el mismo momento que el agregado y se destruyen al momento de destruir el agregado. Es decir, un componente solo debe pertenecer a un agregado.

Una composición se figura por una línea que forma la unión con el añadido y cada mecanismo y con un polígono con base negra al lado del añadido.

En la figura 63 se muestra ejemplo de composición entre la clase Ventana de un entorno de ventanas y sus componentes: título, menú y contenido. En este caso se trata de una composición porque los componentes no tienen existencia propia ni pueden pertenecer a otro agregado y, además, nacen y mueren con su agregado.

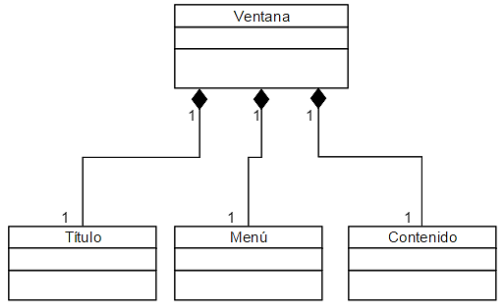


Figura. 63: Composición.

4.2.3.4. Generalización

Se define a una generalización como la relación que se da entre diferentes clases por la que consta una clase llamada superclase que sujetan los atributos y métodos frecuentes a varias clases, que tienen el nombre de subclases. De esta manera las subclases se sistematizan en una superclase y la superclase se domina en otras subclases (especialización).

Se considera a la herencia como una propiedad de las subclases que además de tener sus atributos y métodos propios, constituyen una parte de los atributos y métodos de su superclase.

Se representa a una generalización con la unión de la subclase con la superclase por una línea y trazando un triángulo con fondo blanco al lado de la superclase.

En la figura 64 se mostrará la especialización de la clase Vehículo en las subclases Coche y Camión. Vemos como la superclase tiene dos atributos (Matrícula y Precio), que son los comunes a las clases Coche y Camión, cada una de las cuales posee un atributo específico: Plazas y Tonelaje.

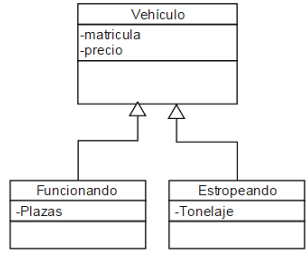


Figura. 64: Generalización.

En una jerarquía de clase y sus clases puede haber más de un nivel como en la siguiente figura.

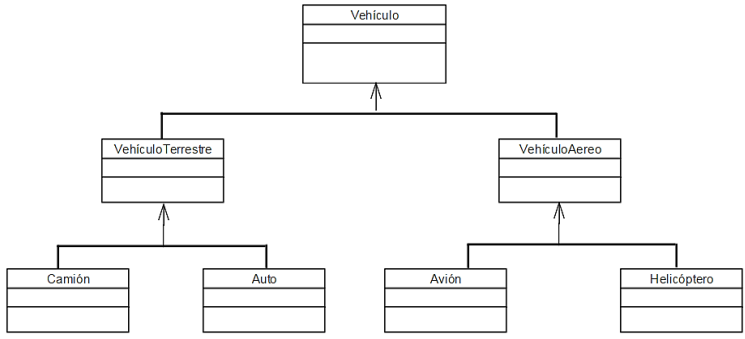


Figura. 65: Generalización de dos niveles.

Existen diferentes tipos de especializaciones:

- Disjuntas o no disjuntas: Una especialización es disjunta si un objeto solo debe pertenecer a una sola subclase; no disjunta, será lo inverso a la disjunta.
- Totales (completas) o parciales (incompletas): Una especialidad es de tipo total si las subclases establecen todos los casos que se puedan de la superclase; arbitraria, en caso inverso.
- Estática o dinámica: Es cuando la especialidad es estática, un objeto es solicitud a partir de su fabricación de la subclase y la superclase en la que ha fue creado. La especialidad es dinámica, un objeto suele marcharse a lo extenso de su vida entre las diferentes subclases de la especialidad.

En la figura 66 se muestra una especialización disjunta porque un auto en concreto solo puede estar funcionando o estropeado, total porque no existirán autos que no se puedan clasificar en los dos tipos indicados (funcionando o estropeado) y dinámica porque un auto en concreto, a lo largo de su existencia, puede pasar de estar funcionando a estar estropeado y viceversa.

Existe herencia simple o herencia múltiple. En una herencia simple, cada subclase deriva de una sola superclase. En caso de ser herencia múltiple, una subclase puede proceder de varias subclases. También, en la jerarquía de clases de la figura 67 poseemos herencia múltiple porque las subclases Camión y Bote proceden cada una de dos superclases.

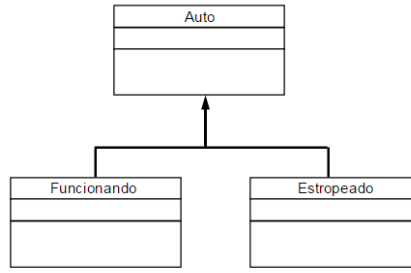


Figura. 66: Especialización disjunta, total y dinámica

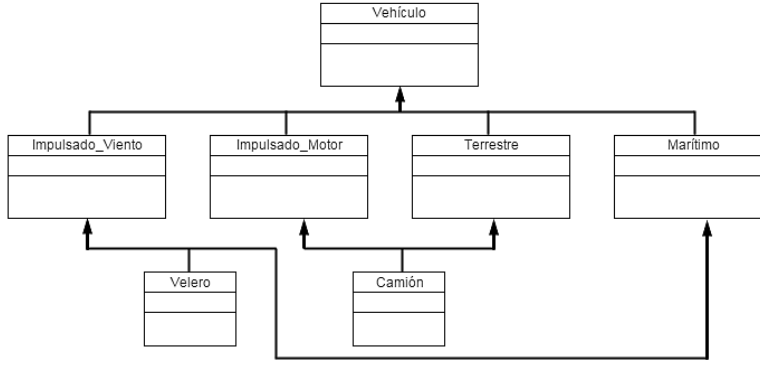


Figura. 67: Herencia Múltiple.

4.2.3.5. Dependencia

Se define a la dependencia como la relación que se da por dos clases la cual una de las clases se instancia a otra clase, es decir que una depende de otra o que una clase solicita de otra para que pueda trabajar. Por ejemplo, en un entorno de ventanas una aplicación gráfica instancia una ventana.

La relación de dependencia se simboliza mediante una flecha con diseño interrumpido que empieza a partir de la clase dependiente.

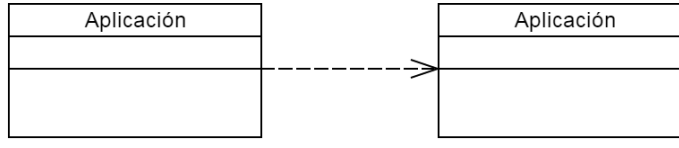


Figura. 68: Relación de Dependencia

4.3. Comparación del modelo de clases con el modelo Entidad-Relación

La analogía que se ha logrado establecer entre el modelo Entidad-Relación y el modelo de clases es la siguiente:

- Dentro de los diagramas Entidad-Relación las entidades se convierten en clases dentro de los diagramas de clases.
- Los atributos que se hallan en las entidades de los diagramas Entidad-Relación se transformaran en atributos de las clases adecuadas.
- De igual manera como los diagramas ER es justo escoger cierto atributo como AIP para cada entidad, en los diagramas de clases no es necesario. Esto se debe a que cada uno de los objetos de una clase en una aplicación orientada a objetos, tiene señalado un identificador que puede referirse a él sin problema a medida que se va elaborando la aplicación.
- Las relaciones normales en los diagramas ER van a ser asociaciones en los diagramas de clases.
- En los diagramas ER las relaciones frágiles (dependencias en existencia y en identificación) se transforman en agregaciones o disposiciones en los diagramas de clases, dependiendo de si son similares el tiempo de vida de los dispositivos con el del añadido.
- Es preciso asociar a cada relación una cardinalidad. La manera de definir las cardinalidades es igual en los dos tipos de diagramas, sin embargo, se usan una simbología desigual.
- En los diagramas ER los atributos de relaciones producen clases asociativas en los diagramas de clases. A estas clases se les fijara un nombre y tendrán como atributos los que formarían parte a las relaciones de los diagramas ER.
- En los diagramas ER las jerarquías de tipos y subtipos corresponden a las jerarquías de clases en los diagramas de clases. [20]

4.4. Diagrama de objetos como caso especial del diagrama de clases

Se considera un caso especial de los diagramas de clase a los diagra-

mas de objetos. En estos diagramas se muestran objetos, es decir las instancias de clases en un momento en el tiempo.
Los objetos en los diagramas de objetos se personifican de manera resumida o de forma específica:

De manera resumida los objetos se simbolizan dentro de en un rectángulo colocando en su interior el nombre del objeto marcado o solo el nombre del objeto, el símbolo de dos puntos (:) y el nombre de la clase a la que el objeto es.
En la figura 69 se mostrará un objeto de la clase Cliente:

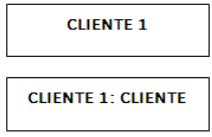


Figura. 69: Objeto de la clase cliente

De manera más especificada un objeto se puede representar mediante un rectángulo con dos cortes. En el corte superior se pone subrayado el nombre del objeto, el símbolo de dos puntos (:) y por siguiente el nombre de la clase. En la parte inferior se pone sus atributos y para cada uno de ellos después del nombre del atributo se escribe el símbolo de igual (=) y el valor que obtiene el atributo para ese objeto. A continuación, se simboliza un objeto de la clase Cliente de modo detallado en la figura 70.

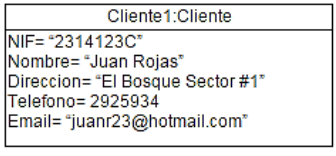


Figura. 70: Representación detallada de un objeto de la clase cliente

Un diagrama de objetos para el diagrama de clases tendría las cuentas bancarias de que ubican uno o varios clientes en un instante en específico de realización de la aplicación. Como, por ejemplo, en la figura 71 se simboliza un diagrama de objetos en el que se muestra el hecho de que el cliente con NIE 12345678A tiene dos cuentas bancarias. [21]

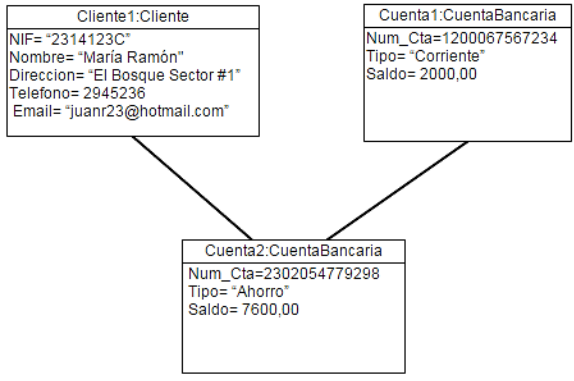


Figura. 71: Diagrama de objetos.

Resumen

- El modelo orientado a objeto nace como parte de las metodologías de análisis, en los años 80 del diseño orientado a objetos, en el que iniciaron su camino durante las épocas pasadas.
- Los datos del modelo orientado a objetos son considerados como un incremento del modelo Entidad-Relación.
- Se define a un objeto como una entidad que permite mostrar un procedimiento preciso
- Cuando usamos la notación UML para los diagramas de clase, se representa una clase con un rectángulo divido en tres secciones.
- Tal como en las entidades se suelen crean relaciones en los diagramas Entidad Relación, de igual forma se crean relaciones entre clases en los diagramas de clases.
- A una relación normal entre las clases se la denomina asociación. Esta suele ser reflexiva si existe una relación de clase consigo misma, binaria si relaciona dos clases, ternaria si relaciona tres clases, y así sucesivamente.
- La agregación es un tipo especial de asociación que personifica un objeto mezclado por distintos objetos.
- Una composición entre clases facilita una relación más grande que una agregación. Naturalmente, asimismo presta el nombre de agregación fuerte.
- Se define a una generalización como la relación que se da entre diferentes clases por la que consta una clase llamada superclase que sujetan los atributos y métodos frecuentes a varias clases, que tienen el nombre de subclases.

- Se considera un caso especial de los diagramas de clase a los diagramas de objetos. En estos diagramas se muestran objetos, es decir las instancias de clases en un momento en el tiempo

Ejercicios propuestos y resueltos

Ejercicios Resueltos

TEMA: Análisis del modelo relacional y los elementos que lo integran.

1. Esquema relacional con información de las personas que forman parte de un directorio.

Directorio (Nombre, FecNac, Ciudad, Celular, CodPro)
Provincias (CodPro, Pro)

Directorio				
Nombre	**FecNac**	**Ciudad**	**Teléfono**	**CodPro**
Rosa Cheve	19/10/85	Machala	0938563142	10
Karla Feijoo	10/01/95	Guayaquil	0979290223	32
Ligia Sánchez	11/02/90	Manta	0997177423	25
Joel Cabrera	02/08/80	Cuenca	0988505376	45

Tabla 32: Datos de un Directorio

Provincias	
CodPro	**Pro**
32	Guayas
45	Azuay
10	El Oro

Tabla 33: Código de Provincia

Para las consultas es necesario mostrar el resultado de cada una y escribir las expresiones del álgebra y cálculo relacional de tuplas y el cálculo de relacional de dominios.

a).- Obtener las ciudades de las personas del directorio.

b).- Obtener las personas que viven en Guayaquil.

c).- Obtener el nombre y la fecha de nacimiento de las personas que tienen menos de 28 años.

d).- Obtener nombre, ciudad y provincia de las personas mayores de 35 años.
e).- Obtener nombre, fecha de nacimiento y teléfono de las personas de la Provincia de El Oro.

Solución

a).- πciudad(Directorio)
t|∃s Є Directorio (t.Ciudad=s.Ciudad))
b).- αciudad=Guayaquil (Directori0)
t| (t Є Directorio) ∧ (t.Ciudad="Guayaquil"))
<c>|∃n, f, t, cp (<n, f, c, t, c p> Є Directorio)
< n, f, c, t, cp >|(<n, a, c, t, c p> Є Directorio∧|="Guayaquil")
c).- πNombre, Fec_Nac (αFec_Nac<28(Directori0))
t|∃s Є Directorio (t.Nombre=s.Nombre) ∧(t.FecNac=s.FecNac) ∧ cNac<28))

Directorio				
Nombre	**FecNac**	**Ciudad**	**Telefono**	**CodPro**
Karla Feijoo	10/01/95	Guayaquil	0979290323	32

Tabla 34 Registro de Directorio

<n, f >∃ c, t, cp (< n, f, c, t, cp> Є Directorio ∧ f< 28

Nombre	**FecNac**
Karla Feijoo	10/01/95
Ligia Sánchez	11/02/90

Tabla 35: Nombre - Fecha de un Directorio

Ciudad
Machala
Guayaquil
Manta
Cuenca

Tabla 36: Tabla Ciudad

2. Se presenta algunos condicionales prácticos de normalización que son más que la relación universal y se usará las formas normales de la primera a la tercera en secuencia, dando lugar a un esquema relacional con más relaciones y normalizado hasta la tercera forma normal. También se presenta ejercicios propuestas de des normalización.

Solicitud de un pedido en una empresa. Se va a registrar los datos de cada solicitud en la base de datos:
Fecha: 28 de Mayo del 2014
Núm. Pedido: 555555
Núm. Proveedor: 1990
Nombre Proveedor: Ecotel
Dirección Proveedor: Marcel Laniado y 10 de agosto, Machala

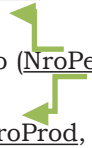
NumPro	Detalle	PreUni	Cantidad	Importe
1102	Celular	60,00	2	120,00
1103	Lámpara de mesa	30,00	3	90.00
1104	Mueble desarmable	150,00	1	150,00
TOTAL				360,00

Tabla 37: Datos de un Encargo

Solución:
La relación universal es que con el grupo repetitivo que se encuentra subrayado como clave principal el atributo NPed:
Pedido (NroPed, FPed, NroPro, NPro, DPro, NroProd, DetalleProd, PUnit, Can, Impor, Total)
La relación pasa a otra forma normal suprimiendo el grupo repetitivo y con una nueva relación con los atributos del grupo repetitivo y la llave principal de la relación.
Pedido (NroPed, FPed, NroPro, NPro, DPro, Total)

Línea Pedido (NroPed, NroProd, DetProd, PUnit, Can, Impor)
Se necesita que la relación Línea Pedido este en la segunda forma normal, y que todos sus atributos no claves dependan de la clave.
(NroPed, NroPro) → DetProd
(NroPed, NroPro) → PUnit
(NroPed, NroPro) → Cant
(NroPed, NroPro) → Impor
Se supone que los valores de los productos son únicos, se entregan con el mismo valor independientemente de a quien se realice la venta. Tenemos que las 2 primeras dependencias no son correctas ya que un producto y su valor no dependen del pedido.
NroPro -> DetProd
NroProd->PUnit

Los atributos DetProdu y Punit no dependen de la clave porque la relación no está en segunda forma normal. Para pasar a esta forma se deberá eliminar los atributos de la relación y crear una nueva relación. Solución
to (NroProd) Crearemos una nueva relación con ellos más el atributo del que dependen. El esquema relacional en forma normal quedará de la siguiente forma:
Pedido (NroPed, FPed, NroPro, NPro, DPro, Total)



Línea Pedido (NroPed, NroProd, DetProd, PUnit, Can, Impor)

Producto (NroProd, DetProd, PUnit)
Si las relaciones están en tercera forma normal tratando de buscar dependencias transitivas, en la relación Pedido tenemos una dependencia transitiva:
NroPed -* NroPro—+ NPro, DPro
Los atributos NPro y DPro no dependen de la clave (NroPed), sino del atributo NroPro.
La relación pedida pasará a tercera forma normal, lo primero será eliminar los atributos que no dependen de la clave y se creará una relación por cada uno de ellos. El esquema quedará de la siguiente forma.
Pedido (NroPed, FPed, NroPro, Total)
Proveedor (NroPro, NPro, DPro)
Línea Pedido (NroPed, NroProd, Can, Impor)
Producto (NroProd, DetProd, PUnit)
Propuestas de desnormalización:

Generalmente al mostrar datos de un pedido y haciendo referencia a la tabla proveedor no proporciona actualizaciones, por lo que es recomendable relacionar Proveedor en la relación Pedido y eliminar la tabla proveedor.

Si obtenemos información de una línea de pedido es imprescindible consultar datos del producto.
Registro de matrícula colegial.
Se presenta una relación universal y las siguientes hipótesis semánticas: Matrícula (ID, Códmateria, Ape, Nom, Nota, Curso, Aula, Lugar)
Hipótesis semánticas:

- El atributo Codmateria guarda las materias en la cual están

matriculados los estudiantes.

- El atributo Aula es el lugar en donde se dictará la docencia de todas las materias, entretanto que Lugar se refiere al sitio del centro en el que se encuentra el aula (el piso, edificio, entre otros.) y estas aulas poseen un código que identifican el sitio en el que se ubica el aula.
- La materia, asociada con la docencia de un solo curso.
- Una materia puede distribuirse a cualquier aula, pero dependiendo el grupo que integre.
- El estudiante matriculado en diferentes materias debe registrar una única nota.

Solución:

En este ejercicio la clave principal será el ID del estudiante, lo que permite mover del orden de los atributos en la relación. Algunos atributos del grupo repetitivo están subrayados, ya que un estudiante puede matricularse en varias materias.

Matrícula (ID, Nom, Apellidos, Codmateria, Nota, Curso, Aula, Lugar)

Debido al grupo repetitivo, la relación no está en 1era forma normal

Ya que existe un grupo repetitivo, la relación no está en primera forma normal. La llevamos a esta forma normal descartando el grupo repetitivo de la relación y estableciendo una nueva. La estructura relacional en primera forma normal se vería de la siguiente manera:

Registro_Matricula (ID, Nom, Apellidos)

Documentación (ID, Codmateria, Nota, Curso, Aula, Lugar)

Para estar en segunda forma normal, las relaciones de documentación necesitan que sus atributos no claves dependen totalmente de la clave, obligando así a cumplir las dependencias funciones totales.

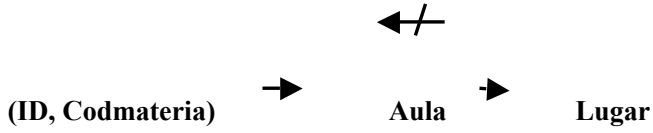
La relación Documentación para que se halle en segunda forma normal es necesario que sus atributos no clave dependan de la totalidad de la clave, por lo tanto, deben cumplir las dependencias funcionales totales a continuación.

(ID, Codmateria) Nota
(ID, Codmateria) Curso
(ID, Codmateria) Aula
(ID, Codmateria) Lugar

Al depender una materia a un único curso la segunda dependencia funcional total no se cumple, entonces tenemos que:
Codmateria -* Curso
Para llevar a segunda forma normal la relación documentación descarta el atributo curso y debe establecer una nueva relación con él y el atributo del que depende. Siendo de la siguiente manera el esquema de segunda forma normal:
Registro_Matricula' (ID, Nombre, Apellidos)

Documentación (ID, Codmateria, Nota, Aula, Lugar)
Documentación (ID, Codmateria, Nota, Aula, Lugar)

Materia (Codmateria, Curso)
La relación materia ya se encuentra en tercera forma normal, y se debe buscar alguna dependencia funcional transitiva. La relación de documentación se halla:



Entendemos que la relación aula necesita de una clave principal. Para llevar la relación documentación a tercera forma normal se descarta el atributo lugar y se crea una nueva relación. El esquema en tercera forma normal será el siguiente:
Matrícula (ID, Nombre, Apellidos)
Documentación (ID, Codmateria, Nota, Nota, Aula)
Ubicación (Aula, Lugar)
Materia (Codmateria, Curso)

Propuestas de desnormalización:
Cuando se presentan los datos de documentación de un estudiante se precisa conocer casi siempre el curso al que pertenece cada materia, y teniendo en cuenta que este dato cambiará con poca frecuencia, se podría proponer añadir el atributo Curso a la tabla Documentación y eliminar la tabla Materia. Podríamos obrar de manera similar si al consultar el Documentación de un estudiante fuese necesario casi siempre

conocer el lugar en el que recibe docencia: eliminaríamos en este caso la tabla Ubicación y colocaríamos el atributo Lugar en la tabla Documentación.

Empleados

Relación universal e hipótesis semánticas:

Empleado (NroEp, NSS, Sección, NroJefeSec, NroCurso, Tema)

Hipótesis semánticas:

- El empleado posee un número único.
- El empleado se ocupa de una sola sección. Esta sección contiene un empleado jefe.

Un empleado debe hacer muchos cursos, y estos tienen un código que lo identifica contiene un solo tema.

Solución:

Se ha colocado como llave principal a NroEp, ya que un Empleado puede hacer varios cursos cada uno con su tema.

Empleado (NroEp, NSS, Sección, NroJefeSec, NroCurso, Tema)

Con la presencia de un grupo repetitivo la relación empleada está en la primera forma normal, y debemos cambiar la forma adecuada descartando el grupo repetitivo creando una nueva relación con los atributos del grupo más una clave principal. Entonces el esquema relacional en primera forma normal sería de la siguiente manera.

Empleado (NroEp, NSS, Sección, NroJefeSec)



Estudios (NroEp, NroCurso, Tema)

La relación estudios no se encuentra en segunda forma normal, y para cambiarla el atributo no clave deberá formar parte de la totalidad de las claves, debiendo ser cierta la siguiente dependencia funcional:

(NroEp, NroCurso) =>Tema

Esta dependencia no es correcta, ya que el tema del curso está asociado con el curso en el que se trate mas no de quien lo haga. Cumpliéndose así la siguiente dependencia.

NroCurso— Tema

Es por esta razón que la relación Estudios debe pasar a segunda forma normal y para ello debemos eliminar de dicha relación el atributo Tema y crear una nueva relación con aquel atributo y del cual depende. El esquema relacional en segunda forma normal será el siguiente:

Empleado' (NroEp, NSS, Sección, NroJefeSec)



Empleado (NroEp, NroCurso)



Cursos (NroCurso, Tema)

Ya se encuentran en forma normal las relaciones Estudios y Curso, debido a q están en segunda forma normal y poseen menos de 2 atributos no clave. La relación Empleado no se encuentra en tercera forma normal, mostrando la siguiente dependencia funcional:



NroEp -> Sección -> NroJefeSec

El atributo NroJefeSec no depende de la clave NroEp, sino del atributo sección. Para pasar a la tercera forma normal debemos eliminar de la relación Empleado el atributo NroJefeSc y crear una nueva relación con aquel atributo y del cual depende. El esquema relacional en tercera forma normal quedará como sigue:

Empleado (NroEp, NSS, Sección)
Sección (Sección, NroJefeSec)
Empleado (NroEp, NroCurso)
Cursos (NroCurso, Tema)

Propuestas de desnormalización:

Al conocer la información de un trabajador es necesario saber de quién se trata si es un jefe, se puede desmoralizar agregando el atributo NroJefeSec a la tabla Empleado y desasociando la relación con la tabla Secciones.

La información de los cursos que han sido hechos por cada trabajador y conocer el tema que trata, si la información no varía mucho se puede eliminar la tabla cursos y en la tabla Estudios el atributo Tema.

Tema: Descripción y aplicación del modelo entidad-relación para el modelado de datos

Referente a este tema se mostrarán varios ejercicios para modelar modelos entidad – relación. Se explica el paso a paso para obtener un modelo entidad- relación y a partir de este poder transformar al esquema relacional sin olvidar cada una de las reglas de transformación vistas anteriormente.

Empleados y departamentos

Para los empleados se necesita almacenar información como un número que lo identifique, nombre, apellido, salario y comisión. Cada uno de estos empleados tiene designado un departamento y se almacenará un número de departamento, nombre y localidad. También es necesario almacenar la relación entre empleados y conocer si tiene designado un jefe o no.

Solución:

En la entidad Empleado como atributos tenemos: un número, su nombre, apellido, salario y comisión. Como atributo identificador principal tomaremos el atributo número ya que para cada empleado es el identificador único.

En la entidad Departamento como atributos tenemos: un número, nombre y localidad. Como atributos identificadores principal tomaremos el atributo número ya que para cada Departamento el número será único.

Para crear las relaciones debemos saber por cada empleado el departamento en donde trabaja, para ello creamos la relación entre Empleado y Departamento con cardinalidad 1,1 junto al Departamento, ya que todo empleado trabaja en un solo departamento; y la cardinalidad 1, n junto al Empleado, ya que en cada departamento trabajará como mínimo un empleado y como máximo varios.

Otra relación seria entre los empleados y conocer cuál es su jefe, para ello se crea una relación reflexiva en la entidad Empleado. Como cardinalidad tendremos 1, N, con la que sabremos cual es el jefe de cada empleado. Si es que lo tiene o no. La otra cardinalidad es 0,N ya que un empleado puede o no tener subordinados.

El diagrama entidad - relación que se propone es el siguiente:

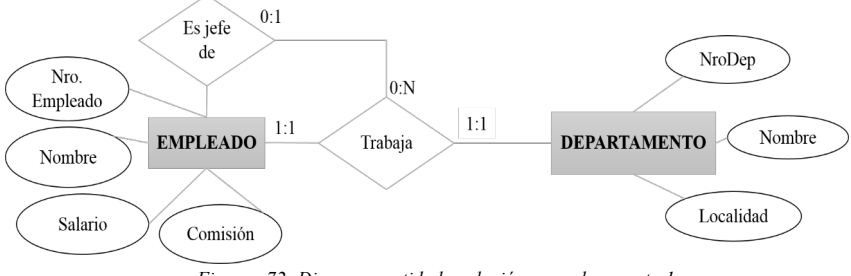
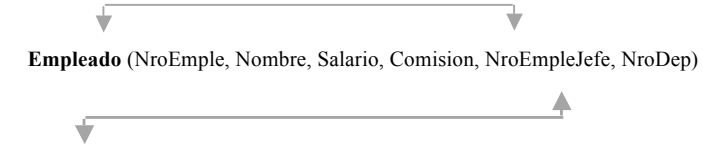


Figura. 72: Diagrama entidad - relación para el supuesto 1

Para obtener el esquema relacional, cada entidad se transformará en una relación en este caso las entidades de Empleado y Departamento con cada uno de sus atributos.
Se analizarán cada una de las relaciones del diagrama entidad - relación:

La relación Trabaja es de cardinalidad 1,N, dando lugar a la cardinalidad N aparezca una clave secundaria con el atributo de clave principal de la otra relación NroDep. El método que se debe aplicar es crear una nueva tabla y creamos en la tabla Empleado una clave secundaria con el atributo NroDep.
El esquema relacional resultante es el siguiente:



Departamento (NroDep, Nombre, Localidad)

Tema: Modelo orientado a objeto

Se proponen varios ejercicios para la modelización de diagramas de clases. Por cada ejercicio se proporciona el enunciado que describe como se desea obtener el diagrama de clase.

Sistema de monitorización de sensores.

- Para un sistema de monitorización de sensores se desea crear un diagrama de clases con métodos para la monitorización. Este sistema además de monitorizar también deberá indicar problemas.
- Como atributos del sensor tenemos fabricante, número de modelo, secuencia de iniciación, factor de escala y unidad de medida, intervalo de muestreo, ubicación, estado, valor actual y umbral de alarma.
- El sistema hace seguimiento de cada sensor ubicados en edificios. Como atributos del edificio tenemos dirección y número de contacto.
- Una característica de los sensores críticos es la tolerancia.
- Para que el sistema se active debe superarse el umbral de cada sensor. A los dispositivos de alarma se los identifica de acuerdo

a su funcionamiento: el estado del dispositivo y la duración de la alarma.

- Al provocarse una alarma el sistema deberá almacenar la fecha, hora, severidad, tiempo de reparación y estado.
- La monitorización de los sensores se da desde que se inicia hasta que se apaga, el intervalo de muestreo toma una muestra, que la convierte utilizando el factor escala y la unidad de medida. Esta muestra se la compara con el umbral, para de esta manera ver si iguala o supera para poder crear la alarma y activar el dispositivo.
-

Solución:

Para la creación del diagrama de clase primeramente creamos la clase Sensor con cada uno de sus atributos. Como se conoce existe otro tipo de sensores con un atributo propio que lo identifica, para ellos debemos crear una subclase con el nombre Sensor Crítico con el atributo tolerancia, pero debido a la herencia entre estas clases además de sus propios atributos tendrá los atributos de la clase Sensor.

Creamos otra nueva clase que será Edificio, porque es necesario conocer la ubicación de cada sensor, se establecerá la relación entre un sensor y Edificio.

Para tener información acerca de los sucesos de alarma es necesario crear una clase que almacena esta información: fecha, hora, severidad, tiempo de reparación y estado. Por cada sensor puede o no tener sucesos de alarma, y la asociación se daría entre Sensor y sucesos.

Para controlar la activación de cada dispositivo se crea la clase Dispositivo Alarma con los atributos Estado y Duración. Se establecerá la relación entre Suceso y Dispositivo Alarma.

Los métodos que se darán a las clases serán:

Iniciar(), método que será invocado cuando el sensor deberá funcionar.

Monitoriza(), método que calculará el valor del umbral sino se encuentra dicho valor se producirá el suceso de alarma.

Activar(), método de la clase Dispositivo Alarma.

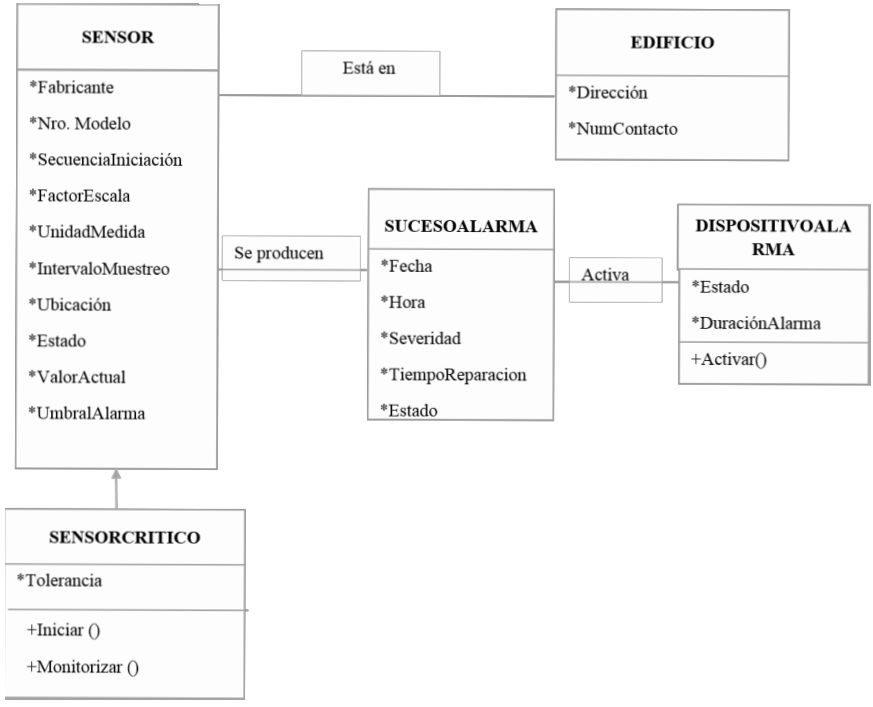


Figura. 73: Diagrama de sistema de monitorización de sensores

Ejercicios Propuestos

TEMA: Análisis del modelo relacional y los elementos que lo integran.

1. Se proporciona el siguiente esquema relacional:

Pedido(RefPed, FPed)
Línea Pedido (RefPed, CodArt, Can, Impor)
Articulo (CodArt, DetProd, PUnit)

Pedido	
Pedido	**FecPed**
E010	05/01/2014
E011	10/01/2014
E012	15/01/2014
E013	25/01/2014

Tabla 38: Tabla pedido

Linea Pedido		
RefPed	**CodArt**	**CantArt**
E010	A0043	10
E011	A0078	12
E012	A0043	5
E013	A0075	20
E010	A0012	15
E011	A0043	5
E012	A0089	50

Tabla 39: Tabla línea de pedido

Articulo		
Cod Art	**DesArt**	**PVPArt**
A0043	Esfero negro fino	0,60
A0078	Esfero verde normal	0,45
A0075	Cuaderno espiral a cuadros x100 hojas	1,50
A0012	Tijera	1,00
A0089	Lápices de colores	1,25

Tabla 40: Tabla Artículo

Por cada consulta indicar el resultado y escribir las expresiones del algebra relacional, cálculo relacional de tuplas y cálculo relacional de dominios:

a).- Adquirir artículos de precio inferior a 0,25 C.

b).- Adquirir la descripción y el precio de los artículos con precio superior a 1€.

c).- Adquirir la descripción de los artículos y las unidades solicitadas en el pedido P0004.

d).- Adquirir por cada pedido realizado con posterioridad al 20 de febrero de 2013, la fecha del pedido, y por cada uno de los artículos solicitados en él, su descripción, el precio del articulo y el número de unidades solicitadas.

2. Se proporciona varios ejercicios prácticos para la normalización en los cuales se deberán aplicar las formas normales desde la primera forma normal hasta la tercera forma normal, para mostrar un esquema relacional con muchas más relaciones y normalizado.

2.1. Liga de futbol

Relación universal:

Liga (ON!, Nombre jugador, Fecha nacimiento, Sueldo, Nacionalidades, idiomas, Nombre equipo, Ciudad, Presidente, Código partido, Fecha,

Resultado)

Hipótesis semánticas:

- Un jugador puede tener una o varias nacionalidades.
- Un jugador puede hablar uno o varios idiomas.
- Un jugador pertenecerá a un solo equipo.
- Un equipo solo pertenecerá a una ciudad con un único presidente.
- Un código de partido es solo los partidos que ha jugado dicho jugador.

2.2. Almacén

Relación universal:

Almacén (con Tienda, Nombre jinda, Código Ciudad, Nombre Ciudad, Provincia, Con Autónoma, Código Art, Nombre Art, Cantidad Art, PVP Art, IVA Art)

Hipótesis semánticas:

- Los artículos se identifican por un código y nombre.
- La mercancía Llevada depende de las necesidades locales.
- La cantidad y precio unitario de un artículo difiere de una tienda a otra depende siendo de la demanda local.
- Cada artículo posee el mismo IVA.
-

2.3. Cursos

Relación universal:

Cursos (Código Curso, Nombre Curso, Dm Estudiante, Nombre Estudiante, Numero Matricula, Centro, Profesor, Texto)

Hipótesis semánticas:

- El identificador único de un curso es su código.
- Un estudiante puede estar matriculado en varios cursos.
- Un estudiante posee números de matrícula distinta respecto a cada curso matriculado.
- Un curso se imparte en un solo centro.
- Un curso se imparte por un solo profesor.
- Un profesor puede impartir varios cursos.
- Un profesor imparte clases en un solo centro.
- Un curso se apoya en distintos textos, y un mismo texto puede servir para varios cursos.

- Tema: Descripción y aplicación del modelo entidad-relación para el modelado de datos
- Referente a este tema se mostrarán varios ejercicios para modelar modelos entidad – relación. Se explica el paso a paso para obtener un modelo entidad - relación y a partir de este poder transformar al esquema relacional sin olvidar cada una de las reglas de transformación vistas anteriormente.

Tema: Descripción y aplicación del modelo entidad-relación para el modelado de datos

Referente a este tema se mostrarán varios ejercicios para modelar modelos entidad – relación. Se explica el paso a paso para obtener un modelo entidad - relación y a partir de este poder transformar al esquema relacional sin olvidar cada una de las reglas de transformación vistas anteriormente.

Liga de futbol

La Liga de Fútbol Profesional (LFP) requiere implementar una base de datos con estadísticas de la temporada.

- Los jugadores serán identificados por un número de ficha, nombre, apellido, fecha de nacimiento, peso y estatura.
- Los equipos se identificarán por su nombre, año de fundación, nombre del presidente, número de socio y estadio.
- Un jugador puede participar en varios equipos a lo largo de su carrera, pero no al mismo tiempo.
- Del contrato que se realiza entre un club y un jugador, se desea almacenar fecha de comienzo, duración, ficha anual y cláusula de rescisión.
- De los partidos jugados se almacenará un código, fecha, resultado y la jornada.
- Todos los jugadores pueden o no participar en todos los partidos, siendo importante sus minutos jugados, goles anotados y tarjetas recibidas.
- De los árbitros se requiere información del número de colegiado, nombre del colegio arbitral, nombre, apellido y numero de temporadas.
- En cada partido hay cuatro árbitros siendo importante almacenar la función que desempeña cada uno

Supermercado

En un supermercado existen varios proveedores, la información que se desea guardar es su nombre, dirección y teléfono.

- Todo proveedor puede suministrar uno o varios artículos a la vez, la información del artículo que se requiere almacenar es el nombre, precio, número y el precio tanto del proveedor como el precio de venta.
- Todo supermercado está estructurado por departamentos, la información que se requiere es el código, director y la serie de empleados. Cada uno de estos departamentos es responsable de un área específica y cada departamento puede vender una clase de artículo.
- De los empleados que trabajan en un supermercado, la información que se necesita almacenar es el nombre, dirección, teléfono, salario y departamento.
- De los clientes que tiene el supermercado, la información que se necesita almacenar es el nombre, dirección, teléfono y saldo.
- Cada cliente realiza un pedido la información a almacenar es el número del pedido, fecha, artículos y cantidad.

Tema: Modelo orientado a objeto.

Se proponen varios ejercicios para la modelización de diagramas de clases. Por cada ejercicio se proporciona el enunciado que describe como se desea obtener el diagrama de clase.

Biblioteca

Una biblioteca tiene copias de libros.

- Los libros poseen un título, año y autor o autores.
- Los autores se caracterizan por su nombre y nacionalidad.
- Un libro puede ser escrito por uno o varios autores.
- Una copia de libro posee un identificador.
- Un libro puede estar en una biblioteca por motivos de préstamos, retraso o en reparación.
- Se almacena información acerca de los préstamos actuales y antiguos.
- De un lector se almacena su ID, nombre, apellidos y dirección.

- Cada libro se presta un máximo de 30 días.
- Si un libro no es devuelto en la fecha límite, se sanciona con dos días sin posibilidad de hacer préstamos de un nuevo libro.

Obtenga el diagrama de clases con atributos para cada clase.

Entidad bancaria

Se desea sistematizar las operaciones de un banco del cual se necesita conocer nombre, OF y dirección. Y de la cual se necesita obtener:

- Lista de sucursales: código de la sucursal, dirección, celular.
- Lista de cuentas corrientes: número de la cuenta, NIF del titular, nombre y apellidos, celular, dirección y saldo.
- Lista de operaciones realizables: código de la operación y descripción.
- Lista de recibos domiciliados: número de recibo, número de cuenta de pago, importe del recibo, entidad emisora (empresa que emite el recibo) y fecha.

Las reglas del sistema son las siguientes:

- El banco posee varias sucursales.
- Cada sucursal posee varias cuentas corrientes.
- Una cuenta corriente puede asociar a uno o más clientes, es decir varios clientes con la misma cuenta.
- Un cliente puede tener varias cuentas, y realizar operaciones distintas en cada una de ellas.
- Cada cuenta puede o no tener uno o varios recibos domiciliados.

Obtener el diagrama de clases indicando para cada clase su nombre y los atributos.

Puerto de Santurce

Se desea conocer y analizar el control de tráfico del puerto de Santurce. Del puerto los datos a almacenar son: el número, la denominación y la razón social.

- Del muelle los datos a almacenar son: número, nombre descriptivo asociado y longitud variable entre 30 y 3000 metros.
- En este puerto transitan barcos de carga y de transbordo, dentro de los de carga hay varios tipos como pesqueros y comerciales.

- Los barcos pesqueros se dedican a la pesca de sardina, de este tipo de barco se necesita almacenar el nombre del barco, el número de licencia de pesca, la empresa propietaria, la longitud del barco en metros y la capacidad total de carga en toneladas.
- De los buques petroleros la información que se necesita almacenar es: nombre, país, empresa propietaria, eslora, la capacidad total de carga en toneladas y la densidad del petróleo.
- De los barcos mercantes la información que se necesita almacenar es: nombre, país, empresa, eslora, capacidad total de carga y el factor de peligrosidad de la mercancía transportada.
- Existe una línea de transbordadores y se debe almacenar el nombre, la eslora, el número de pasajeros y el número de vehículos.
- Cada barco puede realizar operaciones y de cada operación se debe almacenar nombre del barco, eslora, fecha y hora de la operación. Si la operación es de entrada, se almacenará el tiempo de estancia aproximado, y si operación de salida se almacenará el destino del barco y las observaciones que sean necesarias.
- De la tripulación se almacenará ID, nombre, apellidos y sueldo. Si es el capitán, también se requiere la antigüedad.
- Para realizar una operación el barco debe solicitar al capitán del barco.
- Para realizar una operación de entrada, se necesita conocer si hay espacio en el muelle.
- El espacio del muelle debe ser actualizada cada vez que se una operación de entrada o salida.

Se pide obtener el diagrama de clases indicando para cada clase su nombre, los atributos y los métodos.

Glosario

Agregación: La agregación permite crear a partir de más de dos entidades y una relación que las asocia entre sí, una entidad de nivel mucho más alto.

Asociación: Es aquella relación normal que existe entre las clases. Estas suelen ser: reflexiva si existe una relación de clase consigo misma, binaria si relaciona dos clases, ternaria si relaciona tres clases, y así sucesivamente.

Atributo: Son las características de una entidad y son representados de varias maneras; ya sea con un óvalo y el nombre del respectivo atributo en su interior puede ser incorporado directamente con la entidad o con la relación.

Composición: Una composición entre clases resulta ser una relación más grande que una agregación.

Dependencia: Es aquella relación que se da entre dos clases, la cual una de las clases se instancia a otra clase. Es decir, que una depende de otra o que una clase solicita de otra para que pueda trabajar.

Entidad: Se define a una entidad como un objeto del cual se desea almacenar información en la base de datos.

Herencia: Es un tipo de asociación entre una entidad padre y una entidad hijo y se representa por medio de un triángulo inverso que se conecta con las entidades mediante una línea.

Relación: Es la correspondencia entre entidades y es representada por medio de un rombo con el respectivo nombre en su interior.

Bibliografía

[1] A. Silberschatz, H. F. Korth y S. Sudarshan, Fundamentos de Bases de Datos, España: MacGraw Hill, 2002.

[2] C. J. Date, Introducción a los Sistemas de Bases de Datos, México: Pearson Educación, 2001.

[3] M. J. Ramos Martín, A. Ramos Martín y F. Montero Rodriguez, Sistemas Gestores de Bases de Datos, España: McGraw Hill, 2006.

[4] M. Marqués, «Sapientia,» 2011. [En línea]. Available: http://www.uji.es/bin/publ/edicions/bdatos.pdf. [Último acceso: 2016].

[5] J. Sánchez, «jorgesanchez.net,» 2004. [En línea]. Available: http://www.jorgesanchez.net/bd/bdrelacional.pdf.

[6] J. Sanchez, «Principios sobre Bases de Datos Relacionales,» 2004. [En línea]. Available: http://www.jorgesanchez.net/bd/.

[7] J. Sanchez, «Diseño Conceptual de Base de datos,» 2004. [En línea]. Available: http://www.jorgesanchez.net/bd/.

[8] D. M. KROENKE, Procesamiento de bases de datos: fundamentos, diseño e implementación., MÉXICO: Cámara Nacional de la Industria Editorial Mexicana, 2003.

[9] I. M. Jareño, El Modelo Entidad- Relación CASE-METHOD, Wilmington: Días de Santos S.A, 1994.

[10] M. V. N. Cabello, Introducción a la Base de Datos Relacionales, Madrid: Visión Libros, 2010.

[11] V. F. Alarcón, Desarrollo de sistemas de información: una metodología basada en el modelado., Catalunya: Edicions UPC, 2010.

[12] Universidad de Sevilla, «Modelo Entidad-Relación Extendido,» 02 2007. [En línea]. Available: http://www.lsi.us.es/docencia/get.

php?id=1864.

[13] D. C. Costa, Introducción al Diseño de Bases de Datos, Editorial UOC.

[14] A. C. Yera, Diseño y Programación de bases de Datos, Madrid: Visión Libros.

[15] J. M. Vara, «Transformación de Modelos para el Desarrollo de Bases de Datos Objeto-Relacionales,» 07 2007. [En línea]. Available: http://www.ewh.ieee.org/reg/9/etrans/ieee/issues/vol05/vol5issue4July2007/5TLA4_09Vara.pdf.

[16] M. C. Pabón, «Gestion y Modelación de Datos Diseño de BD - Modelo Entidad Relación,» 06 2011. [En línea]. Available: http://cic.javerianacali.edu.co/wiki/lib/exe/fetch.php?media=materias:bd1:3_mer.pdf.

[17] M. A. Rodriguez, «Amazon S3,» 2014. [En línea]. Available: https://s3.amazonaws.com/piazza-resources/hyiw1nus1o81pv/i0dww1j7zt36sw/Normalizacion.pdf?AWSAccessKeyId=AKIAIEDNRLJ4AZKBW6HA&Expires=1454108791&Signature=irOD1NHntyNLUcYHjqkfM6oscUE%3D. [Último acceso: 28 01 2016].

[18] J. M. Cotos Yáñez y J. Á. Taboada González, Sistemas de información medioambiental, España: Netbiblo, 2005.

[19] M. Y. Jimenez Capel, IFCT0310: Bases de datos relacionales y modelado de datos., Malaga: IC Editorial, 2015.

[20] J. M. Piñeiro Gomez, Base de datos relacionales y modelos de datos, España: Paraninfo, 2013.

[21] J. M. Peñiero Gómez, Lenguajes de definicion y modificacion de datos SQ, España: Ediciones Paraninfo, S.A..

Fundamentos de base de datos
Se terminó de imprimir en marzo de 2016 en la
imprenta de la UTMACH, calle Loja y 25 de Junio
(campus Machala)
Esta edición consta de 300 ejemplares.

www.utmachala.edu.ec

El programa de Reingeniería del Conocimiento en la Universidad Técnica de Machala (UTMACH) es un modelo emergente de gestión de la investigación que promueve saberes científicos con pertinencia social. Desde el Vicerrectorado Académico impulsamos la investigación colectivista, donde docentes y estudiantes se engranan en la construcción y divulgación del resultado de sus ejercicios pedagógicos, heurísticos y de vinculación social, en aras de contribuir con el fortalecimiento de nuestras ventajas comparativas y competitivas a nivel transfronterizo.

Mediante este programa estratégico la UTMACH impacta sus imaginarios respecto a la relación de la docencia con la investigación, muestra de ello es la presente obra donde se cristaliza el empoderamiento y profesionalismo de sus actores y redes al servicio de la formación crítica de profesionales de avanzada.

En la UTMACH seguimos conquistando el conocimiento a través de la investigación, por ello en cada acción emprendida ***proyectamos nuestra historia***.

Ing. Amarilis Borja Herrera, Mg. Sc.
VICERRECTORA ACADÉMICA.

REINGENIERÍA DE LA INVESTIGACIÓN

UTMACH

ISBN: 978-9942-24-074-3